

Desarrollo de una aplicación web full-stack con Angular, Node.js y SQLite

José Vicente Berná Martínez

Profesor Titular de Universidad

Dpto. Tecnología Informática y Computación



Universitat d'Alacant
Universidad de Alicante



Grau en Enginyeria Multimèdia
Grado en Ingeniería Multimedia

Índice de contenidos

1. Preamble	6
2. Conexión a BD y CRUD usuarios	6
2.1. IMPORTANTE: Instalación de Node.js, necesario para instalar las dependencias y librerías del proyecto	6
2.2. Inicio del proyecto	7
2.3. Nodemon y script	9
2.4. Primera ruta GET, Postman, CORS y DOTENV	10
2.4.1. Postman	11
2.4.2. CORS.....	14
2.4.3. ENV	17
2.5. Configuración de SQLite	18
2.6. Descarga Código 1.....	20
2.7. Estructuración del proyecto: CONTROLLERS y ROUTERS	20
2.8. Crear modelos: MODELS.....	22
2.8.1. Visualización y gestión de la base de datos con DB Browser for SQLite.....	24
2.8.2. Selección de campos específicos en la consulta	26
2.9. Ruta POST y validación argumentos	27
2.9.1. Validación de argumentos.....	32
2.9.2. Middleware personalizado VALIDATEERRORS.....	34
2.10. 1.8. Otras validaciones y rutas PUT y DELETE	35
2.11. POST: comprobar email, cifrar contraseña y almacenar	40
2.11.1. Comprobar email	40
2.11.4. Cifrar la contraseña	42
2.12. Ocultación de campos sensibles en las respuestas JSON.....	44
2.13. Descarga Código 2.....	48
2.14. Middleware comprobar ROL, PUT y DELETE	48
2.14.1. Comprobación de ROL	48
2.14.2. Implementación del método PUT	49
2.14.5. Implementación de DELETE	54
2.15. Descarga Código 3.....	56
3. Login, JWT y paginación	56

3.1.	2.1. Login.....	56
3.2.	Generar JWT	59
3.2.1.	¿Qué es un JSON Web Token (JWT)?	59
3.2.2.	Implementación del JWT	60
3.2.3.	Clave secreta para generar el token	62
3.2.4.	Token de renovación y control de sesión única	62
3.2.5.	Guardar el token en la base de datos	63
3.2.6.	Integración en el login	63
3.2.7.	Endpoint y controlador de renovación	64
3.2.8.	Middleware validate-jwt.js	65
3.2.9.	Pruebas	66
3.2.10.	Descarga Código 4	69
3.3.	Paginación y Promise.all	69
3.3.1.	Descarga Código 5	72
4.	Inicio del frontend	72
4.1.	Inicio del proyecto	72
4.2.	Diseño basado en componentes	75
4.3.	Primeros componentes	75
4.3.1.	Estructura del proyecto	76
4.3.2.	Anatomía de un componente	76
4.3.4.	Creación de componentes	77
4.4.	3.4. Componentes hijos de auth y admin	78
4.4.1.	Zona de administración	80
4.5.	3.5. Arreglando rutas	83
4.6.	Descarga Código 6	84
4.7.	Estructuración del código	84
4.8.	Descarga Código 7	85
5.	Formato mediante plantilla y login	86
5.1.	Plantilla matrix	86
5.2.	Fragmentación de la plantilla principal (dashboard)	86
5.2.1.	Configuración global (src/index.html)	87
5.2.2.	Componentización de elementos comunes (navbar, sidebar, footer)	87

5.2.3.	Creación del componente breadcrumb.....	88
5.2.4.	Ensamblaje en el contenedor admin-layout.....	88
5.2.5.	Solución problema con la fuente de los iconos	89
5.3.	Descarga Código 8.....	89
5.4.	Fragmentación de la plantilla de autenticación (login)	90
5.4.1.	Ensamblaje en el contenedor AuthLayout	90
5.4.2.	Extracción del componente login	90
5.4.3.	Ajuste de dimensiones del contenedor principal (footer).....	91
5.5.	Routerlink	91
5.5.1.	Limpieza del resto de /commons	92
5.6.	Arrancar backend y crear JWT	94
5.6.1.	Generación de un token comodín para pruebas	94
5.7.	Validación formulario - Template vs Reactivo	95
5.7.1.	Formularios basados en plantillas (Template-driven)	95
5.7.2.	Formularios Reactivos (Reactive forms).....	96
5.8.	Login y formularios reactivos	96
5.8.1.	Configuración del controlador (login.ts)	96
5.8.2.	Enlace del formulario en la vista (login.html)	97
5.8.3.	Validación del formulario	98
5.8.5.	Feedback visual de validación en tiempo real	99
5.8.7.	Inserción de los mensajes en la vista (login.html)	100
5.8.8.	Alternativa para la evaluación del estado global	103
5.9.	Login y servicio user.....	103
5.9.1.	Creación del servicio user.....	103
5.9.2.	Configuración global del cliente HTTP	104
5.9.3.	Inyección del servicio en el componente login	104
5.9.4.	Invocación de la API y manejo de respuestas	105
5.10.	Descarga Código 9.....	107
5.11.	Interfaces.....	107
5.11.1.	Creación de la interfaz LoginForm	108
5.11.2.	Aplicación de la interfaz en el servicio	108
5.12.	Login - almacenar token, acceso o error.....	109
5.12.1.	Lógica de negocio del Login.....	109

5.12.2.	Librería para alertas visuales	109
5.12.3.	Implementación en el controlador (login.ts)	109
6.	5. Securizar frontend	111
6.1.	Backend llamada validar token	111
6.1.1.	Mitigación de vulnerabilidad en tokens huérfanos	111
6.1.2.	Implementación del controlador de renovación	112
6.1.3.	Protección de rutas y renovación del token en el frontend	113
6.1.5.	Automatización de la renovación del token	114
6.2.	Zona dashboard - creando guard	118
6.2.1.	Creación de la guarda de autenticación	118
6.2.2.	Aplicación de la guarda en el router	118
6.2.3.	Lógica de autorización en el guard	119
6.3.	Zona auth - añadir guard	120
6.3.1.	Control directo desde el componente (login.ts)	120
6.3.2.	Control mediante el guard	122
6.4.	Optimización del código	122
6.5.	Logout	124
6.5.1.	Ruta de logout en la BD	125
6.5.2.	Implementar el logout a nuestra interfaz	126
6.6.	Limpieza de tokens	127
6.6.1.	Validación en UserService	127
6.7.	Errores claros al usuario	128
7.	6. Sidebar dinámico por roles	129
7.1.1.	Creación de SidebarService	129
7.1.2.	Adaptación del componente Sidebar	130
7.1.3.	Integración con el ciclo de autenticación	131
7.2.	Página de gestión de usuarios	133
7.2.1.	Adaptación del Backend: Búsqueda, ordenación y contraseñas	133
7.2.2.	7.2. Ampliación del servicio UserService	136
7.2.3.	7.3. Creación del componente paginador	137
7.2.4.	7.4. Componente de formulario (UserForm)	138
7.2.5.	7.5. Integración en la vista principal de Users	141

8.	8. Mensajes dinámicos en el dashboard.....	146
8.1.	Descarga Código 10.....	147

1. Preambulo

El objetivo de este manual es sentar las bases del desarrollo de una app con arquitectura Sigle Page que consta de un front y un back.

Para la capa del servidor en este proyecto (backend) utilizaremos Node.js junto con la librería Express para la creación de la API HTTP. La persistencia de datos la gestionaremos mediante SQLite, una base de datos relacional que almacena la información en un archivo local del sistema, eliminando la necesidad de configurar servidores de bases de datos independientes. En la capa del cliente (frontend), utilizaremos Angular para ofrecer una interfaz de usuario interactiva y estructurada en componentes.

El desarrollo de esta aplicación se dividirá en varias etapas.

2. Conexión a BD y CRUD usuarios

2.1. IMPORTANTE: Instalación de Node.js, necesario para instalar las dependencias y librerías del proyecto

Antes de empezar con el proyecto, necesitamos instalar Node.js, para ello accedemos a la página oficial de Node.js: <https://nodejs.org/es/download>

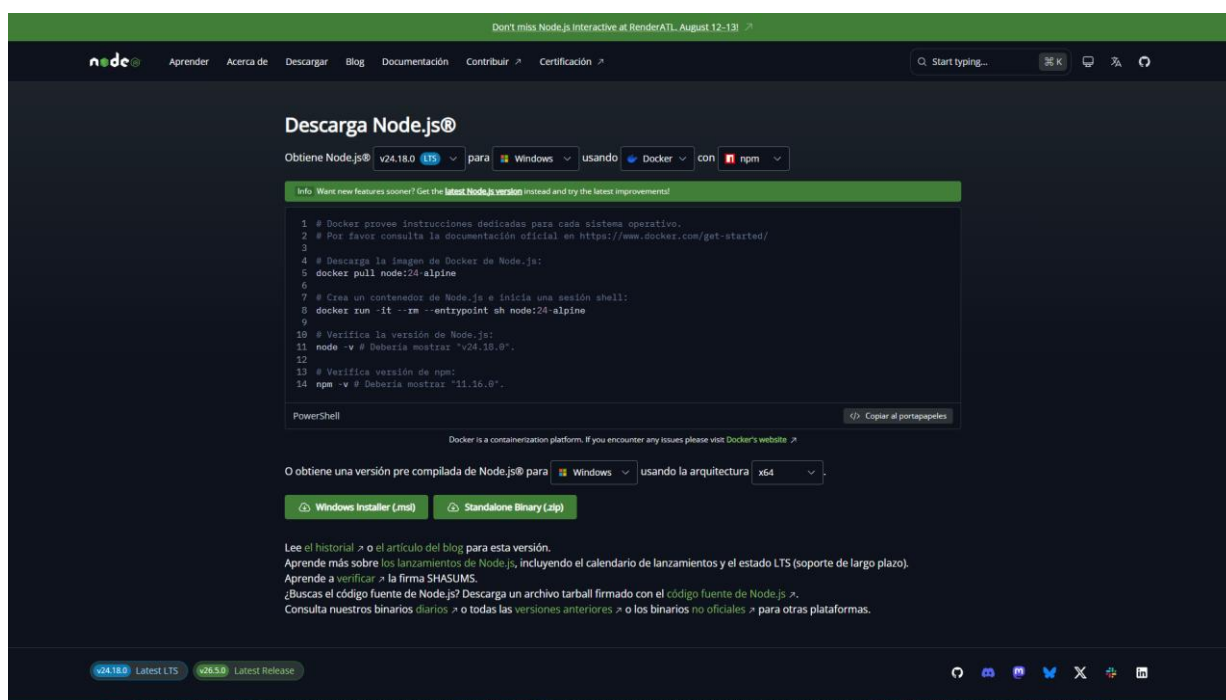


Figura 1. Portal de descarga de NodeJS.

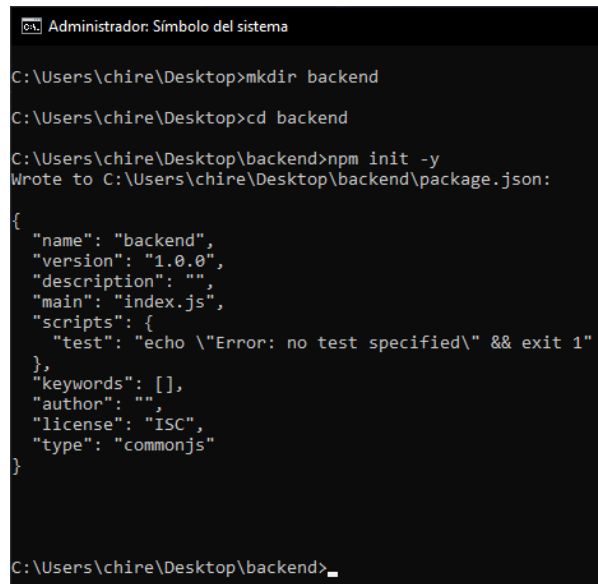
Seleccionamos nuestro sistema operativo y la arquitectura de este (64 bits o 32) y descargamos el instalador, en el caso de Windows un archivo .msi.

Después solamente tendríamos que ejecutar este archivo e instalarlo.

2.2. Inicio del proyecto

Para comenzar el desarrollo del servidor, crearemos la carpeta de trabajo e inicializaremos el entorno de paquetes de Node.js. Esto generará un archivo llamado **package.json**, el cual se encargará de registrar las dependencias y la configuración general del proyecto.

Ejecutaremos en la terminal los siguientes comandos (realizaremos estas acciones en la ubicación donde queramos crear la carpeta del backend):



```
Administrador: Símbolo del sistema

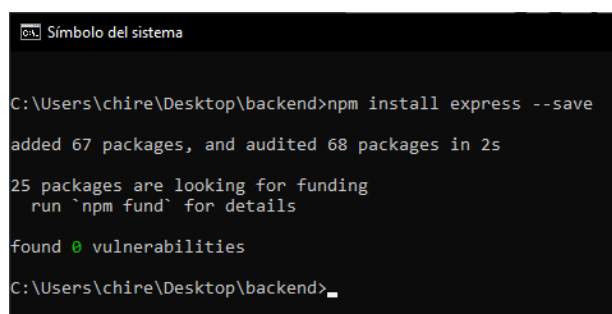
C:\Users\chire\Desktop>mkdir backend
C:\Users\chire\Desktop>cd backend
C:\Users\chire\Desktop\backend>npm init -y
Wrote to C:\Users\chire\Desktop\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Figura 2. Creación de package.json.

En el desarrollo de software, una **dependencia** es un paquete de código creado por otros desarrolladores que podemos descargar e integrar en nuestro proyecto para no tener que escribir todas las funciones desde cero.

Instalaremos Express ejecutando el siguiente comando, que debería de producir un resultado semejante a este:



```
Símbolo del sistema

C:\Users\chire\Desktop\backend>npm install express --save
added 67 packages, and audited 68 packages in 2s

25 packages are looking for funding
  run `npm fund` for details

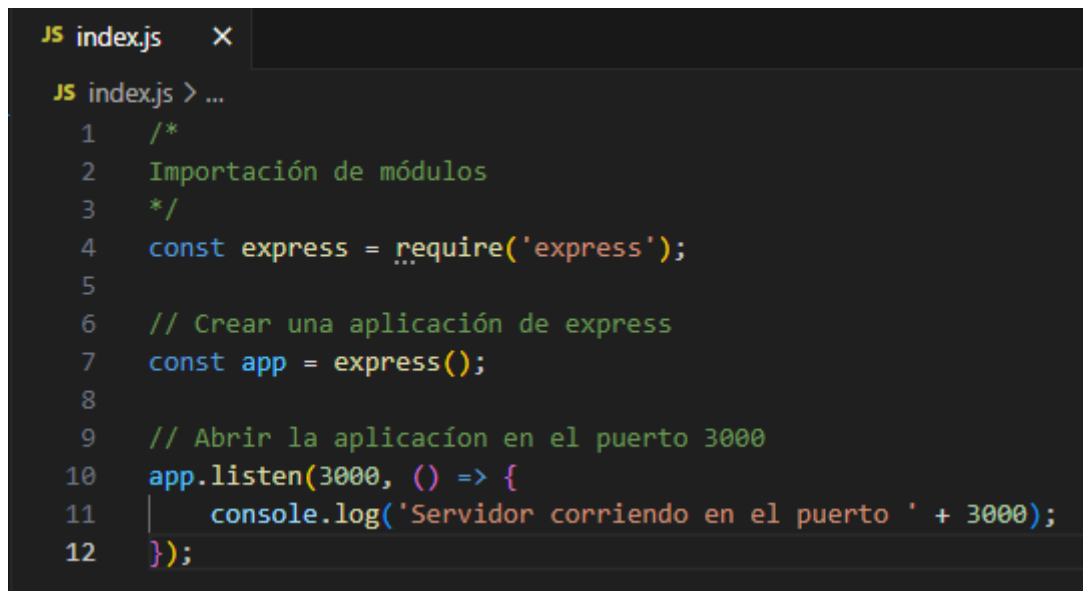
found 0 vulnerabilities

C:\Users\chire\Desktop\backend>
```

Figura 3. Instalación de Express.

Al realizar este paso, se crea automáticamente una carpeta llamada `node_modules` (donde se guarda el código descargado de Express) y un archivo `package-lock.json` (que registra las versiones exactas instaladas para evitar fallos futuros).

Ahora creamos en la raíz de nuestro proyecto el archivo base de nuestro proyecto, `index.js`. En este archivo vamos a importar el módulo de express y utilizarlo para crear una aplicación que escuche un puerto al cual podremos lanzar peticiones. Para ello escribimos el siguiente código en el archivo `index.js`:



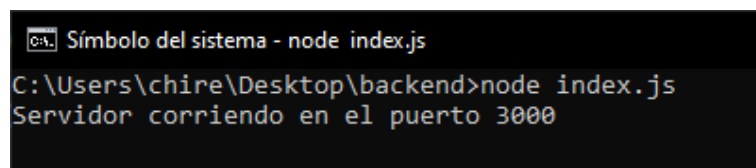
```
JS index.js X
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5
6  // Crear una aplicación de express
7  const app = express();
8
9  // Abrir la aplicación en el puerto 3000
10 app.listen(3000, () => {
11   console.log('Servidor corriendo en el puerto ' + 3000);
12 });
```

Figura 4. Archivo index.js base.

Ahora ya podemos arrancar nuestro servidor por primera vez. Para ejecutar un programa javascript utilizando node lanzamos el comando:

```
node index.js
```

Veremos en la ventana del terminal el mensaje de que el servidor está en ejecución:



```
Símbolo del sistema - node index.js
C:\Users\chire\Desktop\backend>node index.js
Servidor corriendo en el puerto 3000
```

Figura 5. Salida en la terminal de la ejecución de index.js.

Y si abrimos un navegador web y cargamos al URL <http://localhost:3000> podremos ver lo siguiente:

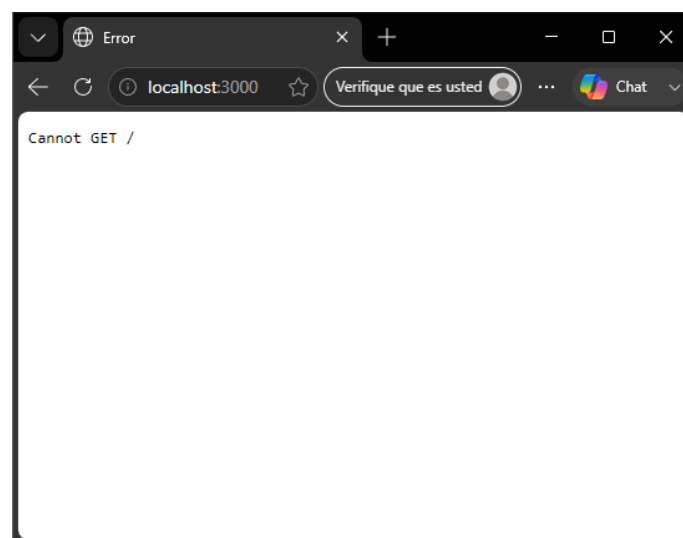


Figura 6. Resultado devuelto al navegador.

Este resultado indica que hay una aplicación escuchando en el puerto 3000. El mensaje "Cannot GET /"

muestra que ese programa no sabe que hacer con la solicitud de "get /", pero nuestro programa está funcionando. Si paramos la ejecución de nuestra aplicación, mediante "Ctrl+C" y tratamos de cargar la misma URL que antes, veremos que ahora el navegador muestra un error como el siguiente:

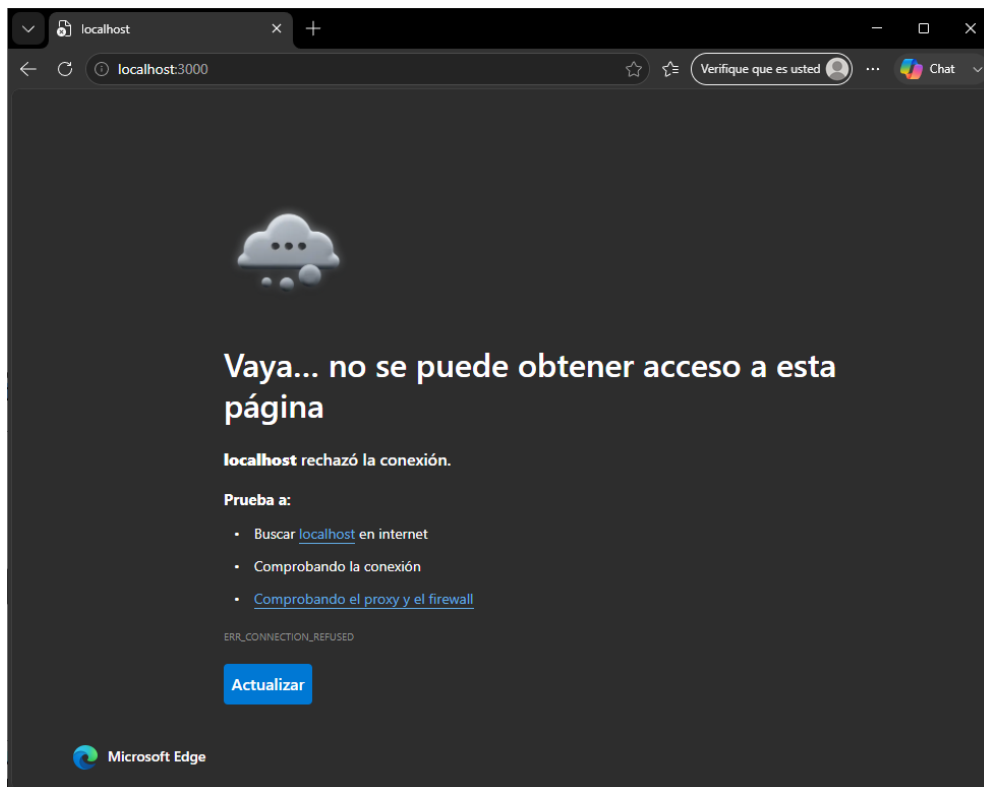


Figura 7. Resultado devuelto al navegador cuando la aplicación no está ejecutándose.

2.3. Nodemon y script

Ahora mismo tal y como tenemos el proyecto, cada vez que hacemos un cambio tenemos que detener el comando "node index.js" y volver a arrancar para que se actualice la ejecución con los cambios que hagamos hecho al proyecto. Por ejemplo, si cambiamos el puerto del 3000 al 3005, tenemos que parar y arrancar el proyecto.

Para evitar esto, vamos a instalar un paquete llamado nodemon que automatizará el proceso de actualizar la ejecución con los cambios.

Desde la consola (por ejemplo la consola de VSC) instalamos el paquete nodemon:

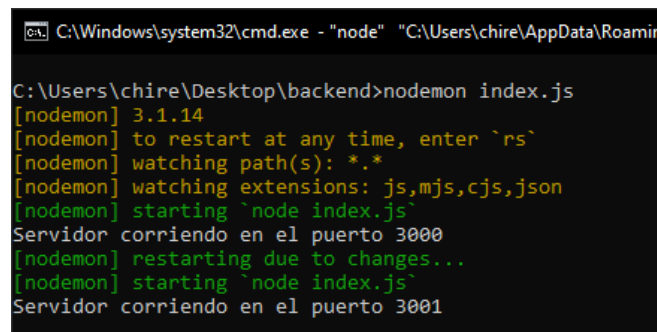
```
node index.js
```

Ahora para ejecutar el programa en lugar de "node index.js" lanzamos "nodemon index.js" y en la consola aparecerá algo como esto:

```
C:\Windows\system32\cmd.exe - "node" "C:\Users\chire\AppData\Roaming\npm\nodemon\nodemon.exe"
C:\Users\chire\Desktop\backend>nodemon index.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Servidor corriendo en el puerto 3000
```

Figura 8. Resultado en la terminal tras ejecutar nodemon.

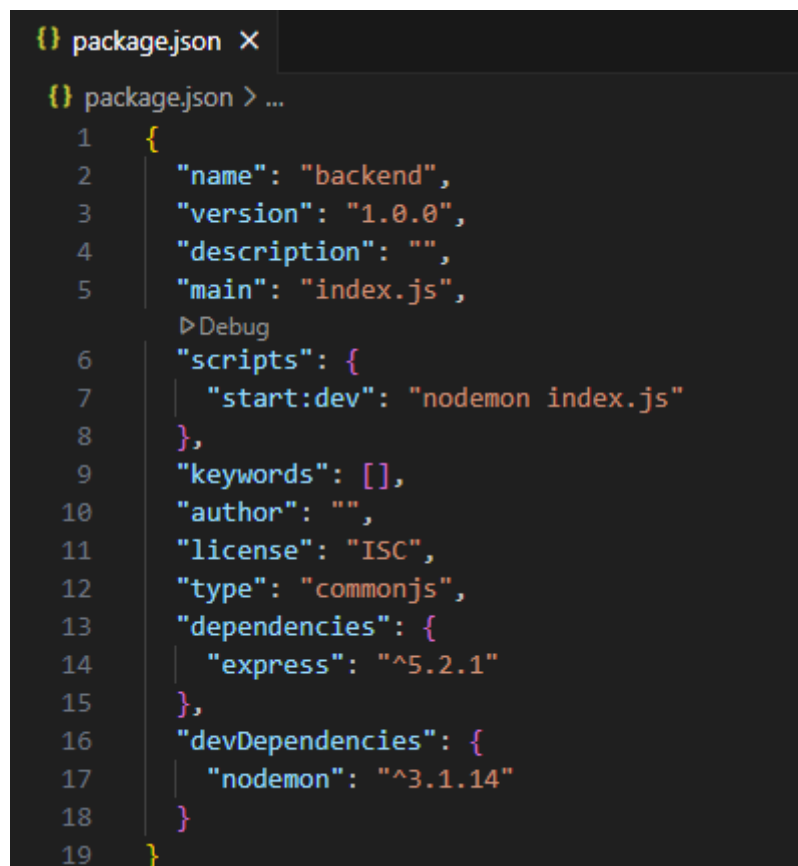
Esto tiene el mismo efecto que el anterior "node index.js" pero con una ventaja, si cambiamos en el código por ejemplo el puerto 3000 por 3001, al guardar el archivo veremos que la consola lanza un mensaje de reinicio:



```
C:\Windows\system32\cmd.exe - "node" "C:\Users\chire\AppData\Roamin...
C:\Users\chire\Desktop\backend>nodemon index.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Servidor corriendo en el puerto 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Servidor corriendo en el puerto 3001
```

Figura 9. Recarga de index.js automática tras guardar el archivo.

Esto nos permite trabajar en nuestro proyecto sin tener que estar parando y arrancando constantemente la ejecución, ya que nodemon se encargará de observar si hay cambios en algún archivo y hacerlo por nosotros. Otra forma de agilizar es utilizando scripts en el package.json de la siguiente forma:



```
{ package.json X
{ package.json > ...
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start:dev": "nodemon index.js"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs",
13   "dependencies": {
14     "express": "^5.2.1"
15   },
16   "devDependencies": {
17     "nodemon": "^3.1.14"
18   }
19 }
```

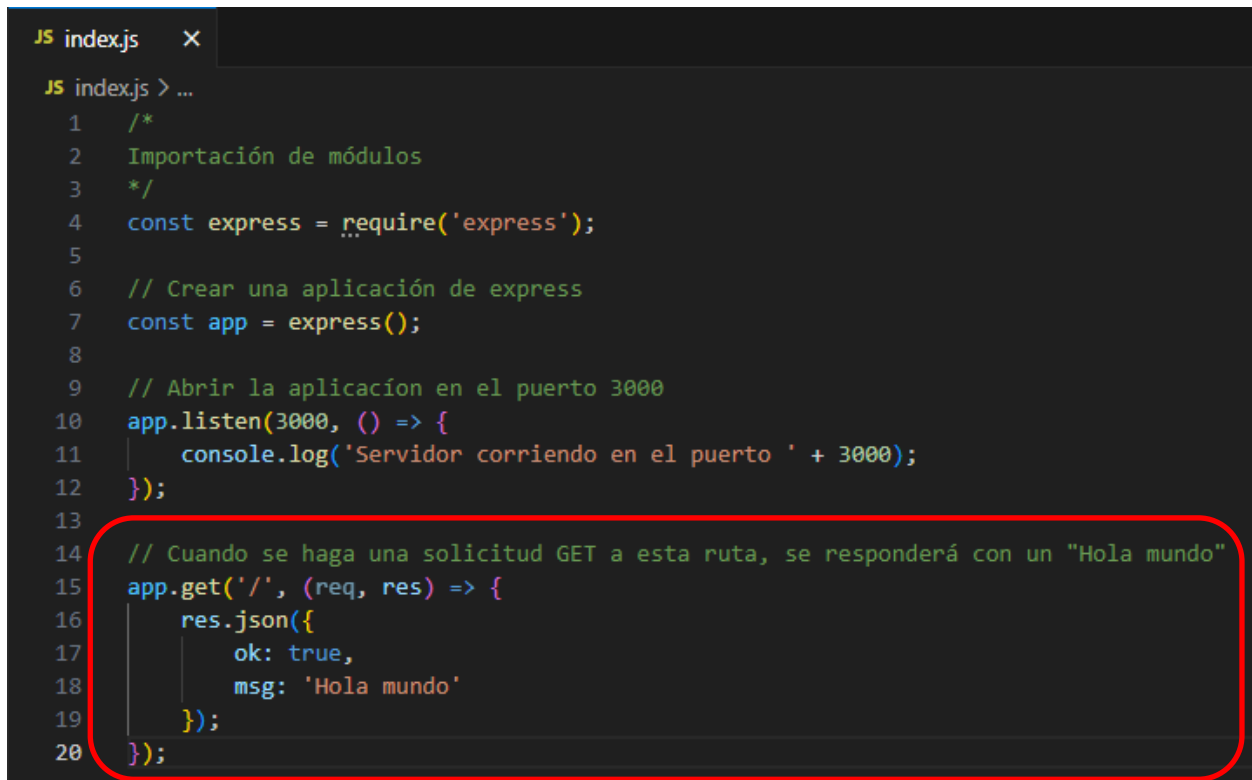
Figura 10. Inclusion de nodemon en el script de arranque de package.json.

Los scripts nos permiten asignar instrucciones a una palabra clave. En el caso de este script, cuando escribamos **npm run start:dev**, lo que hará es desplegar el servidor con el comando **nodemon index.js**.

2.4. Primera ruta GET, Postman, CORS y DOTENV

Vamos a crear nuestra primera ruta y aprender a probarla con la herramienta Postman, para ello vamos a

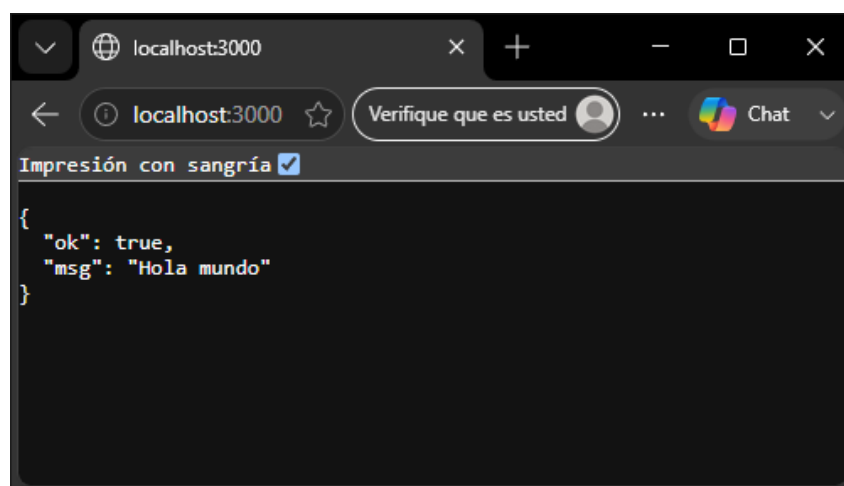
añadir al archivo **index.js**, después de la instrucción `listen`, una instrucción `get` como la siguiente:



```
JS index.js  X
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5
6  // Crear una aplicación de express
7  const app = express();
8
9  // Abrir la aplicación en el puerto 3000
10 app.listen(3000, () => {
11   console.log('Servidor corriendo en el puerto ' + 3000);
12 });
13
14 // Cuando se haga una solicitud GET a esta ruta, se responderá con un "Hola mundo"
15 app.get('/', (req, res) => {
16   res.json({
17     ok: true,
18     msg: 'Hola mundo'
19   });
20 });
```

Figura 11. Creación de un endpoint.

Esta instrucción lo que hace es ante una petición "get /", es decir una petición HTTP de tipo GET al recurso "/", responder con un mensaje, que en este caso está montado como un JSON en el que hay dos campos, el campo "ok" con el valor "true" y el campo "msg" con el valor 'Hola mundo'. Ahora si ejecutamos nuestra aplicación podemos ver el resultado en el navegador.



```
localhost:3000
Verifique que es usted
Impresión con sangría
{
  "ok": true,
  "msg": "Hola mundo"
}
```

Figura 12. Resultado mostrado en el navegador tras la consulta del endpoint creado.

2.4.1. Postman

Pero en el navegador la forma la que se visualiza la información no es adecuada, pues no estamos visualizando toda la información que realmente devuelve nuestra API. Para poder hacer pruebas con APIs la herramienta adecuada es Postman (descargar). Esta herramienta es en parte gratuita y muy útil para probar APIs.

En la web de Postman nos pedirán que creamos una cuenta y después podremos realizar las solicitudes desde la aplicación web o descargar la versión de escritorio, ambas funcionan de igual forma.

Para realizar una solicitud mediante Postman tendríamos que darle al botón de create en la pantalla de inicio para crear una colección (carpeta/proyecto para almacenar solicitudes a un servicio):

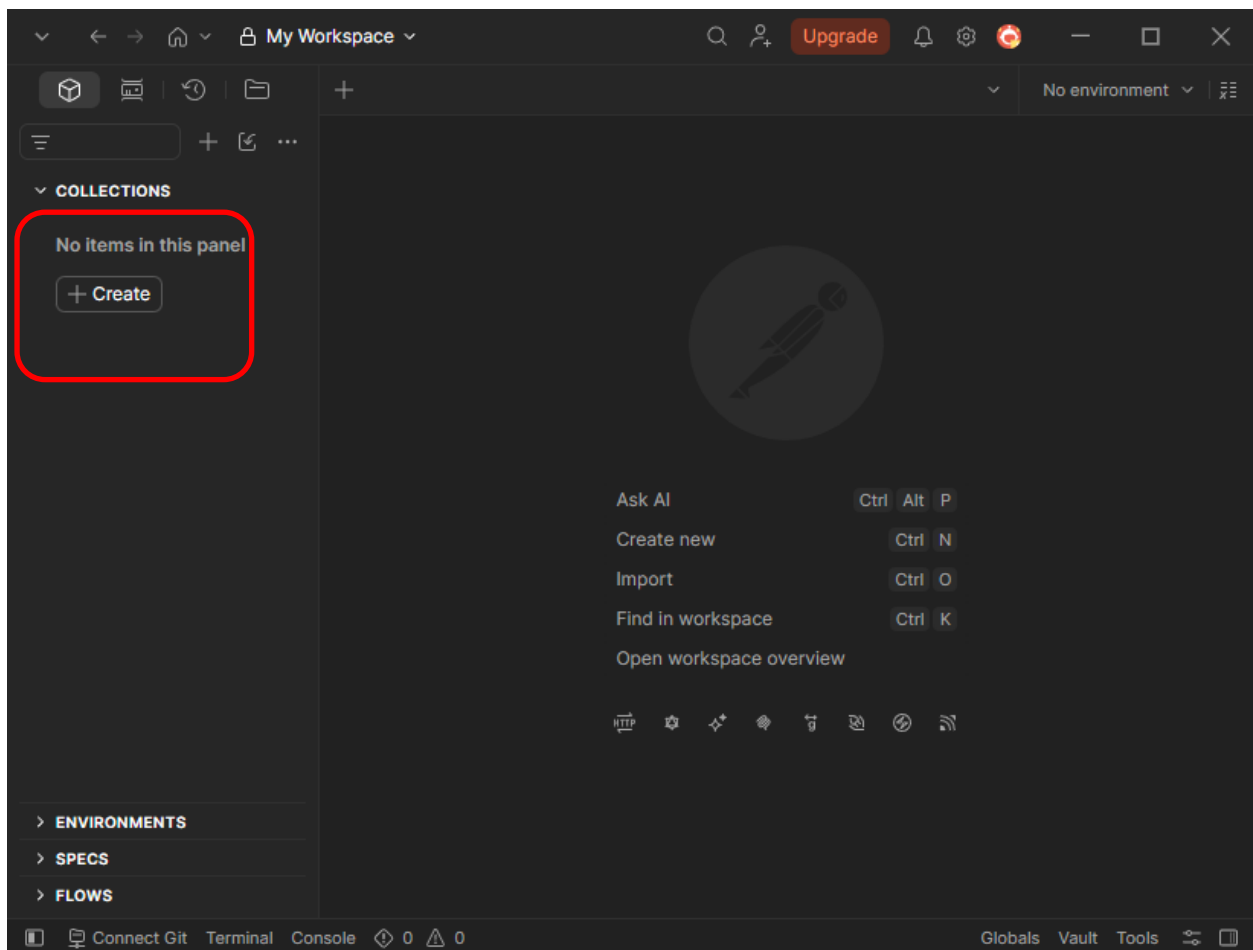


Figura 13. Interfaz de Postman, botón create.

Después de esto veremos que se ha creado una colección así, y podremos crear nuestra primera solicitud con el botón señalado en la siguiente captura:

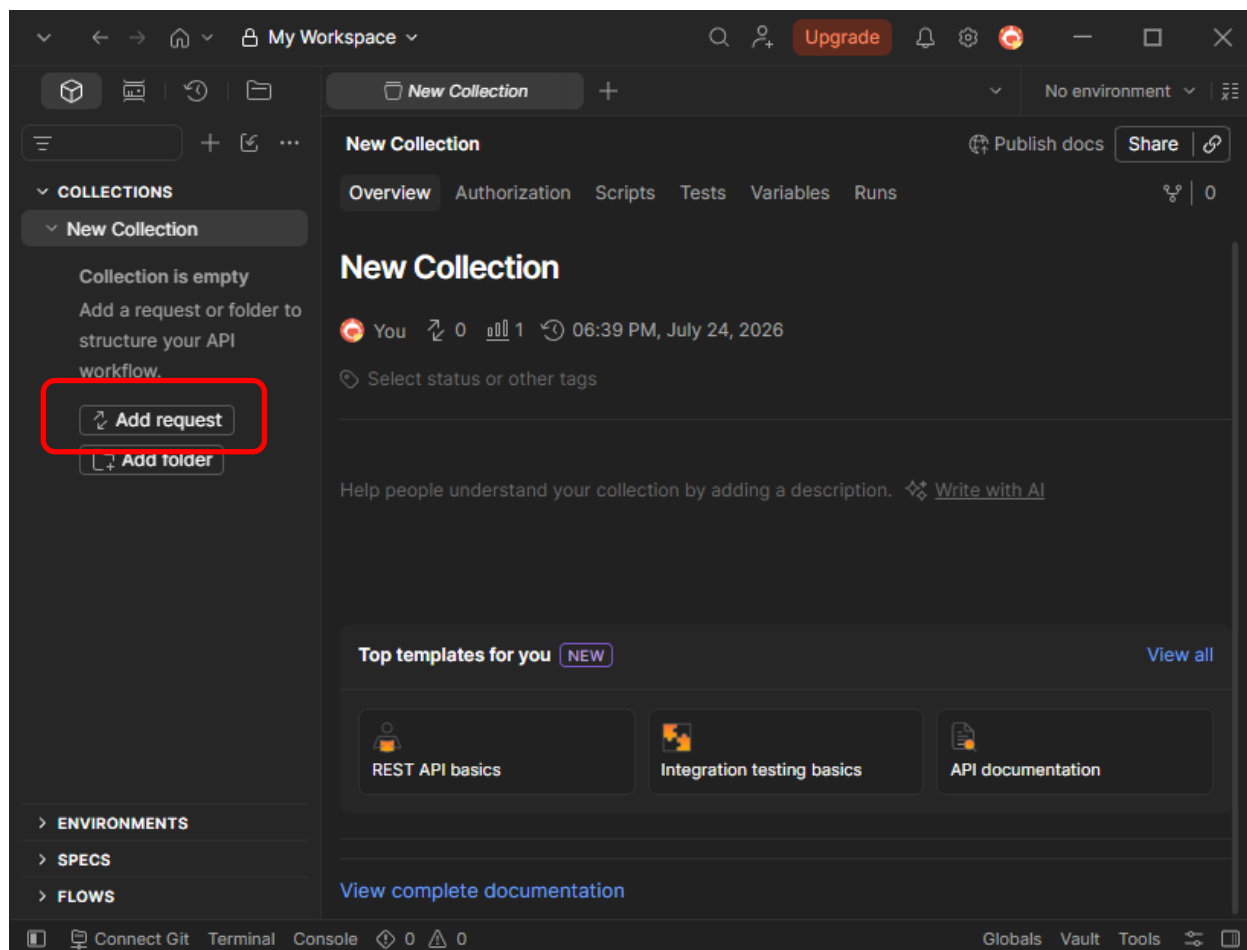


Figura 14. Interfaz de Postman, botón add request.

Si colocamos la ruta principal de nuestro servicio (localhost:3000) en la ruta de nuestra solicitud, deberíamos de obtener algo así:

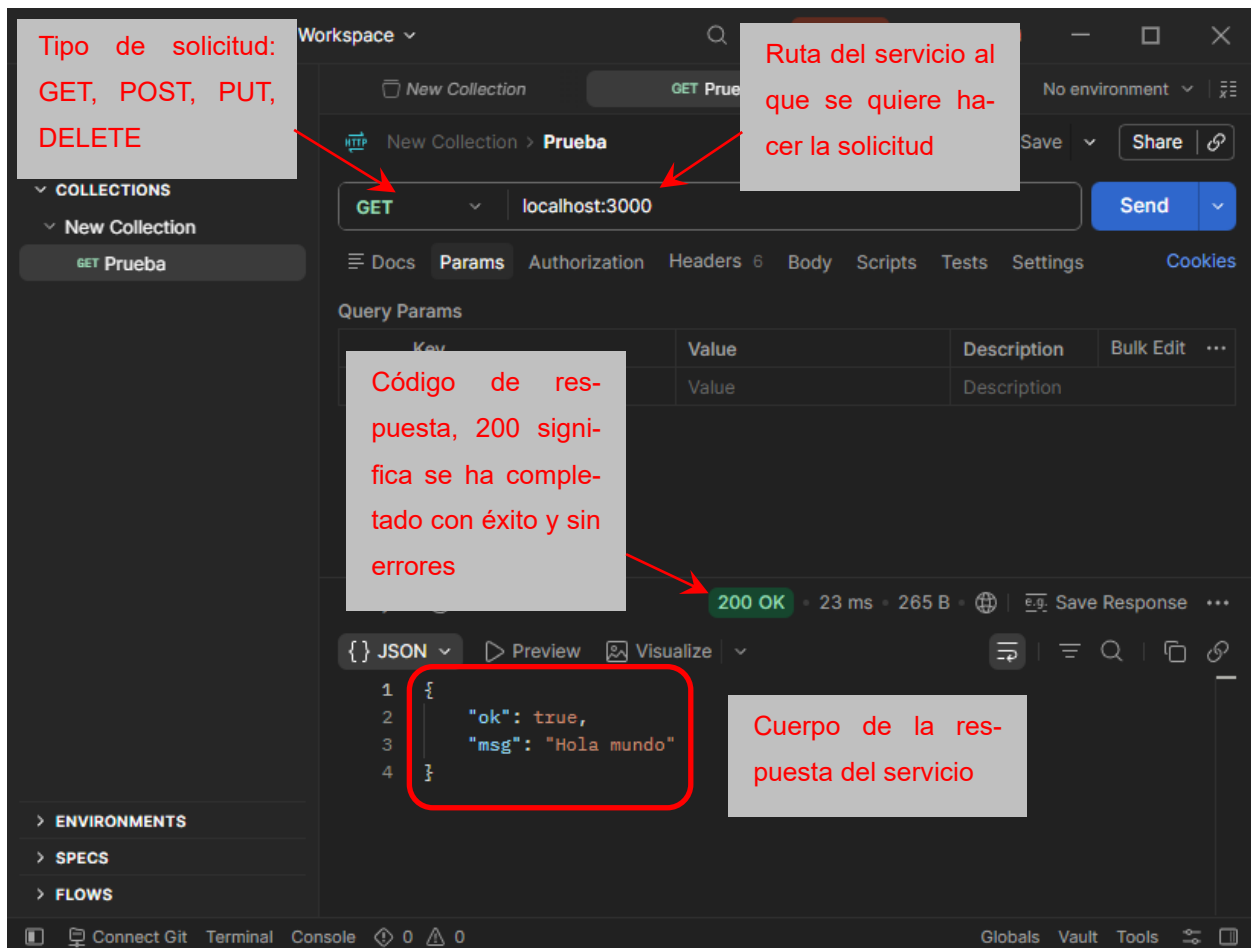


Figura 15. Resultado de lanzar la petición get a nuestro servidor.

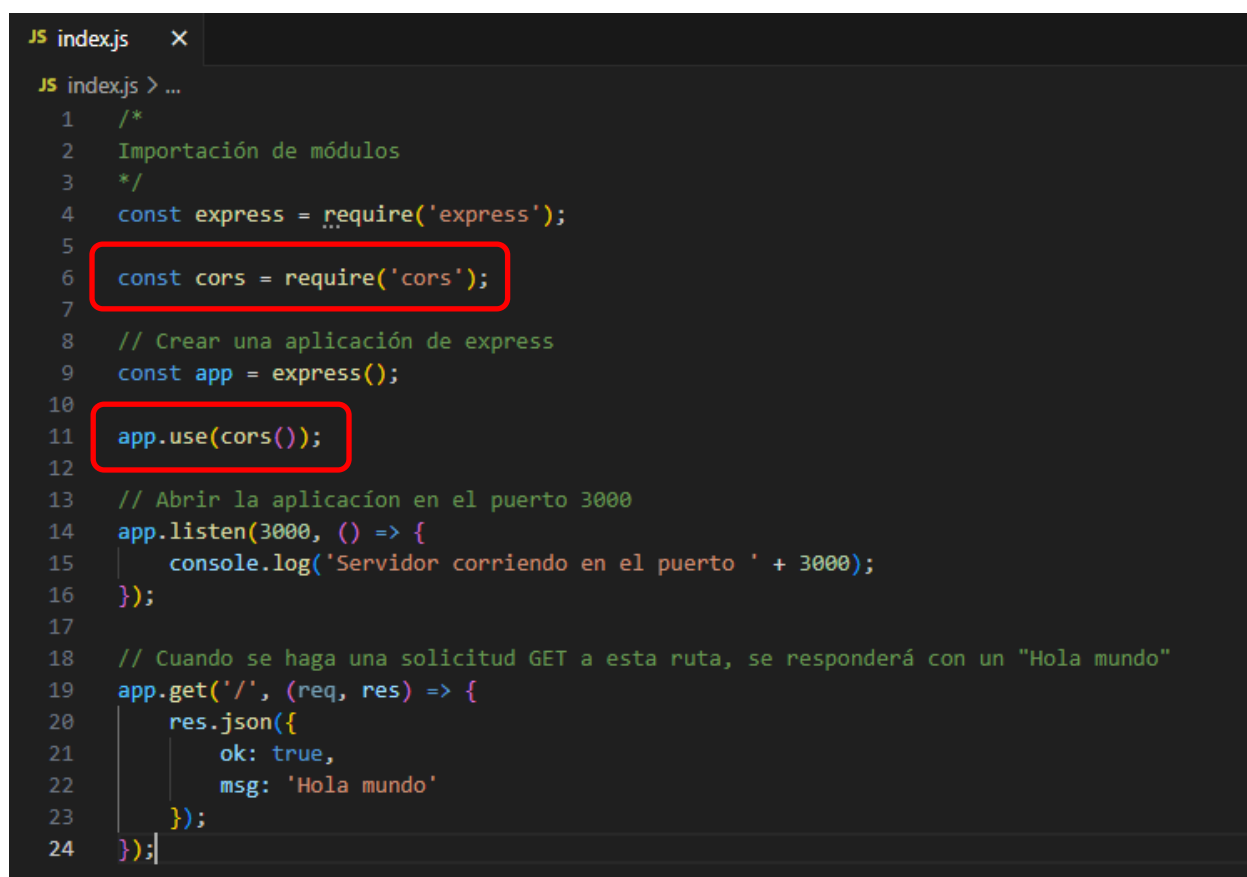
2.4.2. CORS

Ahora para la base de nuestras rutas vamos a añadir un nuevo módulo que permitirá habilitar CORS. CORS (Cross-origin resource sharing, intercambio de recursos de origen cruzado) es un mecanismo que permite que se puedan solicitar recursos restringidos (como por ejemplo, las tipografías) en una página web desde un dominio diferente del dominio que sirvió el primer recurso. En el caso de una API normalmente va a ser utilizada desde un página web que no tiene porque haber sido descargada desde el mismo servidor que reside la API. Habilitar CORS en una API simplemente implica que añada una cabecera en las respuestas que le dice a un navegador que esa API puede ser utilizada aunque la web no haya sido descargado del mismo origen (del mismo servidor) que la API.

Para esto es muy sencillo, simplemente vamos a instalar un paquete llamado CORS (modulo cors) con el siguiente comando:

```
npm install cors
```

Una vez instalado importamos el módulo "cors" y antes de hacer el `app.listen` utilizamos la directiva "use" de express para indicar el uso de un middleware en todo nuestro tratamiento de rutas, es decir, añadimos "`app.use(cors())`" de esta forma:



```
JS index.js X
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5
6  const cors = require('cors');
7
8  // Crear una aplicación de express
9  const app = express();
10
11  app.use(cors());
12
13  // Abrir la aplicación en el puerto 3000
14  app.listen(3000, () => {
15    console.log('Servidor corriendo en el puerto ' + 3000);
16  });
17
18  // Cuando se haga una solicitud GET a esta ruta, se responderá con un "Hola mundo"
19  app.get('/', (req, res) => {
20    res.json({
21      ok: true,
22      msg: 'Hola mundo'
23    });
24  });
```

Figura 16. Inclusión de CORS en el servidor.

Al hacer el "use(cors())", de ahí en adelante todo lo que haga "app" utilizará lo que implementa "cors()". De esta forma es muy sencillo añadir middlewares a express, es decir, funcionalidades que se utilizarán en el tratamiento de las peticiones. Si creamos una prueba a parte de la que ya hemos realizado, podremos observar que, en la pestaña de cabeceras, ha aparecido una nueva.

Cabeceras de la prueba anterior:

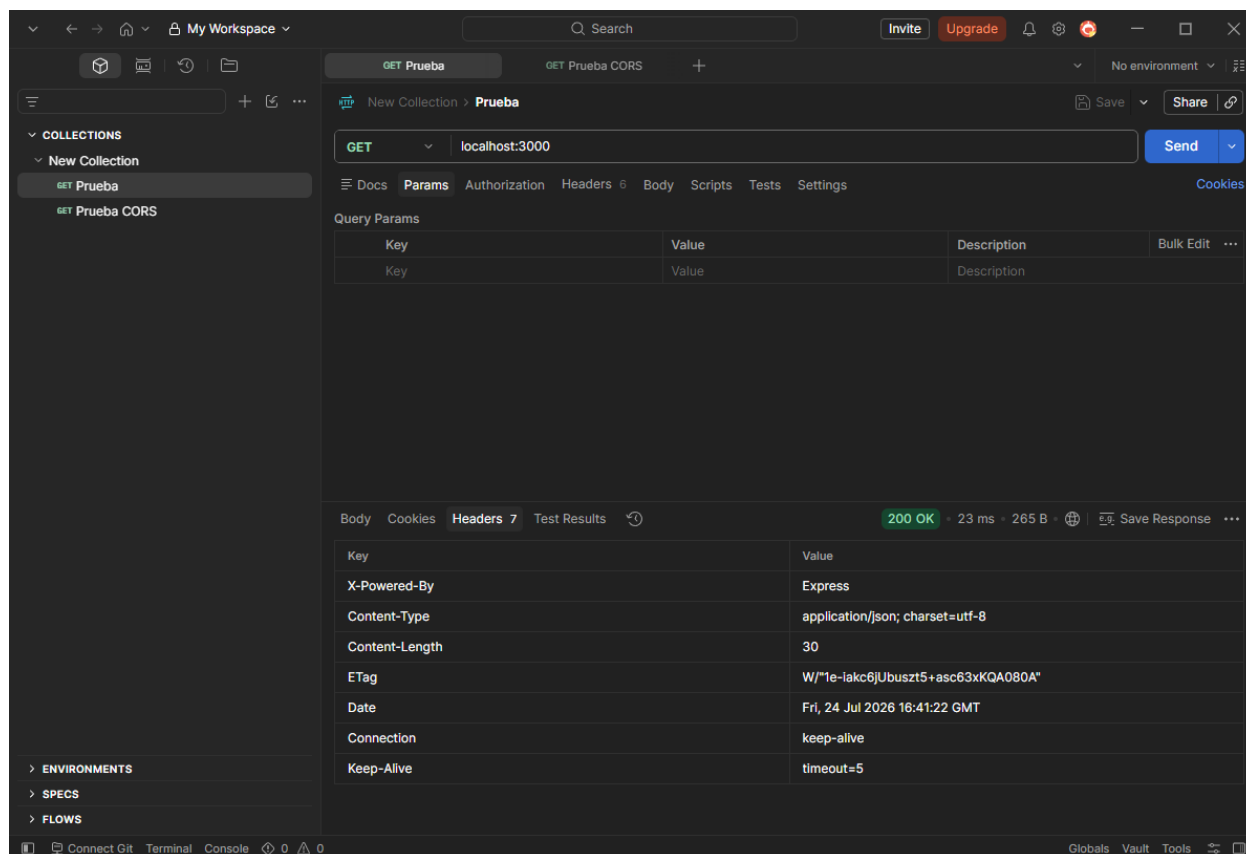


Figura 17. Respuesta de la API sin CORS en el servidor.

Cabeceras de la prueba CORS:

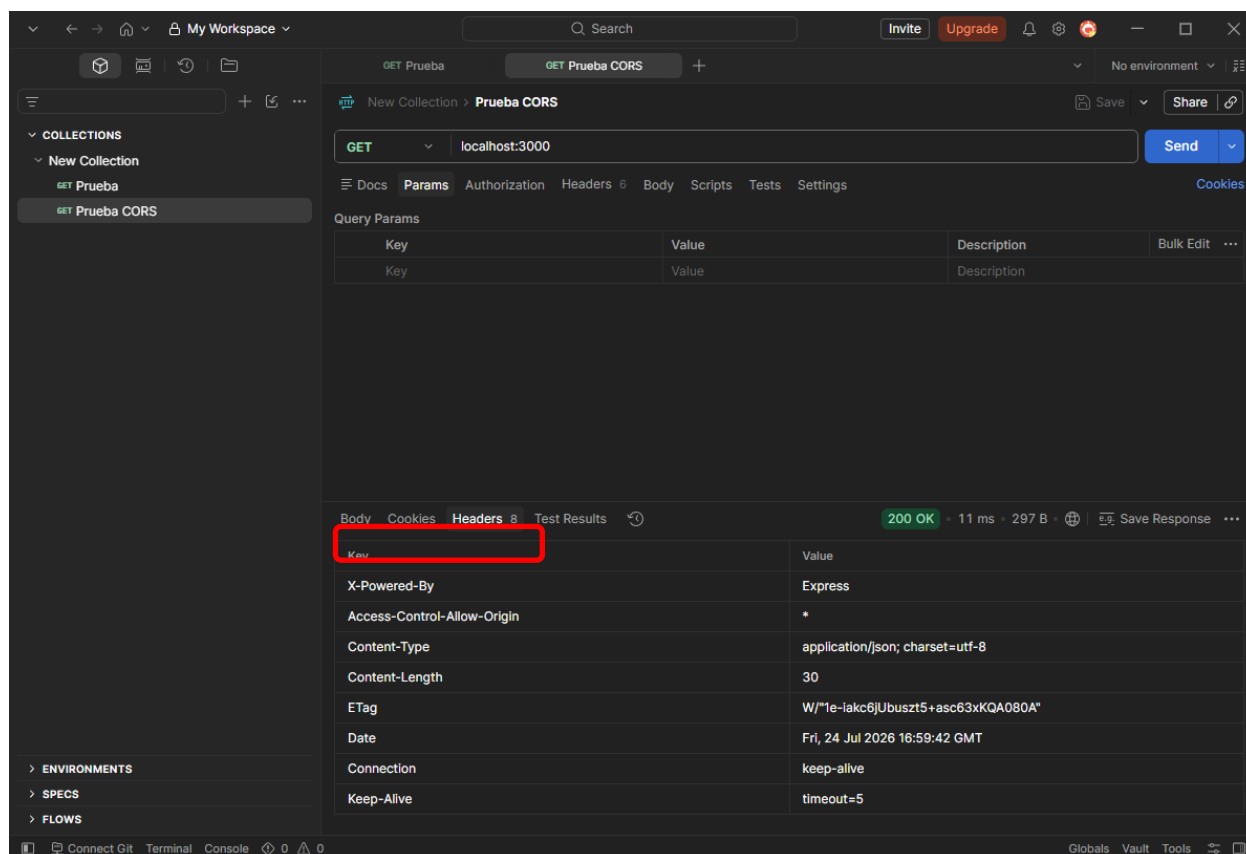


Figura 18. Respuesta de la API con CORS en el servidor.

La nueva cabecera lo que hace es que indica al navegador si los recursos de un servidor pueden compartirse con una página web de un origen o dominio distinto. Funciona como un permiso de seguridad dentro del sistema CORS para saltar la regla que bloquea páginas de diferentes sitios.

Si al probar CORS ves que no aparece la cabecera "Access-Control-Allow-Origin", prueba a reiniciar el servidor o cambiar de puerto y hacer la petición de nuevo.

2.4.3. ENV

Por último, vamos a añadir otro módulo que nos va a permitir crear y utilizar variables de entorno cargadas desde un archivo de variables. Para ello instalamos el módulo DOTENV (modulo dotenv). Es muy útil ver la página de información del módulo porque nos enseña como utilizarlo. Como siempre lo que tenemos que hacer es instalar el módulo.

```
npm install dotenv
```

Y para usarlo, incluiremos en nuestro código la instrucción:

```
require('dotenv').config()
```

Esta instrucción se encargará de buscar el archivo ".env" en la raíz de nuestro proyecto y cargar las variables que en ese archivo estén declaradas en una variable llamada process.env.

Después crearemos un archivo en la raíz del proyecto llamado ".env", en el cual podremos almacenar nuestras variables globales y lo utilizaremos para la configuración de la cadena de conexión a la base de datos. En este archivo escribiremos lo siguiente:

```
PORT=3000
```

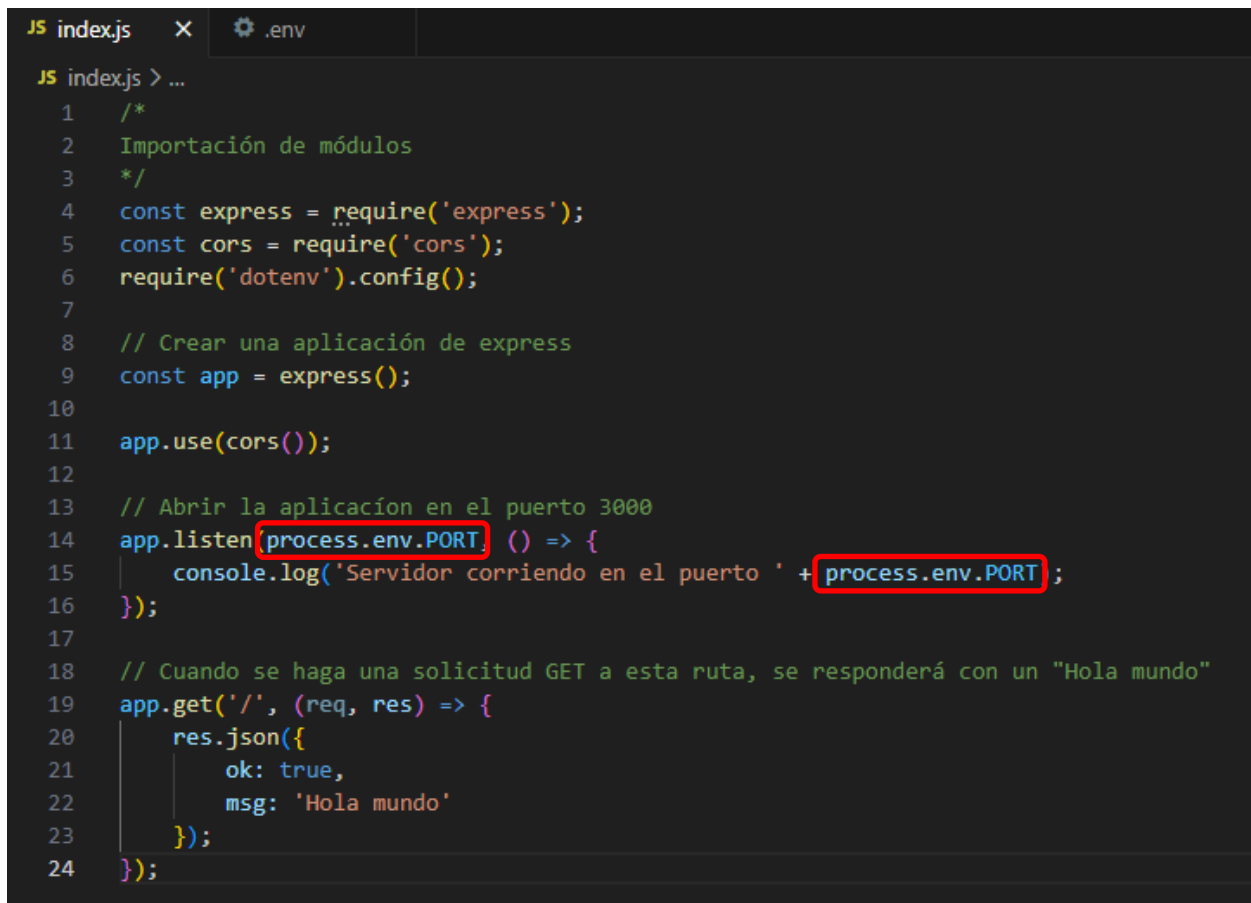
Esta será la variable del puerto de nuestro programa.

Como en nuestro archivo index.js todavía tenemos el número de puerto escrito a mano, tenemos que sustituirlo por lo siguiente:

```
process.env.PORT
```

Esta es la forma de obtener los valores del archivo .env, utilizamos process.env. y el nombre de la variable que pusimos en el archivo .env.

El archivo index.js debería de verse así:



```
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5  const cors = require('cors');
6  require('dotenv').config();
7
8  // Crear una aplicación de express
9  const app = express();
10
11 app.use(cors());
12
13 // Abrir la aplicación en el puerto 3000
14 app.listen(process.env.PORT, () => {
15   console.log('Servidor corriendo en el puerto ' + process.env.PORT);
16 });
17
18 // Cuando se haga una solicitud GET a esta ruta, se responderá con un "Hola mundo"
19 app.get('/', (req, res) => {
20   res.json({
21     ok: true,
22     msg: 'Hola mundo'
23   });
24 });
```

Figura 19. Uso de variables almacenadas en .env..

Ahora tenemos un backend el cual:

- Es capaz de manejar rutas
- Habilita CORS para ser utilizado desde cualquier aplicación.
- Carga variables de entorno mediante DOTENV
- Sabemos examinar con POSTMAN

2.5. Configuración de SQLite

Para configurar nuestra base de datos, que será local y se almacenará en un archivo dentro de la carpeta de nuestro proyecto, lo primero que tendremos que hacer es ejecutar el comando de instalación en la consola:

```
npm install sqlite3 sqlite
```

Después de esto deberemos de añadir la ruta donde se creará nuestra base de datos al archivo .env, para poder acceder a ella de forma cómoda y sin tener que escribirla a mano. Para ello añadiremos esta línea en el .env:

```
DB_PATH=./database/database.sqlite
```

Después crearemos la carpeta para la base de datos y todo lo que tiene que ver con ella. La carpeta se debe de llamar "database" y estará en la raíz del proyecto. Dentro de ella tendremos que crear el archivo de conexión, llamado "configdb.js" que contendrá el siguiente código.

```
const sqlite3 = require('sqlite3');
const { open } = require('sqlite');
```

```
let db;

const dbConnection = async () => {
  try {
    db = await open({
      filename: process.env.DB_PATH || './database/database.sqlite',
      driver: sqlite3.Database
    });

    console.log('DB online (SQLite)');
    return db;
  } catch (error) {
    console.log(error);
    throw new Error('Error al iniciar la BD SQLite');
  }
};

const getDb = () => db;

module.exports = {
  dbConnection,
  getDb
};
```

Index.js debería de verse así:

```
JS configdb.js JS index.js X
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5  const cors = require('cors');
6  require('dotenv').config();
7
8  // Importar el módulo de conexión
9  const { dbConnection } = require('./database/configdb');
10
11 // Crear una aplicación de express
12 const app = express();
13
14 // Conectar a la base de datos SQLite
15 dbConnection();
16
17 app.use(cors());
18
19 // Abrir la aplicación en el puerto 3000
20 app.listen(process.env.PORT, () => {
21   console.log('Servidor corriendo en el puerto ' + process.env.PORT);
22 });
23
24 // Cuando se haga una solicitud GET a esta ruta, se responderá con un "Hola mundo"
25 app.get('/', (req, res) => {
26   res.json({
27     ok: true,
28     msg: 'Hola mundo'
29   });
30 });
```

Figura 20. Conexión a una BD.

Si arrancamos el servidor después de realizar estos cambios en el proyecto veremos que no ha cambiado nada en la respuesta de la solicitud. Sin embargo, en la carpeta database podremos observar que se ha creado el archivo database.sqlite correctamente.

2.6. Descarga Código 1

Todo lo desarrollado hasta ahora está disponible en el “Codigo1.zip” que puedes descargar desde aquí:

[Descargar Código 1](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo1.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo1.zip>

2.7. Estructuración del proyecto: CONTROLLERS y ROUTERS

Tal y como está ahora, todo el manejo de rutas de nuestra API se haría desde index.js. Esto a la larga generaría un código imposible de mantener ya que una API tiene decenas o cientos de rutas y cada una de ellas además desarrolla una lógica de negocio que puede ser muy grande como manipular los datos que le llegan, almacenarlos, recuperar cientos de registros, formatearlos para devolverlos de una forma adecuada, etc. Es por ello que hay que estructurar el proyecto de forma que sea:

- Modular: basado en módulos que separan los elementos relacionados en contenedores independientes. En este caso separaremos por entidades
- Mantenible: que sea fácil analizar el código desarrollado y legible
- Trabajar en paralelo: si se separa en módulos diferentes un desarrollador puede trabajar en un módulo de forma independiente y sin influir sobre el trabajo de otro

Para ello vamos a crear dentro del proyecto tres carpetas nuevas:

- controllers: almacenaremos aquí los controladores
- routes: almacenaremos aquí los routers, es decir los declaradores de rutas
- models: almacenará los modelos, representaciones de las colecciones que almacenará nuestra base de datos y a través de los cuales interactuamos con la base de datos
- Para comenzar a trabajar vamos a suponer que en nuestra base de datos existirá una colección llamada "Users". En nuestra API debe existir entonces una ruta que permita la interacción con esta colección, el llamado CRUD, métodos create, read, update y delete.

Para prepara nuestro código, en index.js eliminaremos el app.get y añadiremos la siguiente linea tras el app.use(cors())

```
app.use('/api/users', require('./routes/users'));
```

Este comando le indica a la aplicación que de ahora en adelante, cualquier cosa que venga con la ruta '/api/users...' lo atenderá el archivo users.js dentro de la carpeta routes (el .js de users no es necesario en los require ya que node asume que es javascript).

Dentro de ese archivo `/routes/users.js` escribiremos lo siguiente:

```
const {Router } = require('express');
const { getUsers } = require('../controllers/users');
const router = Router();
router.get('/', getUsers);
module.exports = router;
```

Lo que hacemos es cargar una clase `Router` que viene de `express`, importar un método `'getUsers'` que veremos más adelante, declarar un objeto `router` de tipo `Router` y ahora sí, con el comando:

```
router.get('/', getUsers);
```

Estamos diciendo que la ruta `'/'` a partir de la ruta base que ha hecho saltar a aquí, es decir `'/api/users'`, se atienda mediante la función `getUsers`.

Por último, se exporta el objeto `'router'` para que pueda ser utilizado desde fuera del módulo (en este caso desde `index.js`)

Y por último en `/controllers/users.js` se coloca el siguiente código:

```
// Función que se llama cuando se recibe una solicitud GET en /api/users
const getUsers = async(req, res) => {
  res.json({
    ok: true,
    msg: 'getUsers'
  });
}
module.exports = { getUsers }
```

Este código simplemente crea un función `'getUsers'` la cual recibe un request y devuelve un response y en el response configura una respuesta como la que ya hemos visto antes.

Comprobaremos que la nueva ruta funciona utilizando Postman con la siguiente solicitud, después de iniciar la aplicación de nuevo con **`npm run start:dev`**:

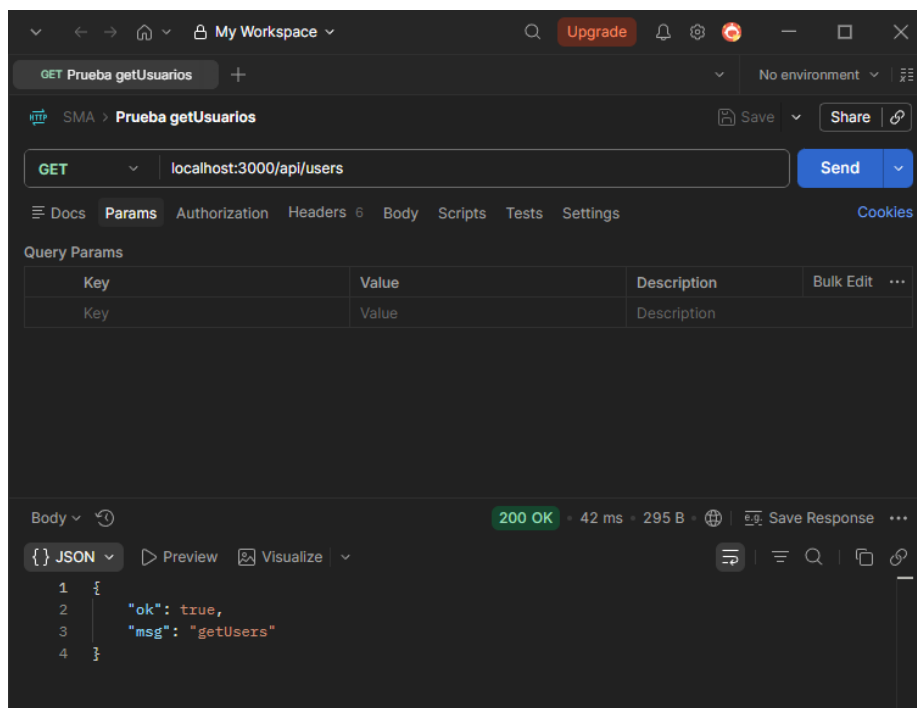


Figura 21. Salida en postman de la nueva ruta.

2.8. Crear modelos: MODELS

Para poder trabajar contra la base de datos debemos tener una estructura definida que nos permita almacenar y recuperar la información de forma organizada.

En SQLite, al ser una base de datos relacional, la estructura de la información se define mediante tablas SQL. Una tabla es la declaración de la estructura donde se almacenan los datos. En ella se especifican las columnas (campos), los tipos de datos (como TEXT o INTEGER) y sus restricciones (como valores obligatorios o únicos).

Un modelo en nuestro código Node.js será un módulo o archivo JavaScript que contendrá la lógica y las consultas SQL necesarias para ejecutar las acciones CRUD (crear, leer, actualizar y eliminar) sobre dicha tabla.

En la carpeta models vamos a crear nuestro modelo users.js. En este archivo importaremos el método para obtener la conexión a la base de datos SQLite y declararemos la consulta SQL para estructurar la tabla:

```
const { getDb } = require('../database/configdb');

// Función para inicializar la tabla 'users' si no existe
const initUsersTable = async () => {
  const db = getDb();
  await db.exec(`
    CREATE TABLE IF NOT EXISTS users (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nombre TEXT NOT NULL,
      email TEXT NOT NULL UNIQUE,
      password TEXT NOT NULL,
      rol TEXT NOT NULL DEFAULT 'USER',
      fecha_alta TEXT NOT NULL DEFAULT (datetime('now')),
      token TEXT
    );
  `);
};

module.exports = {
  initUsersTable
};
```

Declaramos la tabla llamada users indicando las columnas y sus limitaciones en lenguaje SQL:

- id: Identificador único de tipo entero (INTEGER), configurado como clave primaria (PRIMARY KEY) y autoincrementable (AUTOINCREMENT).
- name: Campo de tipo texto (TEXT) marcado como NOT NULL (obligatorio, equivalente a required: true).
- email: De tipo TEXT, obligatorio (NOT NULL) y marcado con la restricción UNIQUE (no se puede repetir en la base de datos).
- password: De tipo TEXT y obligatorio (NOT NULL).
- role: De tipo TEXT, obligatorio (NOT NULL) y con un valor por defecto mediante DEFAULT 'ALUMNO'.
- reg_date: Campo de tipo texto que utiliza la función datetime('now') de SQLite para guardar automáticamente la fecha y hora exacta en la que se crea el registro.
- token: Campo de tipo texto que almacenará el token activo de la sesión del usuario. Al guardar el nuevo token tras cada inicio de sesión, se invalidará el anterior para garantizar que solo exista una

sesión abierta a la vez.

En SQLite, la tabla creada dentro de la base de datos equivale directamente a la colección de datos del modelo.

Ahora para usar el modelo en nuestro controlador solo tenemos que importarlo, añadimos el require en el controlador de usuario.js:

```
const User = require('../models/users');
```

A continuación, dentro del archivo models/users.js, añadimos la función encargada de consultar y retornar los datos guardados en la tabla mediante la sentencia SELECT:

```
// Realiza una solicitud a la base de datos con la sentencia SQL que aparece en
db.all()
const getUsers = async () => {
  const db = getDb();
  return await db.all("SELECT * FROM users");
};

module.exports = {
  initUsersTable,
  getUsers
};
```

Ahora para usar el modelo en nuestro controlador solo tenemos que importarlo. Añadimos la importación en el archivo /controllers/users.js:

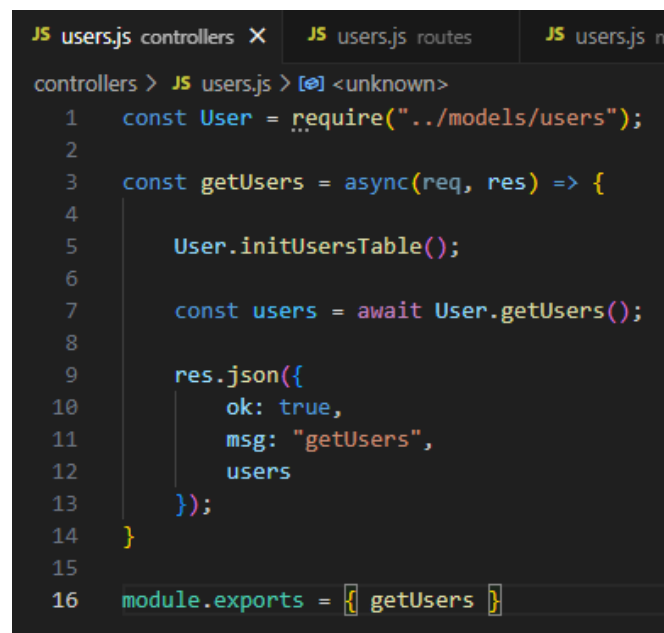
```
const User = require('../models/users');
```

Y por último, dentro de la función getUsers podemos hacer uso del modelo. Para eso añadimos:

```
const users = await User.getUsers();
```

Esto ejecutará la consulta SQL SELECT * FROM users sobre la tabla users y recuperará todos los registros guardados en la base de datos.

Nuestro archivo controller debería de verse así:



```
JS users.js controllers X JS users.js routes JS users.js m
controllers > JS users.js > [e] <unknown>
1  const User = require("../models/users");
2
3  const getUsers = async(req, res) => {
4
5      User.initUsersTable();
6
7      const users = await User.getUsers();
8
9      res.json({
10         ok: true,
11         msg: "getUsers",
12         users
13     });
14 }
15
16 module.exports = { getUsers }
```

Figura 22. Archivos controllers.js.

Si ejecutamos nuestra aplicación ahora veremos que nos sale un mensaje extra en la respuesta a la petición a `/api/users`:

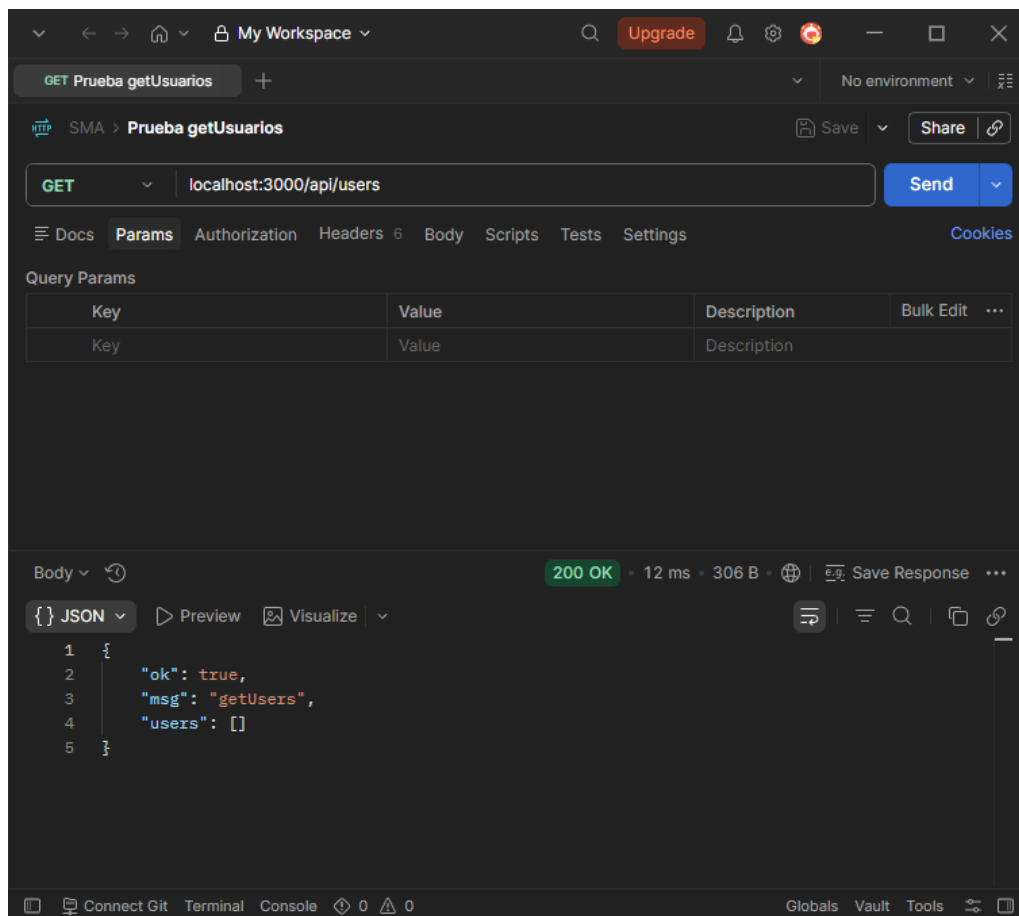


Figura 23. Salida en postman con el nuevo controller.

Los corchetes son el contenido de la tabla de users que en este caso, como no tenemos datos, nos devuelve unos corchetes vacíos. Vamos a ver como gestionar nuestra base de datos con mayor facilidad y comodidad, y a como crear nuevos datos manualmente sin tener que hacer solicitudes mediante Postman para cada nueva información.

2.8.1. Visualización y gestión de la base de datos con DB Browser for SQLite

Para conectarnos a nuestra base de datos SQLite y poder examinar su contenido de forma gráfica, utilizaremos **DB Browser for SQLite**. Esta aplicación gratuita nos permite inspeccionar las tablas, ejecutar consultas manualmente y modificar datos de manera sencilla.

Descargamos e instalamos la aplicación desde su sitio web oficial (<https://sqlitebrowser.org/>) y en la pestaña de download elegiremos la opción que nos convenga, en caso de Windows, elegiremos la siguiente:

Downloads

([Please consider sponsoring us on Patreon](#) 😊)

Windows

Our latest release (3.13.1) for Windows:

- [DB Browser for SQLite - Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite - Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 64-bit Windows](#)
- [DB Browser for SQLite - Standard installer for ARM64 Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for ARM64 Windows](#)

Free code signing provided by [SignPath.io](#), certificate by [SignPath Foundation](#).

Figura 24. Interfaz para descargar SQLiteBrowser.

Durante la instalación en Windows, seleccionamos las opciones por defecto.

Una vez abierta la aplicación, hacemos clic en el botón **"Abrir base de datos"** y seleccionamos el archivo database.sqlite ubicado en la carpeta database de nuestro backend. Esto es lo que deberíamos ver:

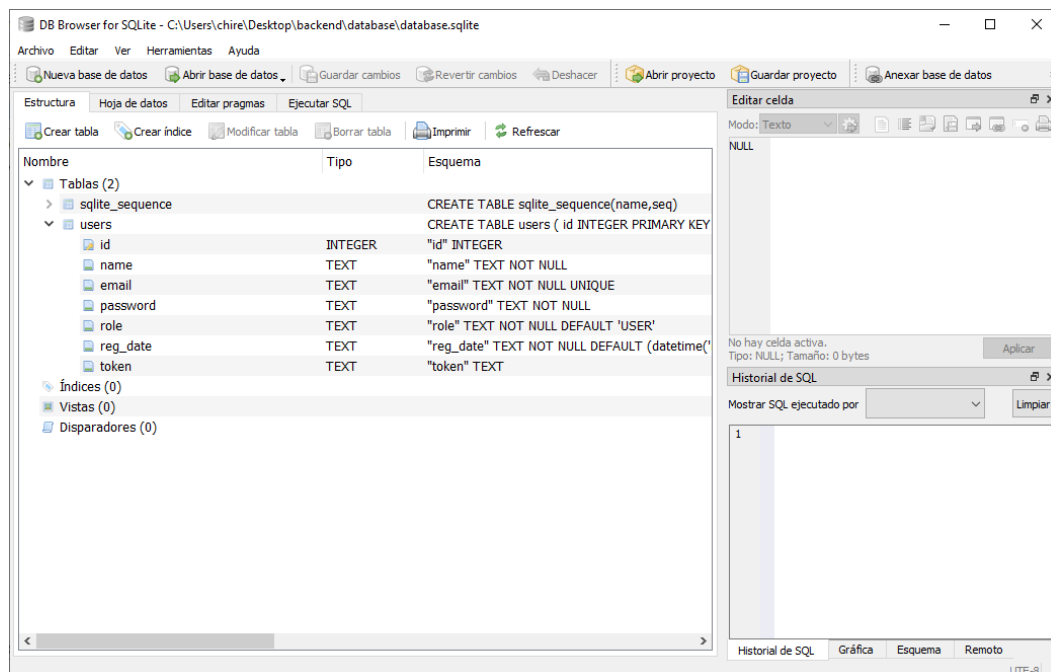


Figura 25. Interfaz SQLiteBrowser.

En la pestaña **"Estructura de la base de datos"**, observaremos las siguientes tablas:

- users: Es la tabla que hemos definido en nuestro modelo, compuesta por los campos id, name, email, password, role, reg_date y token.

- `sqlite_sequence`: Es una tabla del sistema generada automáticamente por SQLite. Se crea de forma transparente para gestionar el contador autoincremental del campo `id`. Guarda el último identificador asignado para evitar la reutilización de identificadores tras el borrado de registros.

Para verificar el funcionamiento de la base de datos y añadir un usuario manualmente:

1. Nos dirigimos a la pestaña **"Hoja de datos"** (Browse Data).
2. En el desplegable "Tabla", seleccionamos **users**.
3. Hacemos clic en el botón "Insertar un nuevo registro en la tabla actual".

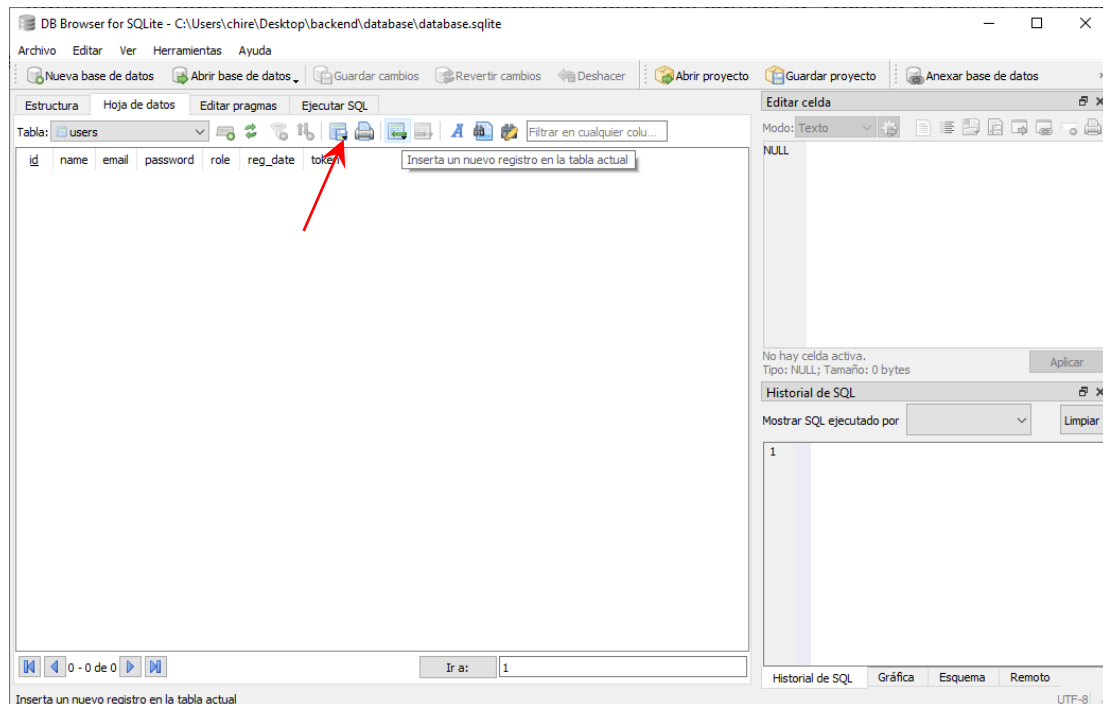


Figura 26. Insertar nuevo registro en la tabla.

4. Rellenamos los campos deseados (el campo `id` y `reg_date` se pueden dejar vacíos o en nulo, ya que la base de datos los autogenera). El campo de `token` tampoco se rellena porque como el usuario todavía no se ha autenticado en la aplicación, no existe ninguna sesión abierta y el campo se queda en NULL.
5. Finalmente, hacemos clic en el botón **"Guardar cambios"** de la barra superior para consolidar los datos en el archivo de la base de datos.

2.8.2. Selección de campos específicos en la consulta

Si en lugar de recuperar todos los datos de los usuarios queremos obtener únicamente campos concretos (por ejemplo, solo el nombre y el email), debemos adaptar la consulta.

La selección parcial de columnas se realiza indicando los nombres exactos de los campos tras la palabra clave `SELECT`, separados por comas.

Modificaremos la función `getUsers` para especificar las columnas deseadas, por ejemplo, `name` e `email`:

```
return await db.all("SELECT name, email FROM users");
```

El controlador se mantiene intacto. De esta forma, la API responderá con un listado de usuarios que contendrá únicamente las propiedades `name` e `email`, omitiendo el resto de columnas de la tabla (como la `password`, la

reg_date o el token).

Después de iniciar el proyecto con **npm run start:dev**, podremos observar realizando la petición con Postman que nos devuelve la información de la siguiente forma:

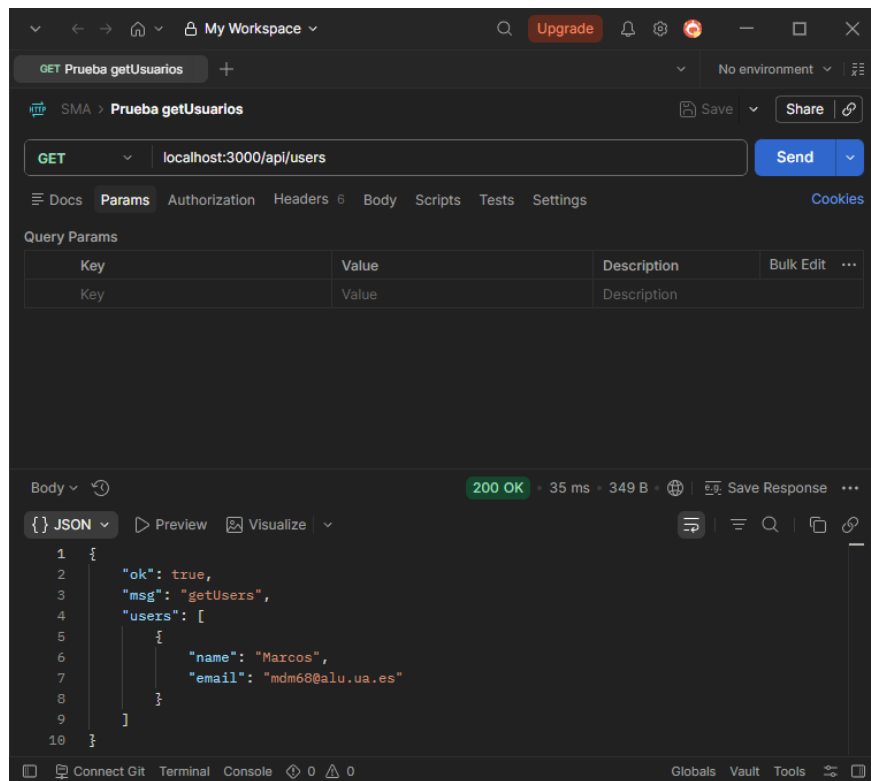


Figura 27. Resultado en postman tras insertar registro y cargar de nuevo el endpoint.

2.9. Ruta POST y validación argumentos

Vamos a trabajar con la validación de argumentos, para eso lo primero que haremos será crear una nueva ruta, la ruta POST, la cual se va a encargar de recibir una serie de argumentos para dar de alta al nuevo usuario. La ruta POST /api/users será la ruta que permitiría crear un usuario nuevo y para ello tiene que recibir los argumentos obligatorios: name, email, password y role. Además email tendrá que ser único (no repetido en la BD, pero esta comprobación la haremos más adelante). El caso es que en esta ruta se tendrán que hacer comprobaciones de que los argumentos son correctos. Vamos por ahora a crear la ruta en nuestro archivo /routes/users.js. Debería quedar así:

```
const { Router } = require('express');
const { getUsers } = require('../controllers/users');
const { createUser } = require('../controllers/users');
const router = Router();

router.get('/', getUsers);
router.post('/', createUser);

module.exports = router;
```

Ahora debemos de crear el método createUser que nos acabamos de inventar para las solicitudes POST que lleguen a la ruta /api/users, para ello accedemos a nuestro archivo /controller/users.js y escribimos lo siguiente debajo de la función getUsers:

```
// Función para crear usuarios, recibe una solicitud POST y crea un usuario con
```

```
los datos de la solicitud.  
const createUser = async(req, res) => {  
  res.json({  
    ok: true,  
    msg: "createUser"  
  });  
}
```

Una vez que tenemos la ruta, aprendamos como utilizar POSTMAN para enviar argumentos. Esto sería lo mismo que si tuviésemos un formulario en el cual rellenamos una serie de campos y se enviara.

En primer lugar, crearemos una solicitud nueva en POSTMAN, duplicando la que ya teníamos de tipo GET, y la cambiaremos a una de tipo POST modificando esta casilla:

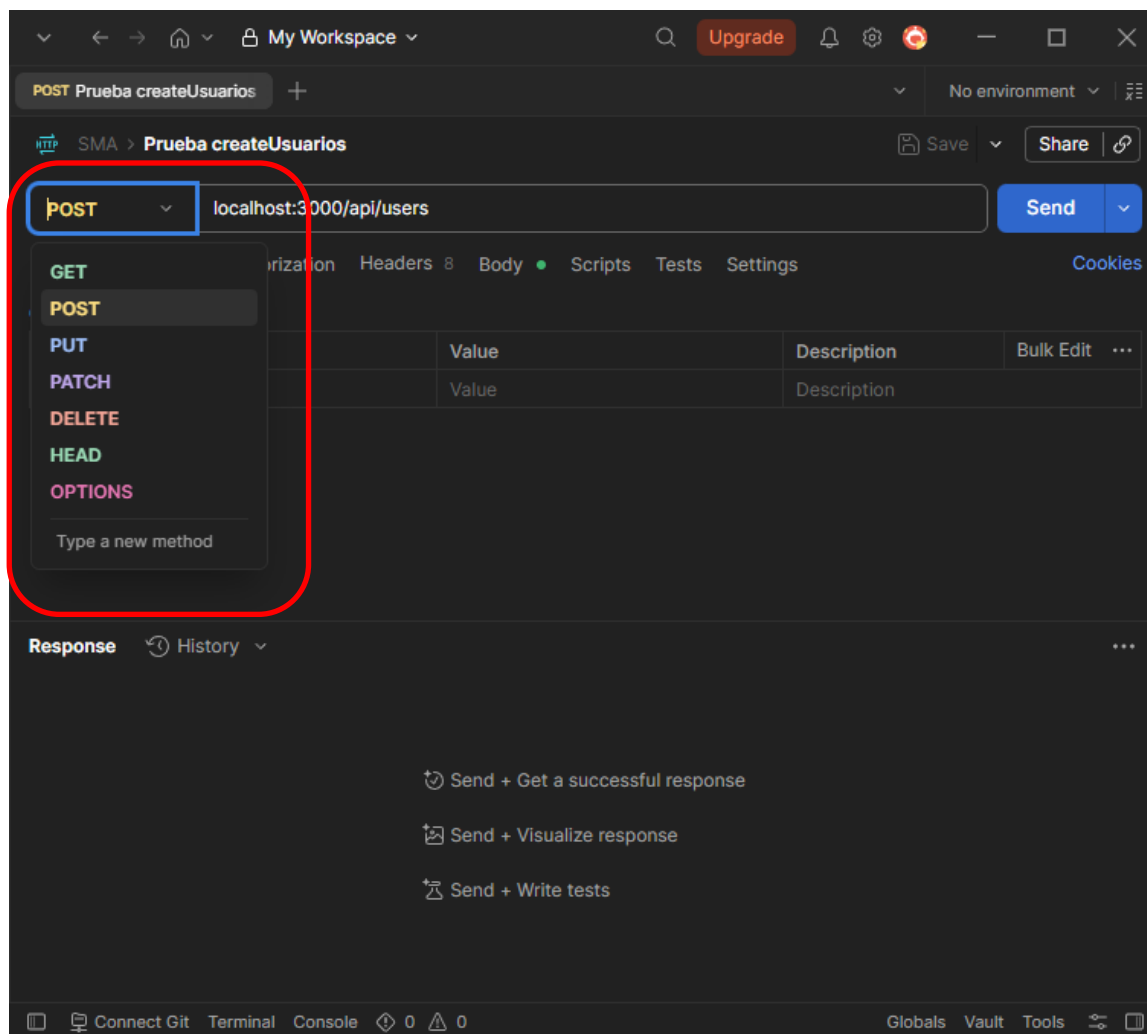


Figura 28. Crear petición POST.

Ahora que ya tenemos nuestra solicitud configurada como es debido, nos iremos a la pestaña **"Body"**, que nos permitirá escribir el cuerpo del mensaje JSON que queremos que nuestra función createUser reciba con el fin de crear un nuevo usuario. Algo así sería lo que deberíamos de colocar en el JSON:

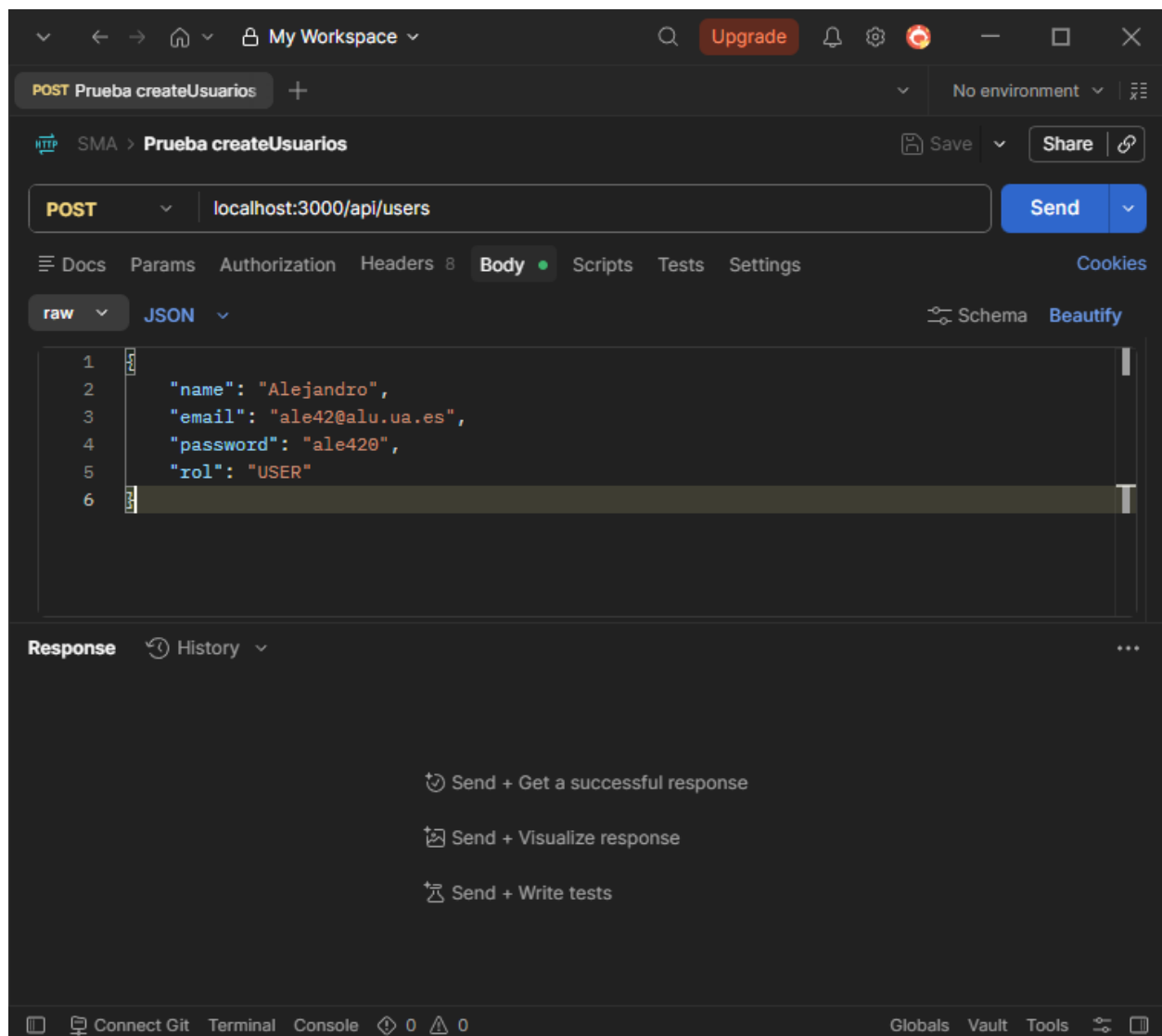
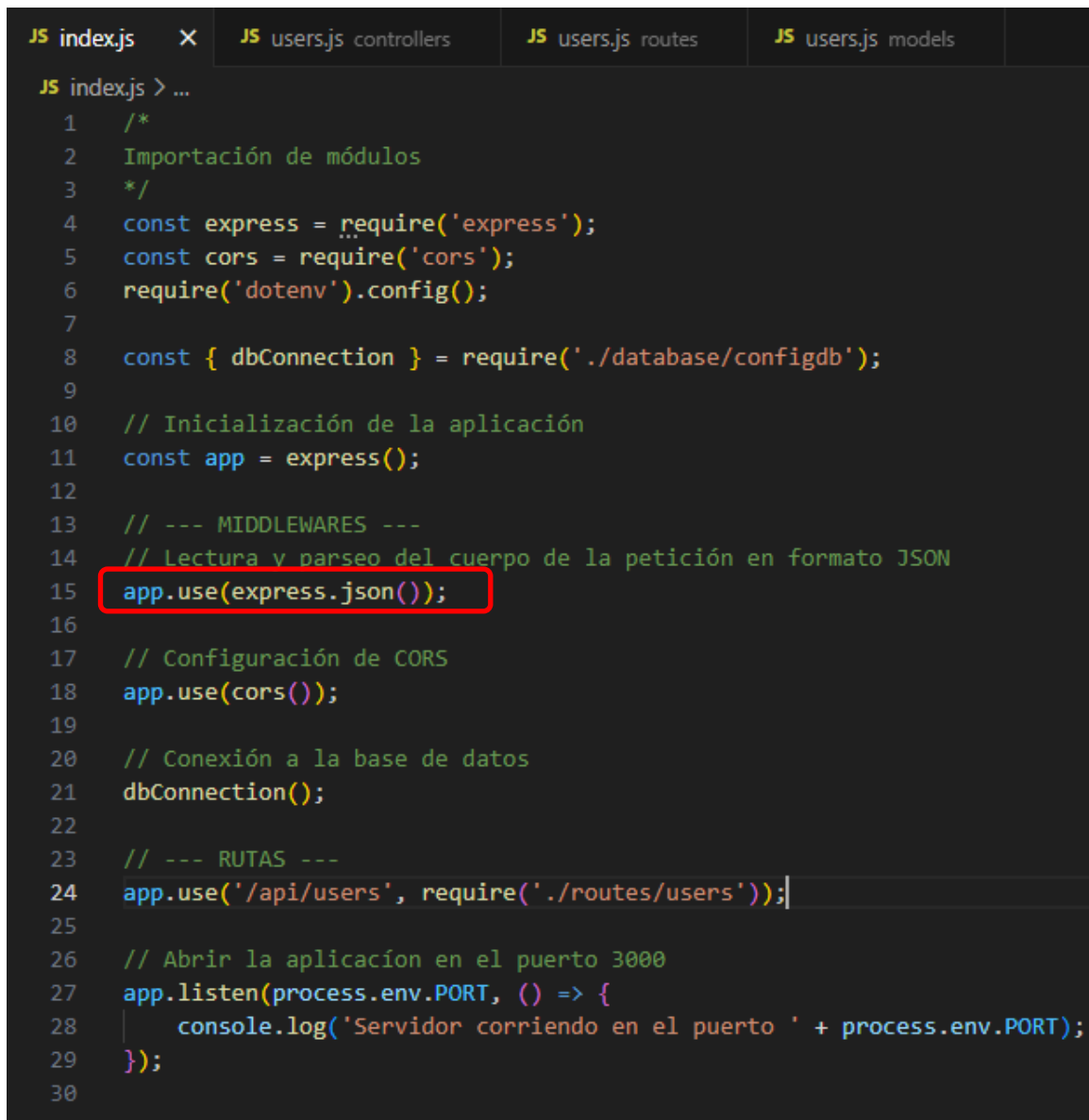


Figura 29. Crear cuerpo para la petición POST.

Para que más adelante podamos manejar los argumentos que nos llegan en la request, estos mismos que se envían en el body, en las cabeceras, en la URL, etc, es necesario que añadamos un middleware más dentro de los 'app.use'.

```
app.use(express.json());
```

Así debería quedar el archivo index.js:



```
JS index.js X JS users.js controllers JS users.js routes JS users.js models
JS index.js > ...
1  /*
2  Importación de módulos
3  */
4  const express = require('express');
5  const cors = require('cors');
6  require('dotenv').config();
7
8  const { dbConnection } = require('./database/configdb');
9
10 // Inicialización de la aplicación
11 const app = express();
12
13 // --- MIDDLEWARES ---
14 // Lectura y parseo del cuerpo de la petición en formato JSON
15 app.use(express.json());
16
17 // Configuración de CORS
18 app.use(cors());
19
20 // Conexión a la base de datos
21 dbConnection();
22
23 // --- RUTAS ---
24 app.use('/api/users', require('./routes/users'));
25
26 // Abrir la aplicación en el puerto 3000
27 app.listen(process.env.PORT, () => {
28   console.log('Servidor corriendo en el puerto ' + process.env.PORT);
29 });
30
```

Figura 30. Habilitar JSON para usarlo dentro de peticiones a la API con Express.

Al introducir este middleware ya podemos manejar todo lo que nos llega en la request como si de un json se tratase. También es más fácil detectar posibles errores porque la información que estamos enviando a través de POSTMAN no está debidamente formada. Si escribiéramos mal la solicitud, como esta, en la que falta una de las comas, detrás del atributo **password**, observaremos que nos da como respuesta un error muy feo:

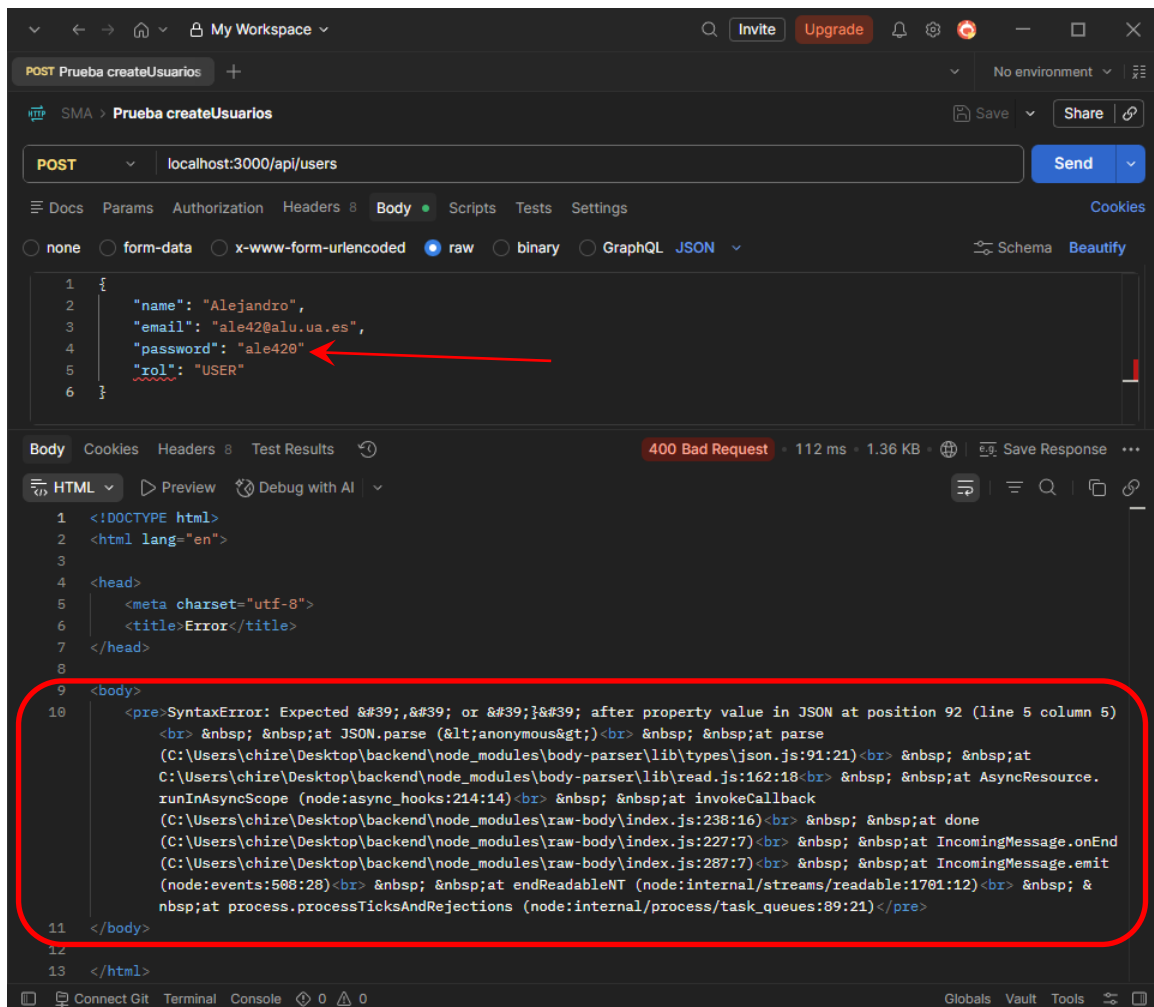


Figura 31. Error cuando la petición no incluye un body JSON bien formado.

Sin embargo, aunque ahora mismo podamos mandar la solicitud POST, nada ocurriría en nuestra BD, porque no tenemos un método creado para realizar la inserción en la base de datos en nuestro archivo `/models/users.js`. Por ello vamos a crear la función `createUser` en este archivo:

```
const createUser = async (userData) => {
  const db = getDb();
  const { name, email, password, role = 'USER' } = userData;

  const result = await db.run(
    "INSERT INTO users (name, email, password, role) VALUES (?, ?, ?, ?)",
    [name, email, password, role]
  );

  return {
    id: result.lastID,
    name,
    email,
    role
  };
};
```

Al terminar de añadirla, la tendremos que sumar a la lista de `module.exports` junto al resto de métodos.

Incluiremos también la función a nuestra función `createUser` en el archivo `/controllers/users.js`, que debería quedar así:

```
const createUser = async (req, res) => {  
  const { name, email, password, role } = req.body;  
  
  // errorVal almacena los errores que la función check adjuntó a la solicitud, que son extraídos por la función validationResult  
  const errorVal = validationResult(req);  
  if (!errorVal.isEmpty()) {  
    return res.status(400).json({  
      ok: false,  
      errores: errorVal.mapped()  
    });  
  }  
  
  // Guardamos el registro en la base de datos SQLite  
  const user = await User.createUser({  
    name,  
    email,  
    password,  
    role  
  });  
  
  res.json({  
    ok: true,  
    msg: "createUser",  
    user  
  })  
}
```

Figura 32. Archivo final con la inclusión del nuevo código.

De esta forma, cuando hagamos una solicitud POST correcta, el usuario se añadirá a la BD.

2.9.1. Validación de argumentos

El siguiente paso en la ruta es la validación de argumentos. Para validar argumentos una forma es en el controlador añadir lógica de negocio para comprobar que nos llega cada uno de los campos necesarios como el nombre, el email, etc. Pero esto sería mucho código y además muy repetitivo en cada ruta. Express nos provee de módulos que facilitan esto.

Para hacer una validación ágil existe una forma muy sencilla de hacerlo, utilizar **express-validator**. Para ello instalamos el módulo:

```
npm install express-validator
```

De este módulo nos interesaran algunos métodos, en concreto el método **check**, que permite comprobar si existe un cierto campo, si es de un cierto tipo u otras validaciones. Para eso importamos el método en `/routes/users.js`:

```
const { check } = require('express-validator');
```

Si miramos la documentación de **express-validator** podremos ver que el método **check** hará las comprobaciones sobre las diferentes partes de la petición como son `req.body`, `req.cookies`, `req.headers`, `req.params` y `req.query`. Si por ejemplo queremos examinar solo el `params` o en alguna cabecera, podemos utilizar otros métodos como el `'param'` o el `'header'`. Pero usando **check** nos aseguramos mirar en todo el contenido y no tenemos que diferenciar en que parte de la request está lo que buscamos. Si hay un error durante el **check** es decir, no se cumplen los requisitos del campo, la misma función añade un nuevo campo a la solicitud POST en el que incluye los errores que se han producido.

Y ahora hacemos uso de este módulo y se hace el control de validaciones. Para ello en `/routes/users.js` se modifica el `router.post` de esta forma:

```
router.post('/', [  
  // Comprueba que el json de la solicitud POST contiene un parámetro que se
```

```
llama "name" y que no está vacío.  
    check('name').not().isEmpty(),  
], createUser);
```

En el /controllers/users.js añadimos una nueva importación:

```
const { validationResult } = require('express-validator');
```

Con esta función podremos extraer los errores que ocurrieron en la función check.

Y al inicio de la función createUser añadimos la comprobación de los errores detectados:

```
// errorVal almacena los errores que la función check adjuntó a la solicitud,  
que son extraídos por la función validationResult  
const errorVal = validationResult(req);  
if (!errorVal.isEmpty()) {  
    return res.status(400).json({  
        ok: false,  
        errores: errorVal.mapped()  
    });  
}
```

Para poner a prueba nuestra validación del campo **"name"** modificaremos nuestra solicitud POST para que contenga un error de escritura en el campo **"name"** y pondremos algo como **"name2"** para que nos salte el error. Deberíamos de ver algo así:

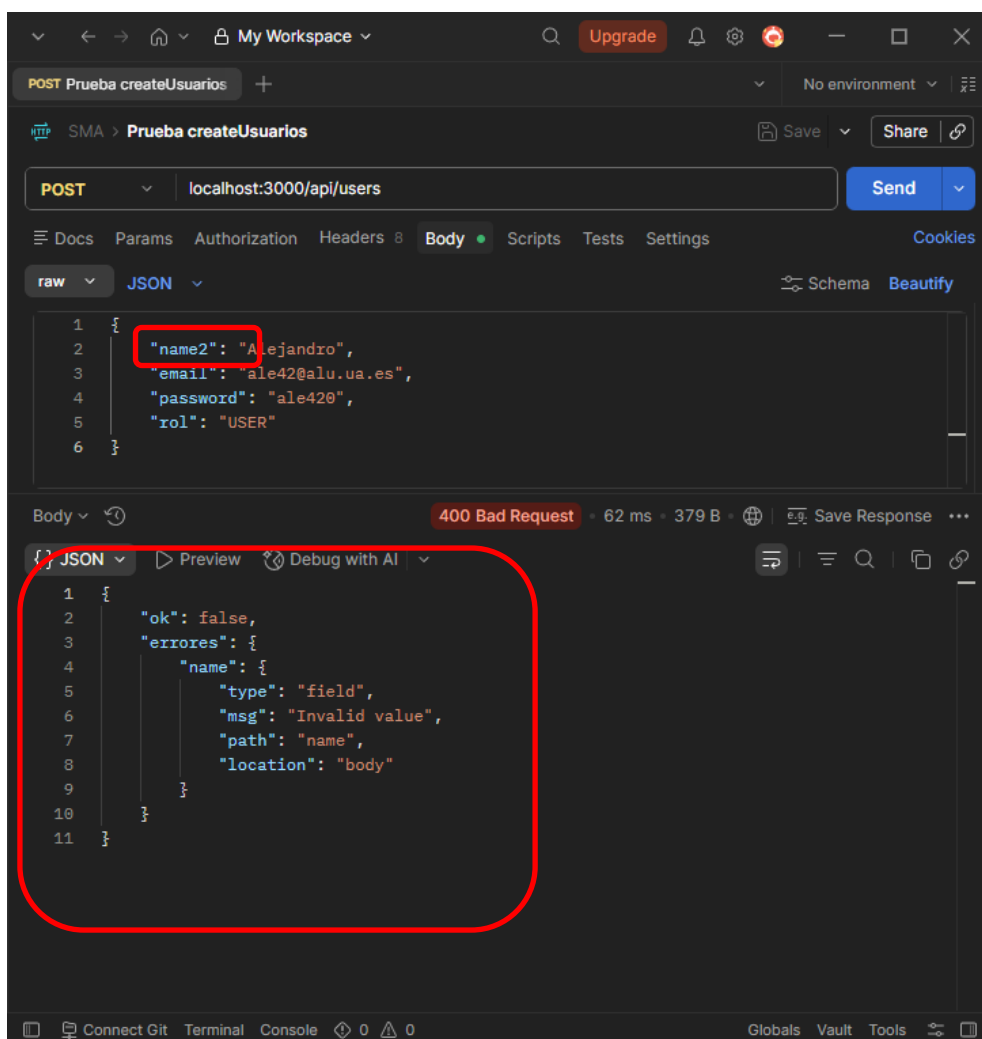


Figura 33. Detección de argumento inválido..

Esta respuesta nos comunica que hemos aportado un campo que no se cumple con lo que espera la función `check`.

Sin embargo, el código de validación de argumentos de la función `createUser` en `/controller/users.js` solo serviría para esa función, y si quisiéramos hacer lo mismo en otra tendríamos que copiar y pegar el mismo código.

Añadir un mismo código repetido una y otra vez sobre los métodos solo indica una cosa, que eso debe hacerse de otra forma. Nunca hay que repetir código pues si en un momento dado tenemos que hacer un cambio tendríamos buscar todas esas porciones de código y porque se genera un código sucio difícil de leer. Para ello vamos a crear nuestro propio middleware, una especie de función similar a la `'check'` pero que nos resuelva este mismo código de comprobación de errores.

2.9.2. Middleware personalizado VALIDATEERRORS

Creamos una nueva carpeta llamada `middleware` (puede llamarse como queráis), y en ella creamos un archivo llamado `validate-fields.js`. Este archivo va a contener el siguiente código:

```
const { response } = require('express');
const { validationResult } = require('express-validator');

const validateFields = (req, res = response, next) => {
  const errorVal = validationResult(req);
  if (!errorVal.isEmpty()) {
    return res.status(400).json({
      ok: false,
      errores: errorVal.mapped()
    });
  }
  next();
}

module.exports = { validateFields }
```

El código de la función `validateFields` se corresponde con el que habíamos puesto en la función `createUser` del archivo `/controllers/users.js`, el cual deberemos borrar, ya que la función `validateFields` lo sustituye. El archivo `/controllers/users.js` debería de quedar así:

```
const createUser = async (req, res) => {  
  const { name, email, password, role } = req.body;  
  
  // Guardamos el registro en la base de datos SQLite  
  const user = await User.createUser({  
    name,  
    email,  
    password,  
    role  
  });  
  
  res.json({  
    ok: true,  
    msg: "createUser",  
    user  
  })  
}
```

Figura 34. Archivo users.js con el código limpio.

Este código permite recibir la req, comprobar si hay errores, en caso de que haya errores devuelve la res con los errores y en caso de que no haya llama a la función next. Next en este caso va a servir para saltar al siguiente paso, ya que la forma de utilizar este middleware es dentro del /routes/users.js, añadiéndolo tras el check así:

```
// Realiza el check y después llama a la función validateFields para que com-  
pruebe si ha habido algún error y nos lo devuelva en la respuesta  
router.post('/', [  
  check('name', 'El argumento name es obligatorio').not().isEmpty(),  
  validateFields  
, createUser];
```

Para que validateFields funcione tendremos que añadir la importación del archivo /middleware/validate-fields al archivo /routes/users.js detrás del resto de importaciones:

```
const { validateFields } = require('../middleware/validate-fields');
```

2.10. 1.8. Otras validaciones y rutas PUT y DELETE

Ahora que ya tenemos un esqueleto con algunas rutas, vamos a completar el resto de rutas para tener una API CRUD completa y a añadir las validaciones necesarias de los argumentos que nos llegan.

Para ello añadiremos en /routes/users.js esto:

```
router.post('/', [  
  check('name', 'El argumento name es obligatorio').not().isEmpty(),  
  check('email', 'El argumento email es obligatorio').not().isEmpty(),  
  check('password', 'El argumento password es obligatorio').not().isEmpty(),  
  validateFields  
, createUser];  
router.put('/:id', [  
  check('name', 'El argumento name es obligatorio').not().isEmpty(),  
  check('email', 'El argumento email es obligatorio').not().isEmpty(),  
  check('password', 'El argumento password es obligatorio').not().isEmpty(),  
  check('id', 'El identificador no es válido').isInt(),
```

```
    validateFields
  ], updateUser);
router.delete('/:id', [
  check('id', 'El identificador no es válido').isInt(),
  validateFields
], deleteUser);
```

Y en `/controllers/users.js` lo siguiente:

```
const updateUser = async(req, res) => {
  res.json({
    ok: true,
    msg: 'updateUser'
  });
}
const deleteUser = async(req, res) => {
  res.json({
    ok: true,
    msg: 'deleteUser'
  });
}
module.exports = { getUsers, createUser, updateUser, deleteUser }
```

Para que la aplicación no dé error al iniciar, deberemos de añadir las nuevas funciones a las importaciones que ya teníamos puestas en el archivo `/routes/users.js`:

```
const { getUsers, createUser } = require('../controllers/users');
```

de forma que quedará así:

```
const { getUsers, createUser, updateUser, deleteUser } = require('../controllers/users');
```

Vamos a comprobar que las nuevas funciones actúan correctamente con POSTMAN. Si modificamos la solicitud POST que habíamos usado anteriormente para crear a nuestro primer usuario y la dejamos vacía, podemos observar que cuando la enviemos, nos dará un error como este:

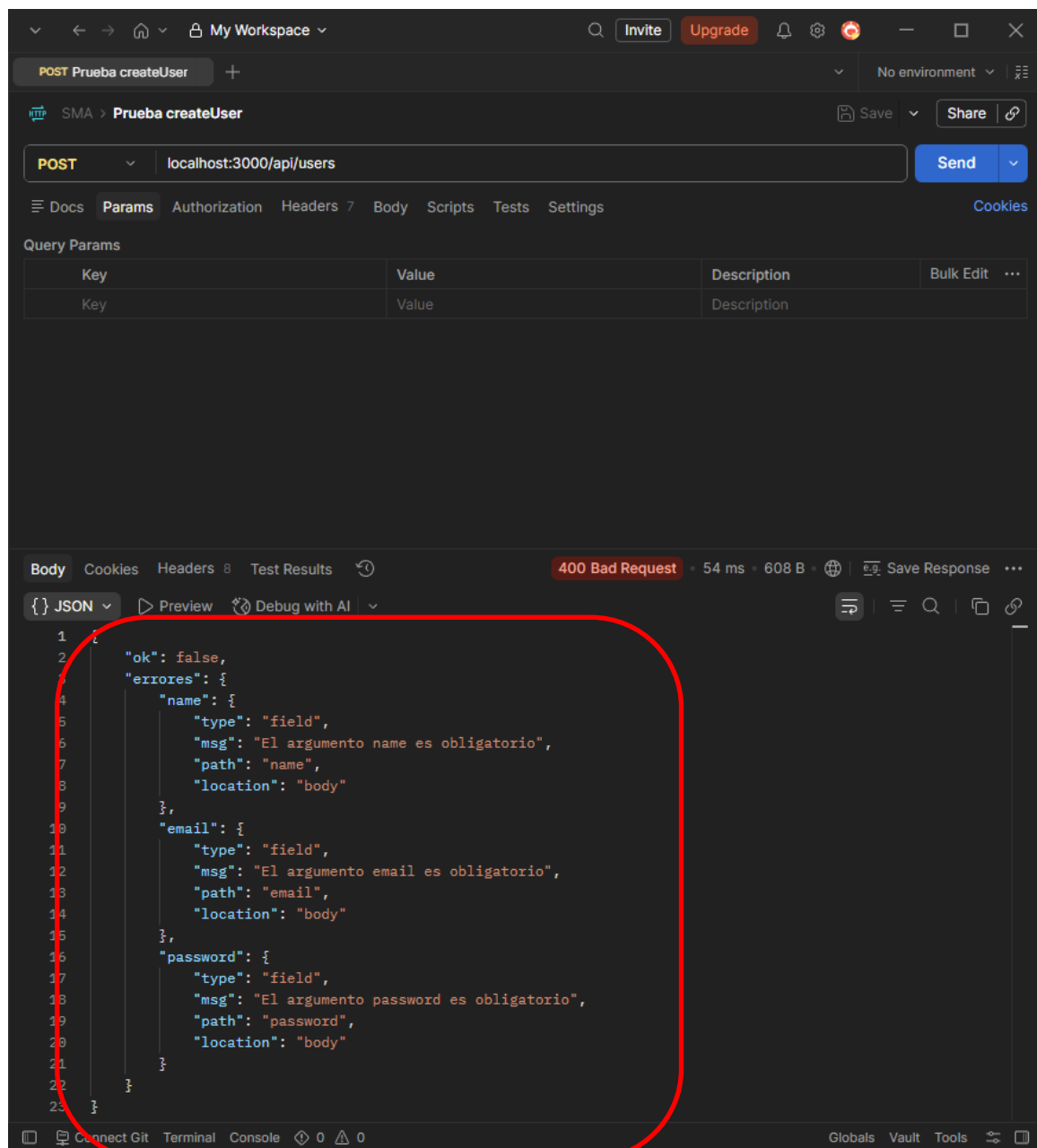


Figura 35. Error generado cuando no hay datos en la petición.

Esto nos indica que nuestra validación de errores funciona correctamente en la función POST. Vamos con la PUT. Creamos una nueva solicitud duplicando la de POST que teníamos creada y cambiamos el tipo de la solicitud a PUT. Si probamos a enviarla, veremos que nos dará un error distinto:

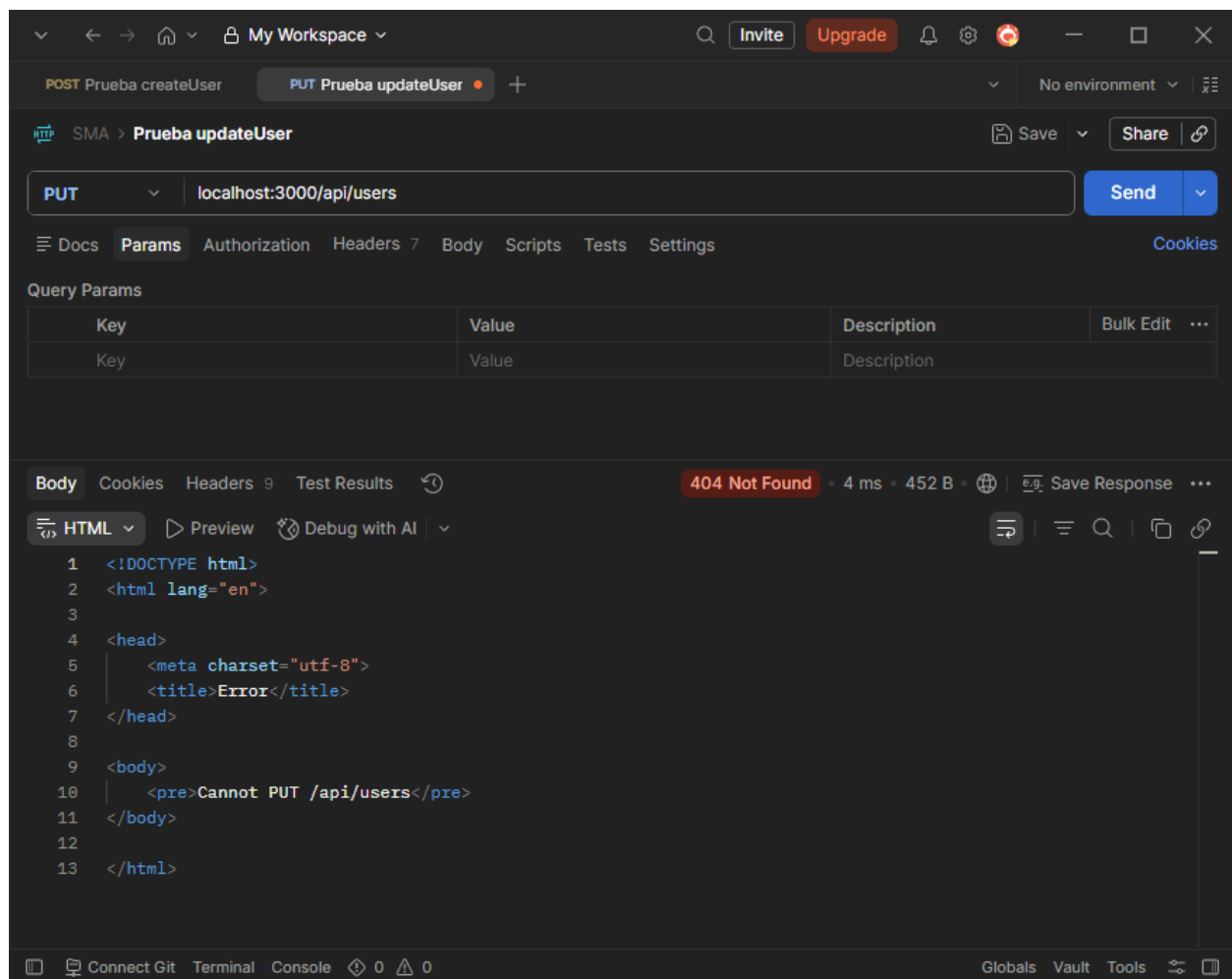


Figura 36. Error al hacer una petición PUT a la que faltan el ID principal.

Esto se debe a que, como hemos visto antes, nuestra solicitud PUT necesita una id de usuario, y no se la hemos proporcionado. Si revisamos nuestra base de datos con DB Browser, podemos observar el campo de id del usuario que creamos con anterioridad y comprobar que nuestra validación funciona, y debería de salir esto:

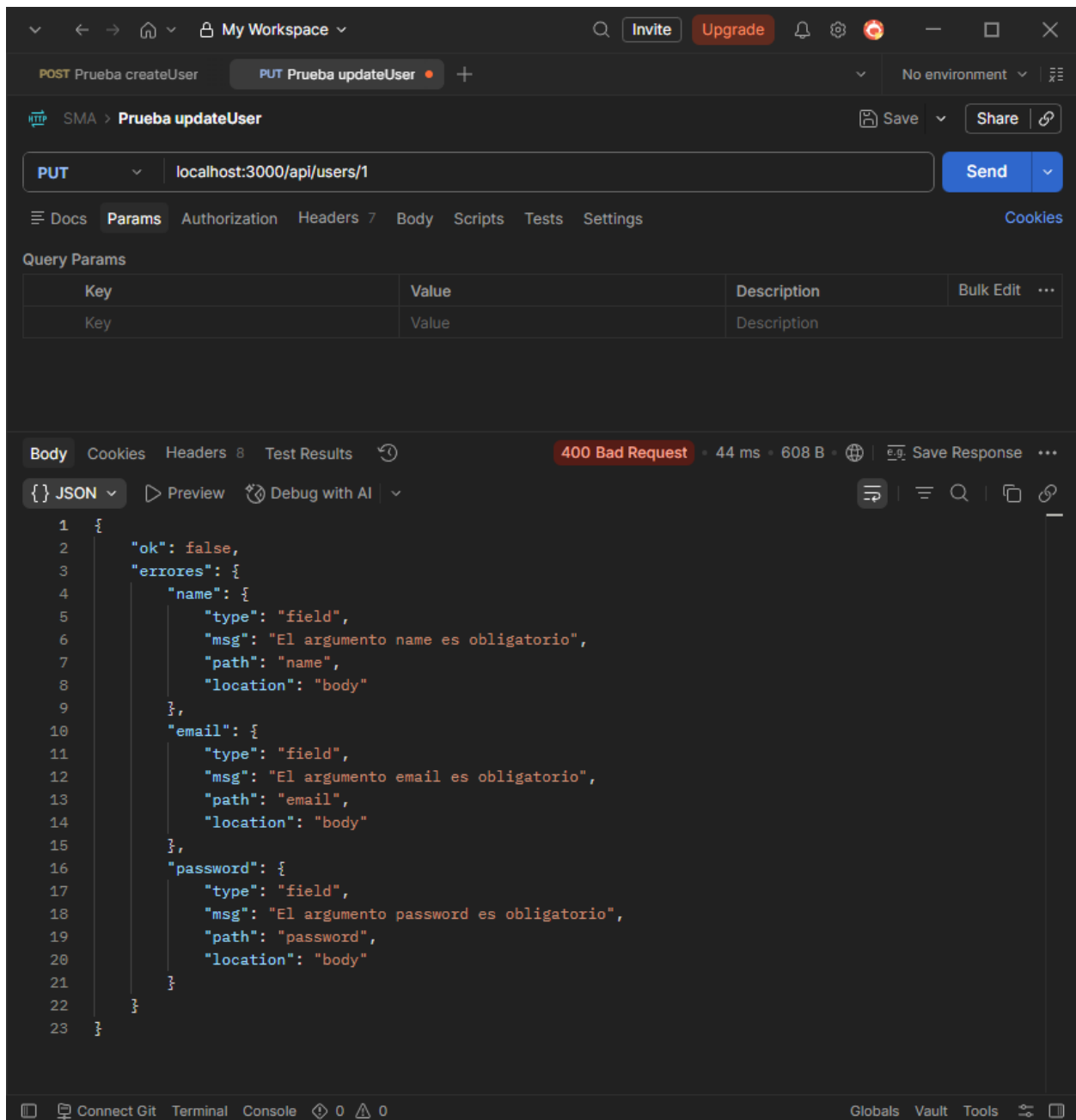


Figura 37. Error al hacer una petición PUT a la que faltan argumentos.

Nuestra solicitud PUT funciona correctamente también, si queremos comprobar si la validación del campo id funciona también, deberemos de colocar letras o caracteres que no sean números, de esa forma debería de darnos un error con un aviso también.

Para comprobar que nuestra solicitud DELETE funciona correctamente duplicaremos nuestra solicitud PUT y le cambiaremos el tipo para que sea de tipo DELETE. Al ya tener puesta la id en la dirección de la solicitud, si la enviamos podremos ver que funciona correctamente:

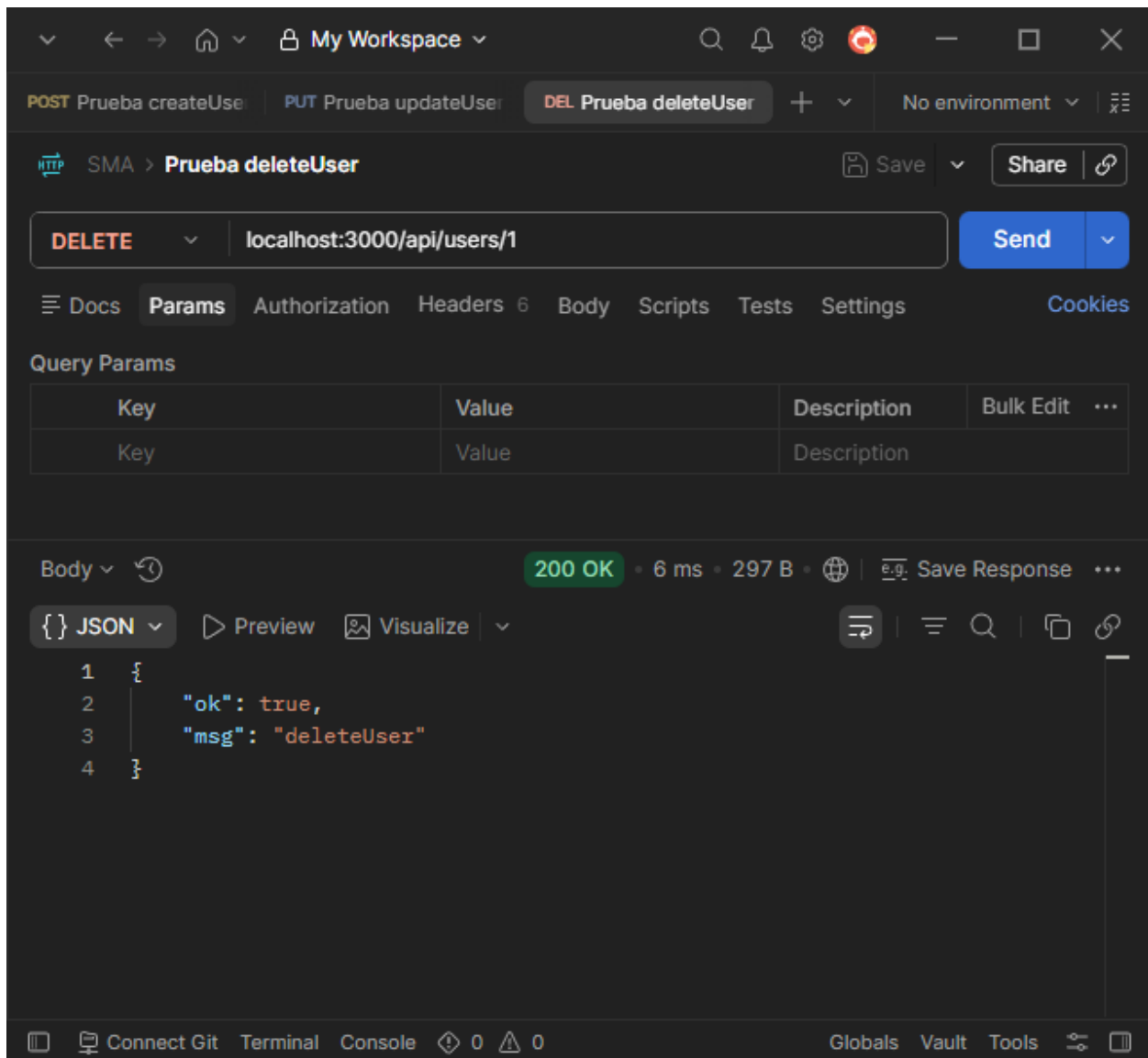


Figura 38. Petición correcta a DELETE.

2.11. POST: comprobar email, cifrar contraseña y almacenar

La ruta POST que tenemos hasta ahora no hace nada, solo devolver un mensaje para informar de que ha sido invocada.

Una ruta POST tiene la finalidad de dar de alta un nuevo registro, en este caso usuario. Para ello ha de comprobar que se recibe toda la información necesaria (esto ya se hace en el router), y después debe insertar el registro pero:

- Debe comprobar primero que no existe ya un registro dado de alta con ese mismo email
- Debe cifrar la contraseña. Las contraseñas deben cifrarse siempre de forma que en caso de que alguien acceda a la base de datos, nunca tenga en realidad la contraseña de los usuarios.

2.11.1. Comprobar email

Para evitar que se registren usuarios duplicados, debemos realizar una consulta previa a la base de datos que compruebe si ya existe un registro con el mismo correo electrónico. Si el correo ya se encuentra en uso, interrumpimos el proceso devolviendo un mensaje de error explicativo con el código de estado HTTP 400

(Bad Request). Si no existe, la ejecución continúa con el flujo habitual.

2.11.2. Consulta en el modelo (/models/users.js)

En el módulo del modelo añadimos la función encargada de consultar la base de datos SQLite. Utilizamos el método `db.get()`, el cual devuelve un único registro si encuentra coincidencia o `undefined` si no existe ninguna fila con ese correo:

```
const getUserByEmail = async (email) => {  
  const db = getDb();  
  return await db.get("SELECT * FROM users WHERE email = ?", [email]);  
};
```

2.11.3. Validación en el controlador (/controllers/users.js)

En la función `createUser` del controlador desestructuramos los campos necesarios de `req.body` y ejecutamos la verificación llamando a la función del modelo:

```
const { email, password } = req.body;  
const emailExists = await User.getUserByEmail({ email: email });  
if (emailExists) {  
  return res.status(400).json({  
    ok: false,  
    msg: 'Email ya existe'  
  });  
}
```

La función `createUser` debería de verse así:

```
const createUser = async (req, res) => {
  const { name, email, password, role } = req.body;

  try {
    // Comprobamos si el email ya existe en la BD
    const emailExists = await User.getUserByEmail(email);
    if (emailExists) {
      return res.status(400).json({
        ok: false,
        msg: "Email ya existe"
      });
    }

    // Guardamos el registro en la base de datos SQLite
    const user = await User.createUser({
      name,
      email,
      password,
      role
    });

    res.json({
      ok: true,
      msg: "createUser",
      user
    });
  } catch (error) {
    console.log(error);
    return res.status(400).json({
      ok: false,
      msg: "Error creando usuario"
    });
  }
}
```

Figura 39. Función *createUser*.

Si la variable `emailExists` obtiene un resultado de la base de datos, la condición del `if` se evalúa como verdadera, deteniendo la petición y enviando la respuesta de error en formato JSON. En caso contrario, el código omite la estructura condicional y continúa con el guardado del nuevo registro.

2.11.4. Cifrar la contraseña

El siguiente paso es cifrar la contraseña. Para eso vamos a utilizar la librería `bcryptjs`. Para poder usarla primero la instalamos:

`npm install bcryptjs`

Después en `/controllers/users.js` la importamos:

```
const bcrypt = require('bcryptjs');
```

Y la usamos de la siguiente forma:

```
const salt = bcrypt.genSaltSync(); // generamos un salt, una cadena aleatoria
password = bcrypt.hashSync(password, salt); // y aquí ciframos la contraseña
```

La función `createUser` debería de quedar así:

```
const createUser = async (req, res) => {
  const { name, email, password, role } = req.body;

  try {
    const emailExists = await User.getUserByEmail(email);
    if (emailExists) {
      return res.status(400).json({
        ok: false,
        msg: "Email ya existe"
      });
    }

    // Generamos un salt, una cadena aleatoria
    const salt = bcrypt.genSaltSync();
    // y aquí ciframos la contraseña
    const cpassword = bcrypt.hashSync(password, salt);

    // Guardamos el registro en la base de datos SQLite con la nueva contraseña cifrada
    const user = await User.createUser({
      name,
      email,
      password: cpassword,
      role
    });

    res.json({
      ok: true,
      msg: "createUser",
      user,
      // Devolvemos la password original
      pass: password,
      // y la password cifrada para ver las diferencias
      cpass: cpassword
    });
  } catch (error) {
    console.log(error);
    return res.status(400).json({
      ok: false,
      msg: "Error creando usuario"
    });
  }
}
```

Figura 40. Función `createUser` con cifrado.

Para comprobar que funciona correctamente, utilizaremos la solicitud POST que ya teníamos creada en POSTMAN de la siguiente forma, y nos dará el resultado que se ve en la imagen:

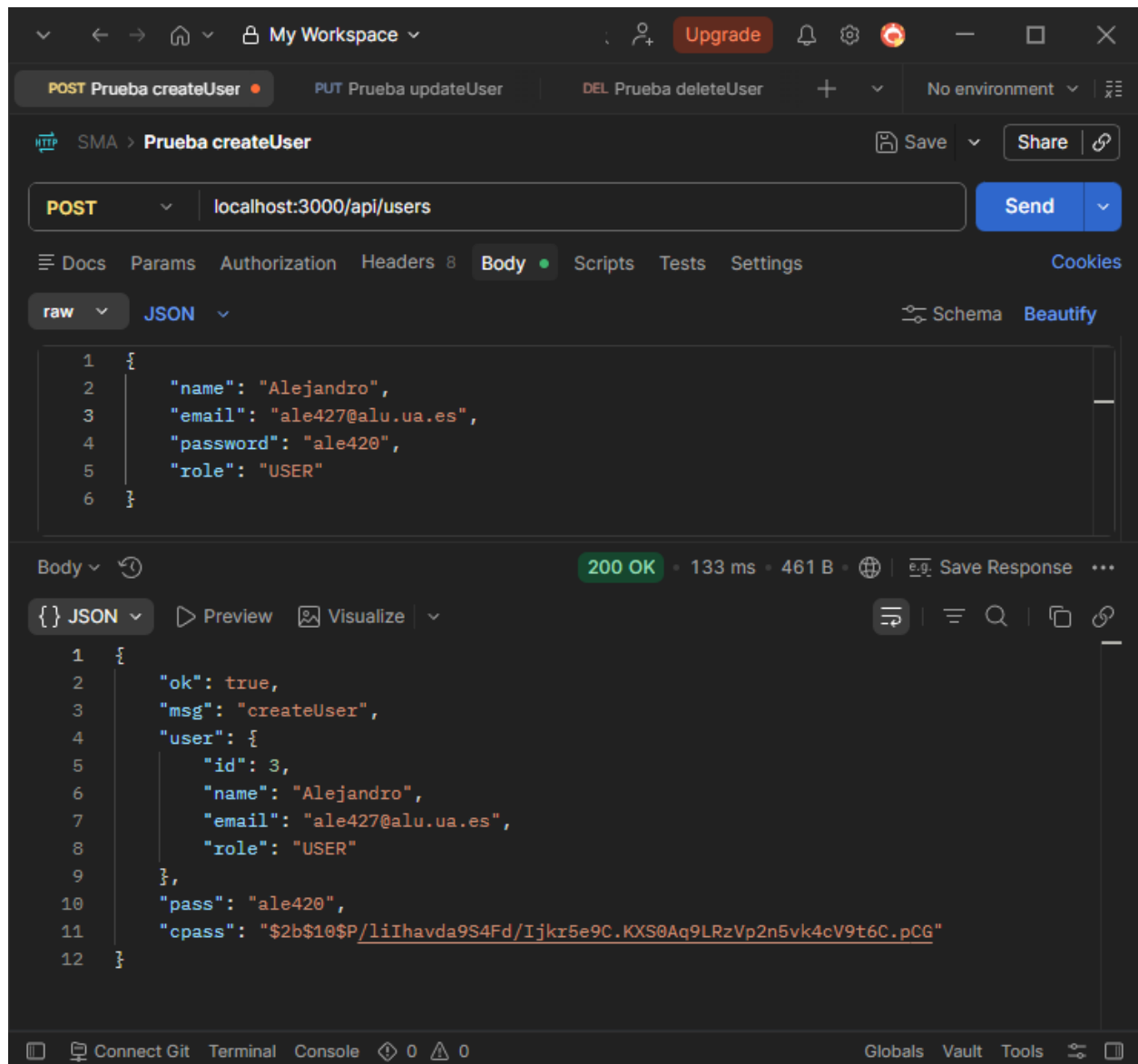


Figura 41. Resultado de la función en Postman.

2.12. Ocultación de campos sensibles en las respuestas JSON

Para garantizar la seguridad de la aplicación, debemos asegurarnos de que la información confidencial (como la contraseña almacenada) nunca se envíe en la respuesta JSON al frontend.

Para llevar a cabo esta tarea, tendremos que crear una función nueva en el archivo `/models/users.js`, llamada `formatUser`, que contendrá el siguiente código:

```
// Función personalizada para limpiar datos sensibles del usuario
const formatUser = (user) => {
  if (!user) return null;

  // Extraemos password para descartarlo y nos quedamos con el resto
  const { password, ...cleanUser } = user;
  return cleanUser;
};
```

Esta función la llamaremos en el return de todas las funciones del archivo `/models/users.js`, consiguiendo de

esta forma que ninguno exponga datos sensibles cuando sea llamado. Así debería de quedar el archivo `/models/users.js`:

```
const { getDb } = require('../database/configdb');

// Función para inicializar la tabla 'users' si no existe
const initUsersTable = async () => {
  const db = getDb();
  await db.exec(`
    CREATE TABLE IF NOT EXISTS users (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      email TEXT NOT NULL UNIQUE,
      password TEXT NOT NULL,
      role TEXT NOT NULL DEFAULT 'USER',
      reg_date TEXT NOT NULL DEFAULT (datetime('now')),
      token TEXT
    );
  `);
};

const createUser = async (userData) => {
  const db = getDb();
  const { name, email, password, role = 'USER' } = userData;

  const result = await db.run(
    "INSERT INTO users (name, email, password, role) VALUES (?, ?, ?, ?)",
    [name, email, password, role]
  );

  // Incluimos el id autogenerado junto con los datos recibidos
  return formatUser({
    id: result.lastID,
    ...userData,
    role
  });
};

const getUsers = async () => {
  const db = getDb();
  const users = await db.all("SELECT * FROM users");
  return users.map(user => formatUser(user));
};

const getUserByEmail = async (email) => {
  const db = getDb();
  const user = await db.get("SELECT * FROM users WHERE email = ?", [email]);
  return formatUser(user);
};

// Función personalizada para limpiar datos sensibles del usuario
const formatUser = (user) => {
  if (!user) return null;

  // Extraemos password para descartarlo y nos quedamos con el resto
  const { password, ...cleanUser } = user;
```

```
    return cleanUser;
};

module.exports = {
  initUsersTable,
  createUser,
  getUsers,
  getUserByEmail
};
```

Podemos observar con una solicitud GET que ya teníamos creada de pasos anteriores, que la respuesta ya no contiene la contraseña de los usuarios, lo que significa que hemos conseguido eliminarla de la respuesta, y la estamos protegiendo correctamente:

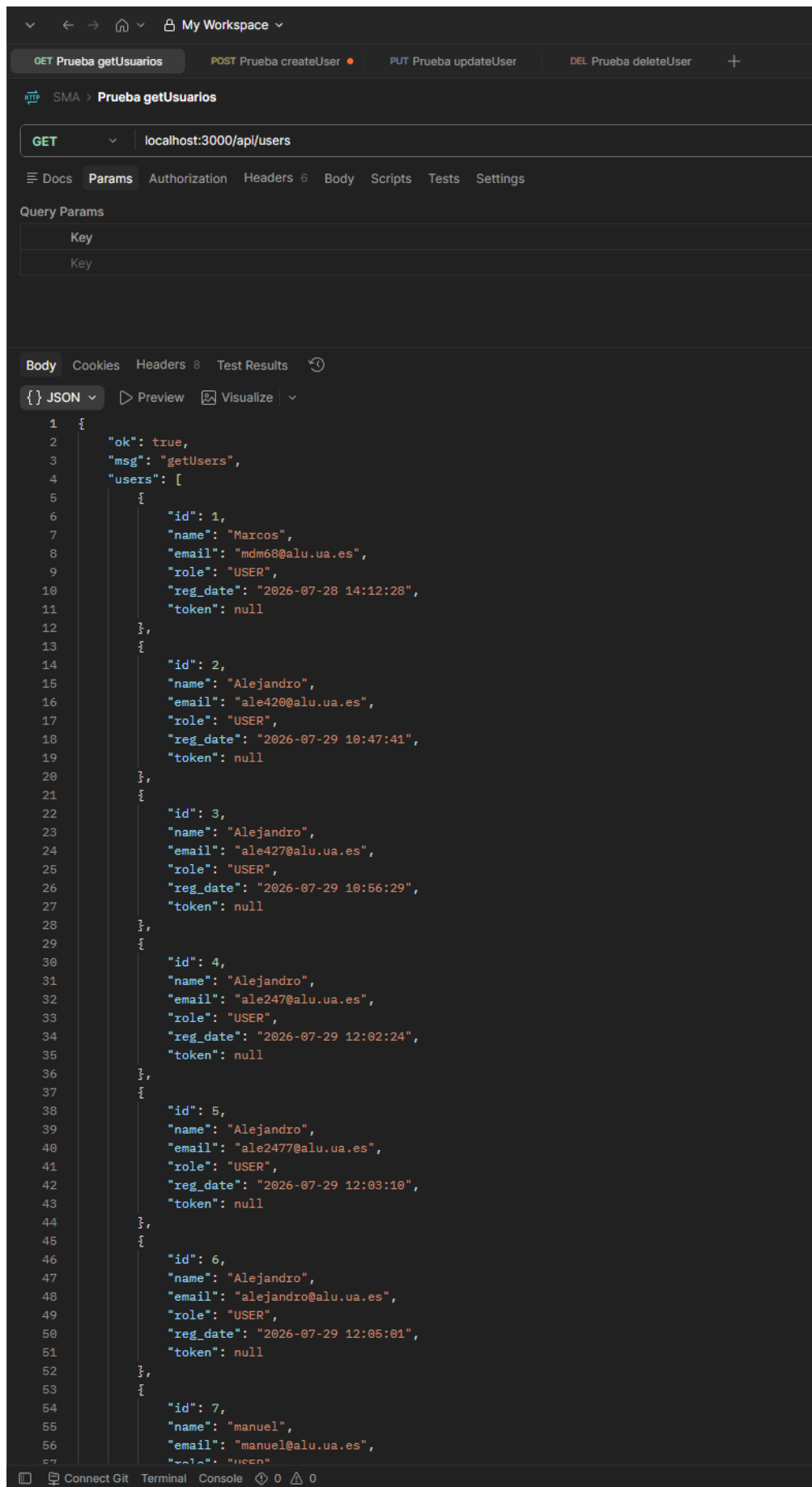


Figura 42. Resultado de la función en Postman.

2.13. Descarga Código 2

Todo lo desarrollado hasta ahora está disponible en el "Codigo2.zip" que puedes descargar desde aquí:

[Descargar Código 2](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo2.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo2.zip>

2.14. Middleware comprobar ROL, PUT y DELETE

Ahora vamos a programar algunas optimizaciones y los métodos PUT y DELETE.

2.14.1. Comprobación de ROL

Al igual que hicimos en su momento con el archivo /middleware/validate-fields.js, vamos a crear uno nuevo en esa misma carpeta pero para comprobar que el ROL del usuario existe y es válido, lo llamaremos "validate-role.js". El código de este archivo es este:

```
// Importamos response de la librería express para tener los métodos de "res"
const { response } = require('express');

// Lista de roles permitidos.
// Si alguien envía un rol que no está aquí, dará error.
const roleList = [ 'USER', 'ADMIN' ];

// Comprueba si el rol enviado es válido
const validateRole = (req, res = response, next) => {
  const role = req.body.role;

  if (role && !roleList.includes(role)) {
    return res.status(400).json({
      ok: false,
      msg: 'Rol no permitido'
    });
  }

  next();
};

module.exports = {
  validateRole
};
```

Este método habría que colocarlo en las solicitudes del archivo /routes/users.js de la siguiente forma:

```

const { Router } = require('express');
const { check } = require('express-validator');
const { getUsers, createUser, updateUser, deleteUser } = require('../controllers/users');
const { validateFields } = require('../middleware/validate-fields');
const { validateRole } = require('../middleware/validate-role');

const router = Router();

router.get('/', getUsers);

router.post('/', [
  check('name', 'El argumento name es obligatorio').not().isEmpty(),
  check('email', 'El argumento email es obligatorio').not().isEmpty(),
  check('password', 'El argumento password es obligatorio').not().isEmpty(),
  validateFields,
  validateRole
], createUser);

// Tanto el PUT como el DELETE que estamos empleando necesitan una id de usuario, que estaría en la URL. Esto se puede saber observando que en vez de "/" pone("/:id)".
// Ejemplo: /api/users/123456 sería la id que se le pasaría a la request como parámetro del campo id.

// Este método actualiza los campos de un usuario ya existente, que se especificaría con la id correspondiente.
router.put('/:id', [
  check('name', 'El argumento name es obligatorio').not().isEmpty(),
  check('email', 'El argumento email es obligatorio').not().isEmpty(),
  check('password', 'El argumento password es obligatorio').not().isEmpty(),
  check('id', 'El identificador no es válido').isInt(),
  validateFields,
  validateRole
], updateUser);

// Este método borra un usuario existente, que se especificaría con la id correspondiente.
router.delete('/:id', [
  check('id', 'El identificador no es válido').isInt(),
  validateFields
], deleteUser);

module.exports = router;

```

Figura 43. Rutas con validación de roles.

2.14.2. Implementación del método PUT

Para gestionar la actualización de los datos de un usuario existente, se implementa la ruta PUT `/:id`. Esta operación permite modificar campos generales de la información del usuario en la base de datos garantizando la integridad de los datos y la seguridad del sistema.

Para llevar a cabo esta funcionalidad se requiere coordinar el módulo del modelo y el controlador del módulo de usuarios.

2.14.3. Definición de las consultas en el modelo (models/users.js)

En el modelo de datos se definen dos funciones necesarias para el proceso de actualización:

1. **getUserById**: Recupera un registro específico utilizando su identificador único.

```

const getUserById = async (id) => {
  const db = getDb();
  const user = await db.get("SELECT * FROM users WHERE id = ?", [id]);
  return formatUser(user);
}

```

2. **updateUser**: Ejecuta la sentencia SQL UPDATE para guardar los nuevos campos y retorna la información actualizada omitiendo los datos sensibles.

```

// Actualizar los datos del usuario en la base de datos
const updateUser = async (id, userData) => {
  const db = getDb();
  const { name, email, role = 'USER' } = userData;

  await db.run(
    "UPDATE users SET name = ?, email = ?, role = ? WHERE id = ?",
    [name, email, role, id]
  );
}

```

```
const updatedUser = await getUserById(id);
return formatUser(updatedUser);
};
```

2.14.4. Lógica de negocio en el controlador (controllers/users.js)

La función `updateUser` dentro del controlador gestiona la recepción de la petición, aplica las comprobaciones de seguridad y coordina la persistencia en la base de datos siguiendo esta secuencia de pasos:

1. **Protección de contraseña:** Se desestructura el cuerpo de la petición (`req.body`) separando explícitamente el campo `password`. De este modo, la contraseña queda descartada y no se permite su modificación a través de esta ruta.
2. **Verificación de existencia:** Se comprueba si el identificador recibido en la URL (`req.params.id`) corresponde a un usuario registrado. Si no existe, se retorna un código de estado HTTP 404 Not Found.
3. **Control de duplicidad de correo electrónico:** Se verifica si el nuevo correo enviado ya está registrado por otro usuario. Si el correo pertenece a un usuario diferente, se interrumpe el proceso con un error HTTP 400 Bad Request. Si el correo pertenece al mismo usuario en modificación, la validación se aprueba.
4. **Ejecución y respuesta:** Se envía el objeto con los datos procesados a la función del modelo y se retorna el usuario actualizado en formato JSON.

```
const updateUser = async (req, res) => {
  const { id } = req.params;

  // Descartamos la contraseña por seguridad y extraemos el email para su validación
  const { password, email, ...object } = req.body;

  try {
    // Comprobamos la existencia del usuario
    const user = await User.getUserById(id);
    if (!user) {
      return res.status(404).json({
        ok: false,
        msg: "No existe un usuario con ese identificador"
      });
    }

    // Comprobamos la disponibilidad del correo electrónico
    const emailExists = await User.getUserByEmail(email);
    if (emailExists && emailExists.id !== Number(id)) {
      return res.status(400).json({
        ok: false,
        msg: "El correo electrónico ya pertenece a otro usuario"
      });
    }

    // Asignamos el correo verificado
    object.email = email;
```

```
// Ejecutamos la actualización
const updatedUser = await User.updateUser(id, object);

res.json({
  ok: true,
  msg: "updateUser",
  user: updatedUser
});

} catch (error) {
  console.log(error);
  return res.status(500).json({
    ok: false,
    msg: "Error actualizando el usuario"
  });
}
};
```

Para comprobar que las funciones van correctamente, tenemos que obtener primero el identificador de un usuario, con la solicitud GET de todos los usuarios de la siguiente forma:

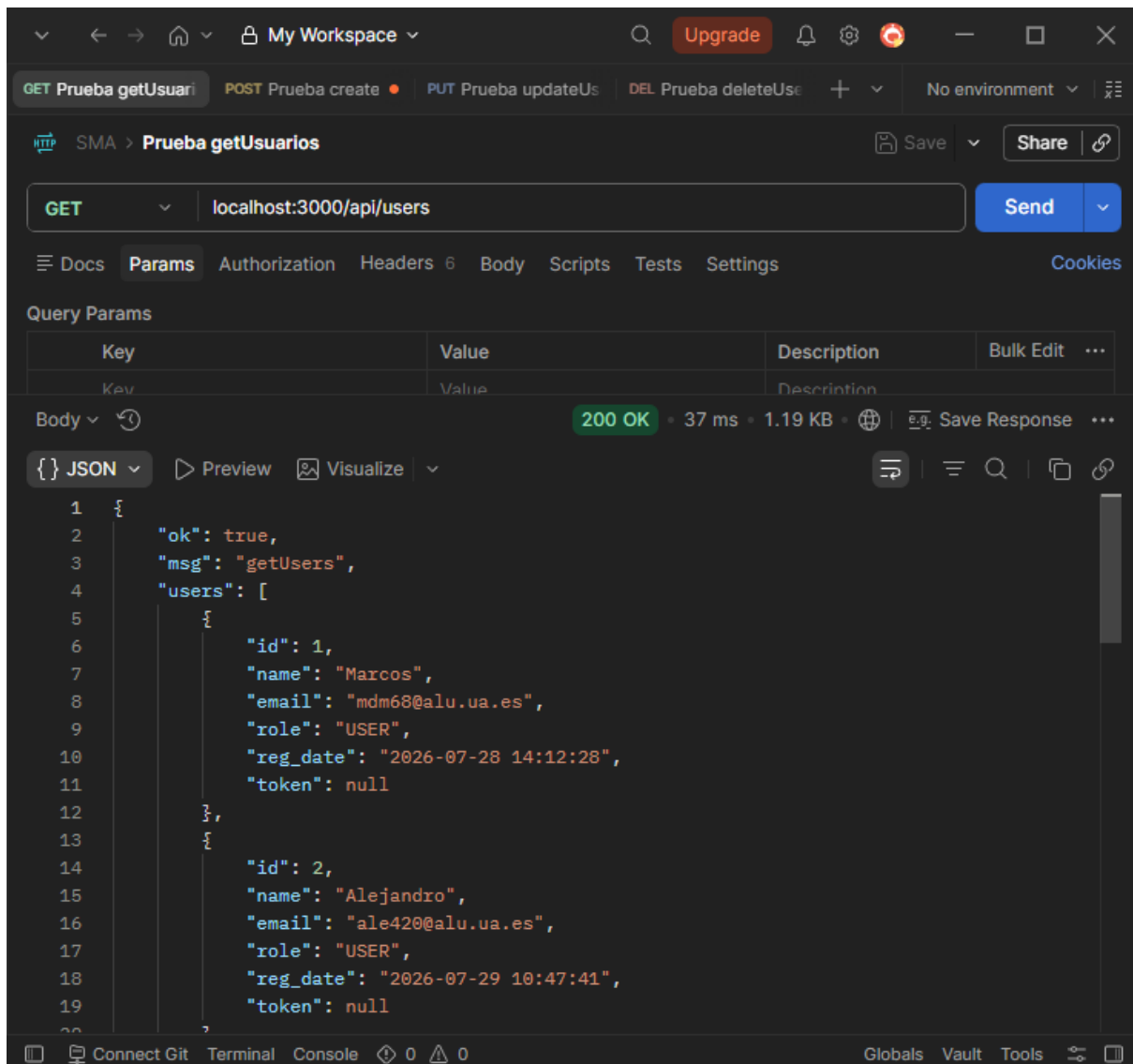


Figura 44. Resultado de la ejecución de la función.

Con los resultados de la solicitud obtenemos las ids de los usuarios de nuestra BD. Cogeremos la id del usuario que queramos modificar y lo ponemos en la dirección de nuestra solicitud PUT que teníamos creada, ponemos en el body un correo que ya existe en la BD y tendría que salirnos este error:

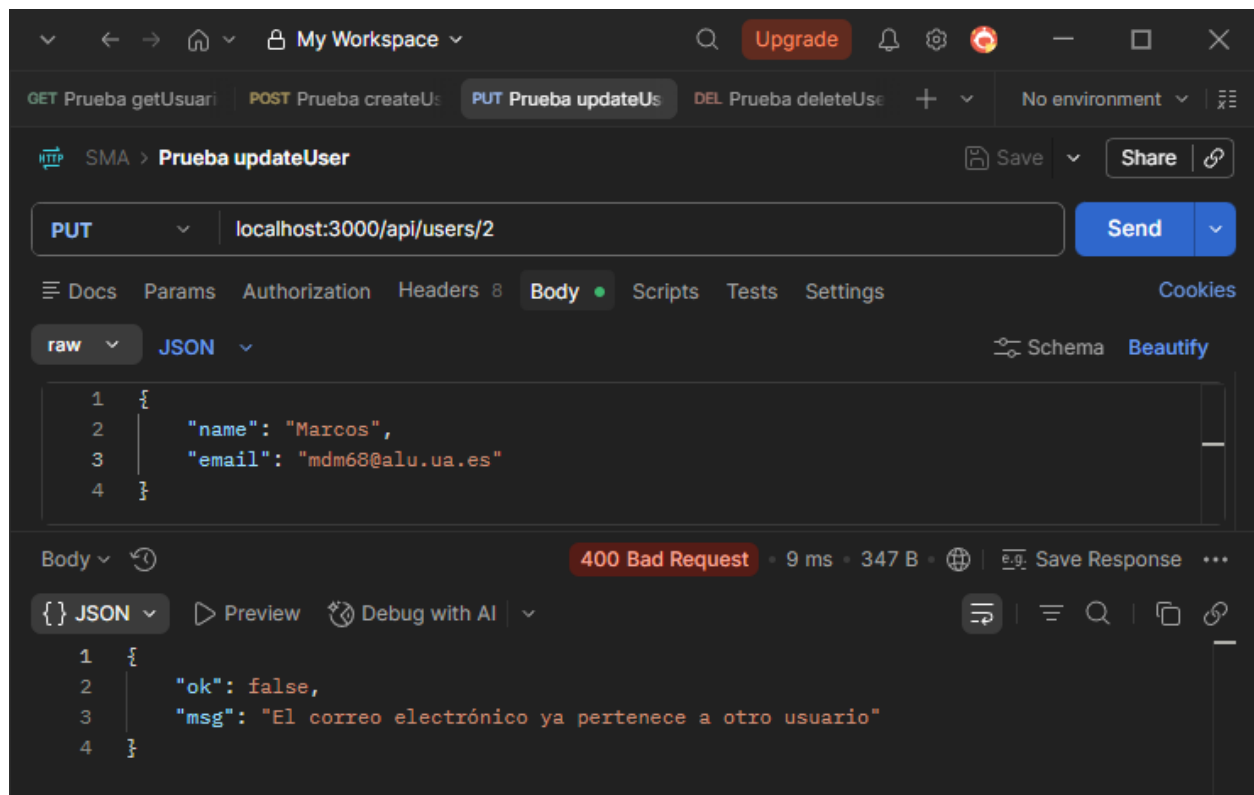


Figura 45. Construcción de BODY.

Ahora probaremos con un correo que no exista en la BD, para verificar si el cambio se produce correctamente y se guarda en la BD como es debido. Deberíamos de ver un mensaje similar a este:

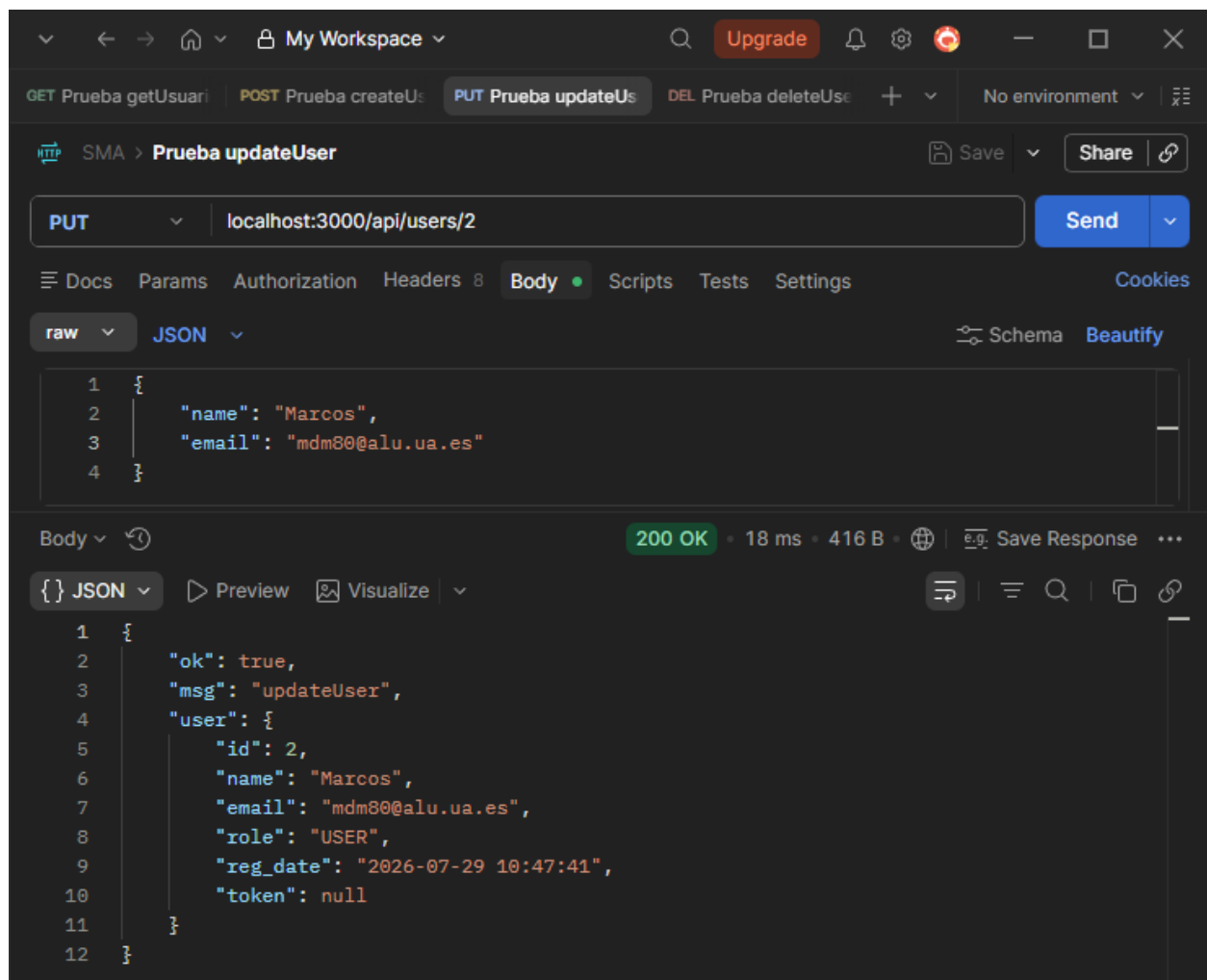


Figura 46. Resultado de invocar el endpoint.

Y si buscamos el id del usuario ("2" en este caso) en DB Browser, para comprobar si los cambios se han producido correctamente:

DB Browser for SQLite - C:\Users\chire\Desktop\backend\database\database.sqlite

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Deshacer Abrir proyecto Guardar proyecto

Estructura Hoja de datos Editar pragmas Ejecutar SQL

Tabla: users

	id	name	email	password	role	reg_date	token
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	Marcos	mdm68@alu.ua.es	password123	USER	2026-07-28 14:12:28	NULL
2	2	Marcos	mdm80@alu.ua.es	ale420	USER	2026-07-29 10:47:41	NULL
3	3	Alejandro	ale427@alu.ua.es	\$2b\$10\$P/1iIhavda9S4Fd/...	USER	2026-07-29 10:56:29	NULL
4	4	Alejandro	ale247@alu.ua.es	\$2b\$10\$97q3z9AX6wn8k5he5aYyjuX2hpK1...	USER	2026-07-29 12:02:24	NULL
5	5	Alejandro	ale2477@alu.ua.es	\$2b\$10\$q9CA8x8U5HWRBy02r1QIeezjPD9F...	USER	2026-07-29 12:03:10	NULL
6	6	Alejandro	alejandros@alu.ua.es	\$2b\$10\$ELezIzjFP416smpI8zch..QH5BCm...	USER	2026-07-29 12:05:01	NULL
7	7	manuel	manuel@alu.ua.es	\$2b\$10\$KHKqEZdwG7JfEQcvvm7eZ.d4Wen8...	USER	2026-07-29 12:07:03	NULL
8	8	juan	juan@alu.ua.es	\$2b\$10\$.CKgdSPK9pO9gDr7zPzDCOWEaz2S...	USER	2026-07-29 12:08:42	NULL

Figura 47. Resultado de la acción sobre la BD.

Como podemos observar el usuario de id 2, que antes se llamaba Alejandro y tenía de email ale42@alu.ua.es,

ahora tiene los datos que habíamos incluido en la solicitud PUT.

2.14.5. Implementación de DELETE

Para llevar a cabo la eliminación de un usuario se habilita el endpoint DELETE /:id. Esta operación requiere el envío del identificador del usuario como parámetro en la URL y se ejecuta en dos fases: comprobación de existencia y borrado del registro.

2.14.6. Definición de la consulta en el modelo (models/users.js)

Se crea la función deleteUser que recibe el identificador, recupera la información del usuario para su posterior retorno en la respuesta, y ejecuta la instrucción SQL DELETE:

```
// Borrar un usuario de la base de datos
const deleteUser = async (id) => {
  const db = getDb();
  const user = await getUserById(id);
  if (!user) return null;

  await db.run("DELETE FROM users WHERE id = ?", [id]);
  return formatUser(user);
};
```

2.14.7. Lógica de control en el controlador (controllers/users.js)

En el controlador se extrae el parámetro id de req.params y se realizan los siguientes pasos:

1. Se consulta en la base de datos si el usuario existe mediante getUserById. Si no se encuentra, se detiene el flujo respondiendo con un error HTTP 400.
2. Si el usuario existe, se invoca a deleteUser para remover la fila de la tabla users.
3. Se devuelve una respuesta con estado HTTP 200 confirmando la eliminación e incluyendo el objeto del usuario eliminado (sin la contraseña).

```
const deleteUser = async (req, res) => {
  const { id } = req.params;

  try {
    const user = await User.getUserById(id);
    if (!user) {
      return res.status(400).json({
        ok: false,
        msg: "El usuario no existe"
      });
    }
  }

  const deletedUser = await User.deleteUser(id);

  res.json({
    ok: true,
    msg: "Usuario eliminado",
    user: deletedUser
  });
};
```

```
} catch (error) {  
  console.log(error);  
  return res.status(500).json({  
    ok: false,  
    msg: "Error borrando usuario"  
  });  
}  
};
```

Para comprobar que funciona correctamente tenemos que utilizar una solicitud de tipo DELETE en Postman con la id del usuario que queramos borrar, en este caso voy a utilizar la misma que en la solicitud PUT anterior, el usuario de id 2. Debería de salirnos algo así tras enviar la solicitud:

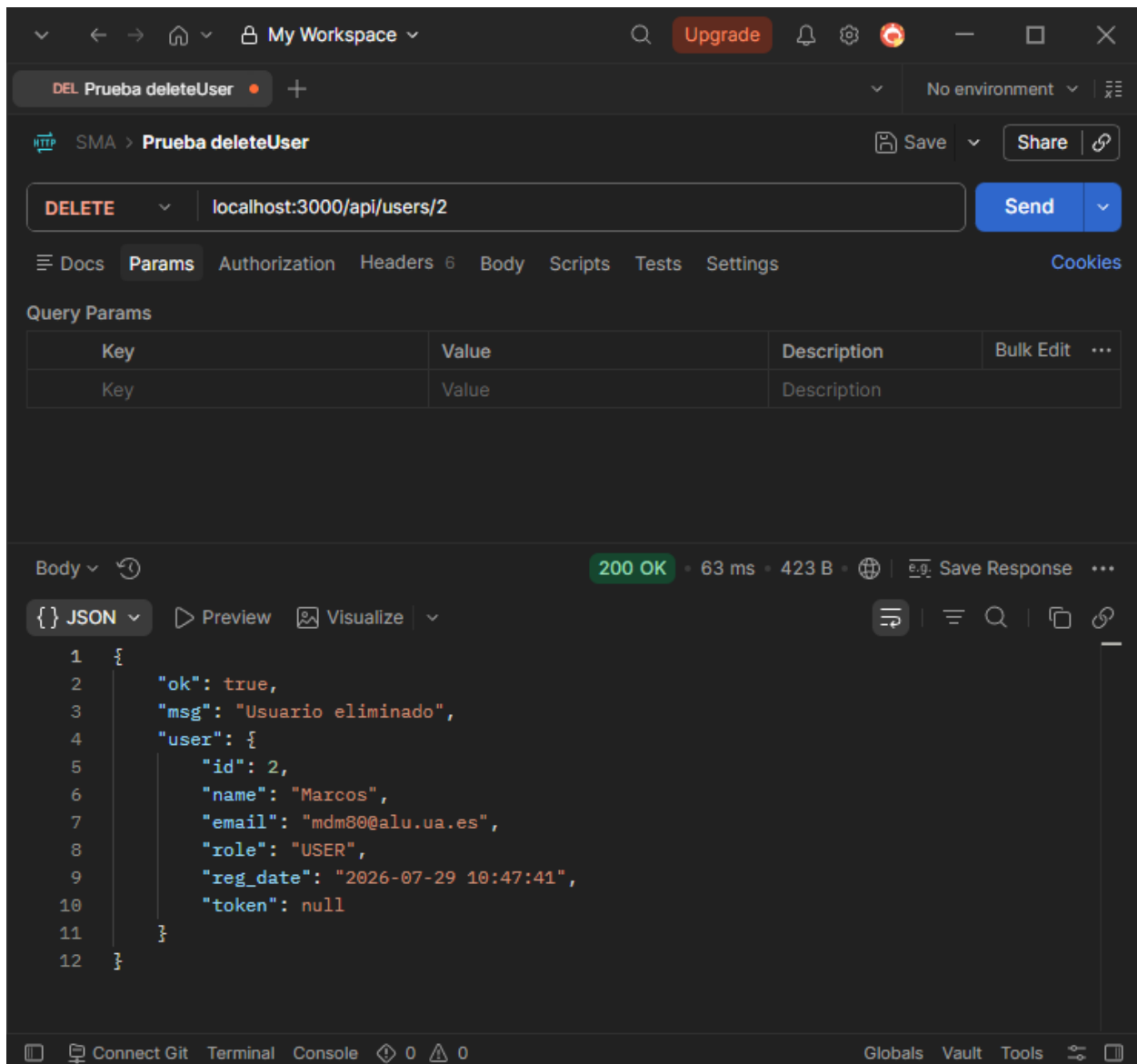


Figura 48. Resultado de invocación del endpoint sobre Postman.

Después realizamos una solicitud GET para obtener todos los usuarios o consultamos la base de datos en DB Browser y deberíamos de observar que el usuario con id 2 en este caso ha desaparecido de la lista:

DB Browser for SQLite - C:\Users\chire\Desktop\backend\database\database.sqlite

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Deshacer Abrir proyecto Guardar proyecto

Estructura Hoja de datos Editar pragmas Ejecutar SQL

Tabla: users

	id	name	email	password	role	reg_date	token
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	Marcos	mdm68@alu.ua.es	password123	USER	2026-07-28 14:12:28	NULL
2	3	Alejandro	ale427@alu.ua.es	\$2b\$10\$P/1iIhavda9S4Fd/...	USER	2026-07-29 10:56:29	NULL
3	4	Alejandro	ale247@alu.ua.es	\$2b\$10\$97q3z9AX6wn8k5he5aYyjux2hpK1...	USER	2026-07-29 12:02:24	NULL
4	5	Alejandro	ale2477@alu.ua.es	\$2b\$10\$q9CA8x8U5HWRBy02r1QIeezjPD9F...	USER	2026-07-29 12:03:10	NULL
5	6	Alejandro	alejandros@alu.ua.es	\$2b\$10\$ELezIzjFP416smpI8zch..QH5BCm...	USER	2026-07-29 12:05:01	NULL
6	7	manuel	manuel@alu.ua.es	\$2b\$10\$KHKqEZdwG7JfEQcvm7eZ.d4Wen8...	USER	2026-07-29 12:07:03	NULL
7	8	juan	juan@alu.ua.es	\$2b\$10\$.CKgdSPK9pO9gDr7zPzDCOWEaz2S...	USER	2026-07-29 12:08:42	NULL

1 - 7 de 7

Ir a: 1

Figura 49. Resultado de la acción sobre la BD.

Efectivamente ya no existe el usuario con id 2, la función actúa como es esperado.

2.15. Descarga Código 3

Todo lo desarrollado hasta ahora está disponible en el “Codigo3.zip” que puedes descargar desde aquí:

[Descargar Código 3](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo3.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo3.zip>

3. Login, JWT y paginación

3.1. 2.1. Login

Para generar nuestro login lo que vamos a hacer es una nueva ruta que se encargará de atender las operaciones de login.

Esta ruta será `/api/login`, y siguiendo la estructura del proyecto, tendrá un router en `/routes/auth.js` y su controlador en `/controllers/auth.js`. Vamos a llamar a los archivos `auth` porque así ahí podremos dejar todo aquello que tenga que ver con la autenticación.

Necesitaremos indicar la nueva ruta `/api/login` y esto se hace en el `index.js`, añadimos la ruta después de la de `/api/users`:

```
app.use('/api/login', require('./routes/auth'));
```

Después en `/routes/auth.js` utilizaremos un esquema similar al utilizado en `/routes/users.js`, para recibir en una petición `post` un `password` y un `email`, validar que estos campos no están vacíos y si todo es correcto pasar el manejo de la ruta al controlador `/controllers/auth.js`. Para eso en `/routes/auth.js` escribimos el siguiente código:

```
const { Router } = require('express');
// Importamos la función login del archivo /controllers/auth.js
const { login } = require('../controllers/auth');
// Importamos la función check para poner las condiciones a los parámetros de
// las consultas
const { check } = require('express-validator');
// Función que creamos anteriormente para imprimir los errores de la solicitud
// en la respuesta
const { validateFields } = require('../middleware/validate-fields');

const router = Router();

router.post('/', [
  check('password', 'El argumento password es obligatorio').not().isEmpty(),
  check('email', 'El argumento email es obligatorio').not().isEmpty(),
  validateFields,
], login);
module.exports = router;
```

Por último creamos el controlador `/controllers/auth.js` en el que escribimos el código siguiente:

```
const { response } = require('express');
const bcrypt = require('bcryptjs');
const User = require('../models/users');

// Función que lleva a cabo el login y devuelve un token
const login = async(req, res = response) => {
  res.json({
    ok: true,
    msg: 'login',
    token: 'token'
  });
}
module.exports = { login }
```

Una vez montada la ruta y el controlador vamos a hacer el login propiamente. Para ello tendremos que hacer dos cosas:

- Comprobar que el email que nos han enviado pertenece a un usuario registrado en la base de datos.
- Comprobar que la contraseña que nos han enviado se relaciona con la contraseña cifrada que hay almacenada en la base de datos

Para ello, recordemos que recibimos dos argumentos, `email` y `password`, los extraemos del `body` de la request y los almacenamos en variables.

```
const { email, password } = req.body;
```

Ahora para comprobar si el email pertenece a un registro de la BD vamos a hacer una consulta y recuperando al usuario que tenga ese email, si existe, para ello hacemos un bloque try/catch, en el catch hacemos el control de error, con un console.log del error y un return de un error genérico y en el try hacemos la primera parte de las comprobaciones, recuperar al usuario de la base de datos con el email buscado:

```
try {
  const user = await User.getUserByEmailAndPassword(email);
  if (!user) {
    return res.status(400).json({
      ok: false,
      msg: 'Usuario o contraseña incorrectos',
      token: ''
    });
  }
} catch (error) {
  console.log(error);
  return res.status(400).json({
    ok: false,
    msg: 'Error en login',
    token: ''
  });
}
```

Como podemos ver en el código, estamos usando una función nueva del modelo User, llamada getUserByEmailAndPassword. Como su nombre indica, esta función nos permite obtener la contraseña en la solicitud por email, cosa que habíamos cambiado anteriormente con formatUser para que no pudiera filtrarse la contraseña de un usuario al frontend. El código de la función que deberá de escribirse en /models/users.js es:

```
// Hace lo mismo que getUserByEmail pero devuelve la contraseña, de forma que lo
podemos utilizar para el login
const getUserByEmailAndPassword = async (email) => {
  const db = getDb();
  return await db.get("SELECT * FROM users WHERE email = ?", [email]);
}
```

En los mensajes de error de Login no se deben proporcionar pistas sobre si el usuario existe o no existe o si el error está en que la contraseña es incorrecta. Simplemente se debe indicar que alguno de los argumentos no es correcto.

Una vez que tenemos al usuario (si no existiera el usuario con el email que nos pasan devolvería error), pasamos a comprobar la password. Para ello lo que vamos a ver es si la password que hay almacenada en la base de datos proviene de la password que nos pasan. No podemos descifrar la password de la base de datos, pero si comprobar si están relacionados. Para eso el módulo bcrypt nos proporciona una función. Después de obtener el usuario añadimos la comparación:

```
// Comparamos la contraseña de la petición con el hash devuelto
const validPassword = bcrypt.compareSync(password, user.password);
if (!validPassword) {
  return res.status(400).json({
    ok: false,
    msg: 'Usuario o contraseña incorrectos',
    token: ''
  });
}
```

Ahora sí, en caso de que el password sea válido la función devuelve true y por tanto supera la comprobación.

Ahora ya solo quedaría generar un JWT válido para devolverlo junto a la respuesta. Pero veamos que funciona correctamente el código comprobándolo creando una nueva solicitud POST a `/api/login` y enviando unos datos de un usuario ya existente:

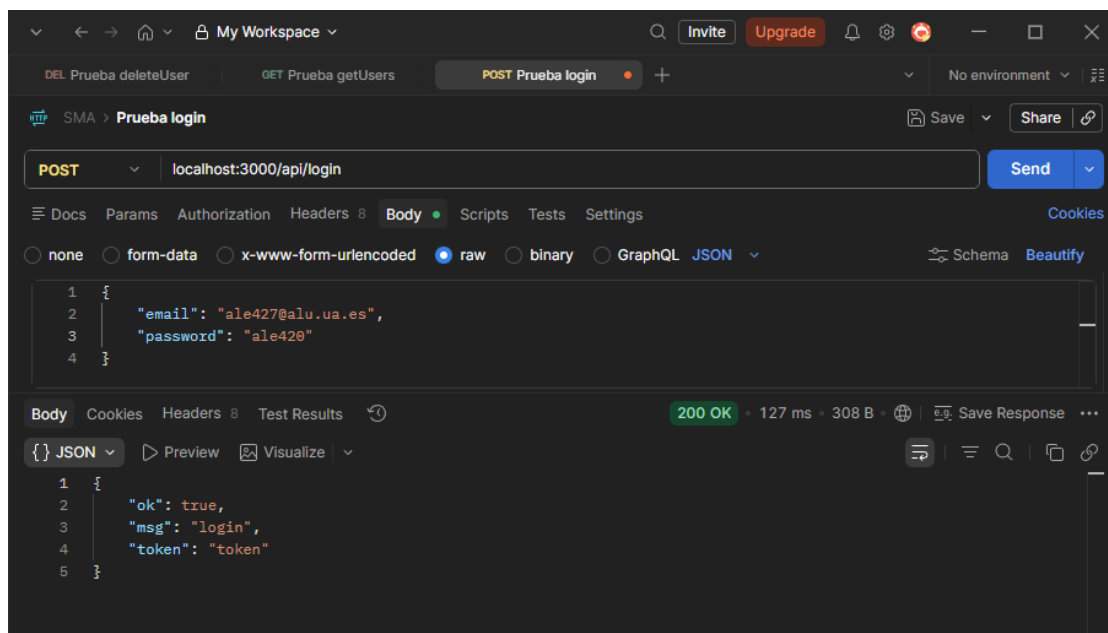


Figura 50. Resultado de la llamada a login.

Si enviamos por ejemplo la contraseña o el email incorrectos, debería de salir un error así:

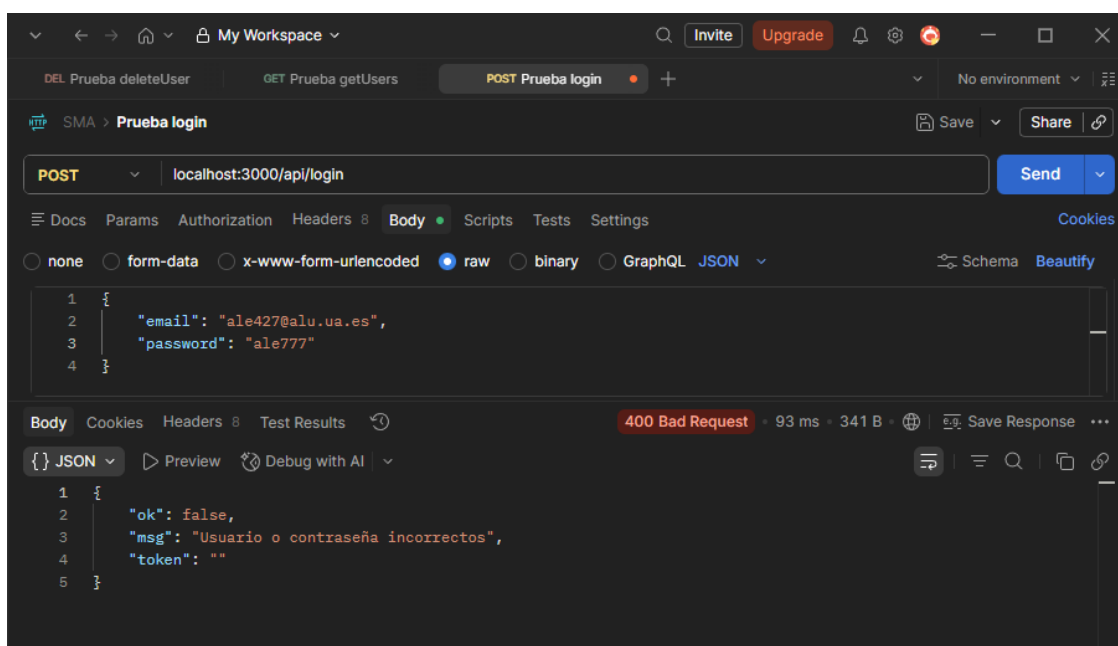


Figura 51. Resultado de la llamada a login cuando el usuario es incorrecto.

3.2. Generar JWT

3.2.1. ¿Qué es un JSON Web Token (JWT)?

Es una cadena de texto codificada que permite transmitir datos de forma segura entre el cliente y el servidor.

Su objetivo principal no es ocultar la información, sino garantizar mediante una **firma digital** que los datos son auténticos y no han sido manipulados. Un JWT está compuesto por tres partes distintas separadas por puntos (.):

HEADER.PAYLOAD.SIGNATURE

En la página de jwt.io podemos entender mejor la estructura del token:

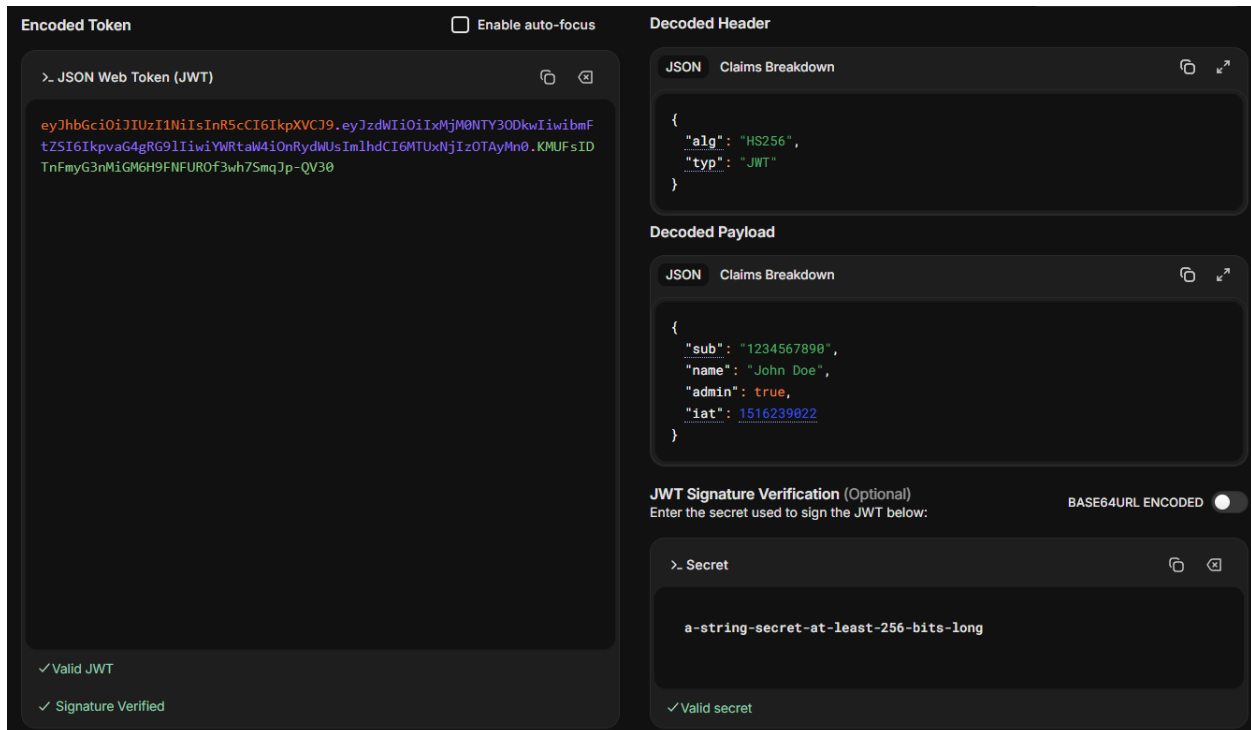


Figura 52. Contenido de un JWT.

1. **Encabezado (Header - primera parte):** Indica el tipo de token y el algoritmo empleado para la firma.
 - Datos codificados: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
 - Significado decodificado: {"alg": "HS256", "typ": "JWT"}
2. **Carga útil (Payload - segunda parte):** Contiene los datos del usuario y de la sesión que necesita el servidor (como el identificador, el rol o la fecha de emisión).
 - Datos codificados: eyJpZCI6Miwicm9sZSI6IIVTRVIlLCJpYXQoOjE3NzE2MTE4ODN9
 - Significado decodificado: {"id": 2, "role": "USER", "iat": 1771611883}
3. **Firma (Signature - tercera parte):** Es el resultado de cifrar la combinación del Header, el Payload y una **clave secreta** que solo conoce el servidor.
 - Datos codificados: SflKxwRJSMekKF2QT4fwpMeJf36POk6yJV_adQssw5c
 - Función: Si alguien intenta modificar un dato del Payload (por ejemplo, cambiar el id), la firma dejará de coincidir y el servidor rechazará la petición.

3.2.2. Implementación del JWT

Ahora vamos a generar nuestros tokens. Para eso vamos a generar tokens que incluyan en su payload 2 campos, el ID del usuario que se ha validado y su rol, ya que será importante para saber a que opciones tiene acceso sin necesidad de acceder a la base de datos.

Para eso vamos a instalar un módulo que nos va a permitir generar JWT:

```
npm install jsonwebtoken
```

Para generar JWTs vamos a crearnos una función aparte a la cual le pasemos los campos que queremos incluir y nos devuelva el token ya creado. Esa función la vamos a crear dentro de una carpeta nueva llamada helpers (puede llamarse como quieras) y dentro el archivo /helpers/jwt.js. Este archivo contendrá el siguiente código:

```
// Importamos la librería para crear y verificar tokens JSON Web Token
const jwt = require('jsonwebtoken');

// Función que genera un token con los datos básicos del usuario
const generateJWT = (id, role) => {
  // Retornamos una Promesa para poder usar async/await al llamar a esta función
  return new Promise((resolve, reject) => {

    // Definimos la información útil que guardaremos dentro del token
    const payload = {
      id,
      role
    };

    // Creamos y firmamos el token con el contenido, una clave secreta y su
    tiempo de vida
    jwt.sign(
      payload,
      'ñalkjsd', // Clave secreta interna para firmar el token (debe mantenerse
    protegida)
    {
      expiresIn: '24h' // Establecemos que el token caducará pasadas 24 horas
    },
    (err, token) => {
      // Si ocurre un fallo durante la creación, rechazamos la promesa con un
    error
      if (err) {
        console.log(err);
        reject('No se pudo generar el JWT');
      } else {
        // Si todo sale bien, devolvemos el token generado
        resolve(token);
      }
    }
  );
});

module.exports = { generateJWT };
```

Este función en realidad le vamos a dar forma de promesa, ya que una promesa permite devolver el éxito o el fracaso de una operación, y esto va a ser muy útil cuando la invoquemos dentro de un bloque try/catch, ya que si la función no puede ejecutarse, el error será capturado por el catch.

Ahora la forma de invocar esta promesa simplemente será desde /controllers/auth.js invocándola:

```
const token = await generateJWT(user.id, user.role);
```

Por último se devuelve en la respuesta el token que se ha generado. En caso de error en la generación del

token, como se ha dicho el error será manejado por la parte del catch del bloque try/catch.

También añadiremos a la respuesta JSON el atributo token de forma que quede así:

```
res.json({
  ok: true,
  msg: 'login',
  token
});
```

3.2.3. Clave secreta para generar el token

Ahora para finalizar vamos a establecer dónde almacenar nuestra clave de firma secreta y como podemos comprobar ahora mismo si un JWT es correcto. Para ello vamos a crear una nueva variable en nuestro archivo de variables de entorno .ENV que se llame por ejemplo JWTSECRET y se asignamos una cadena de texto complicada, como por ejemplo:

```
JWTSECRET=82i44Ç$$$&jsjsJKS23rkfsdbvksldWEDEVFG
```

Y en la promesa donde generamos el JWT vamos a firmar utilizando esta variable de entorno (recuerda que se cargan al principio en index.js gracias a DOTENV):

```
jwt.sign(payload, process.env.JWTSECRET, {
```

Ahora cuando generemos un JWT podemos comprobarlo en www.jwt.io y ver que se corresponde con la firma:

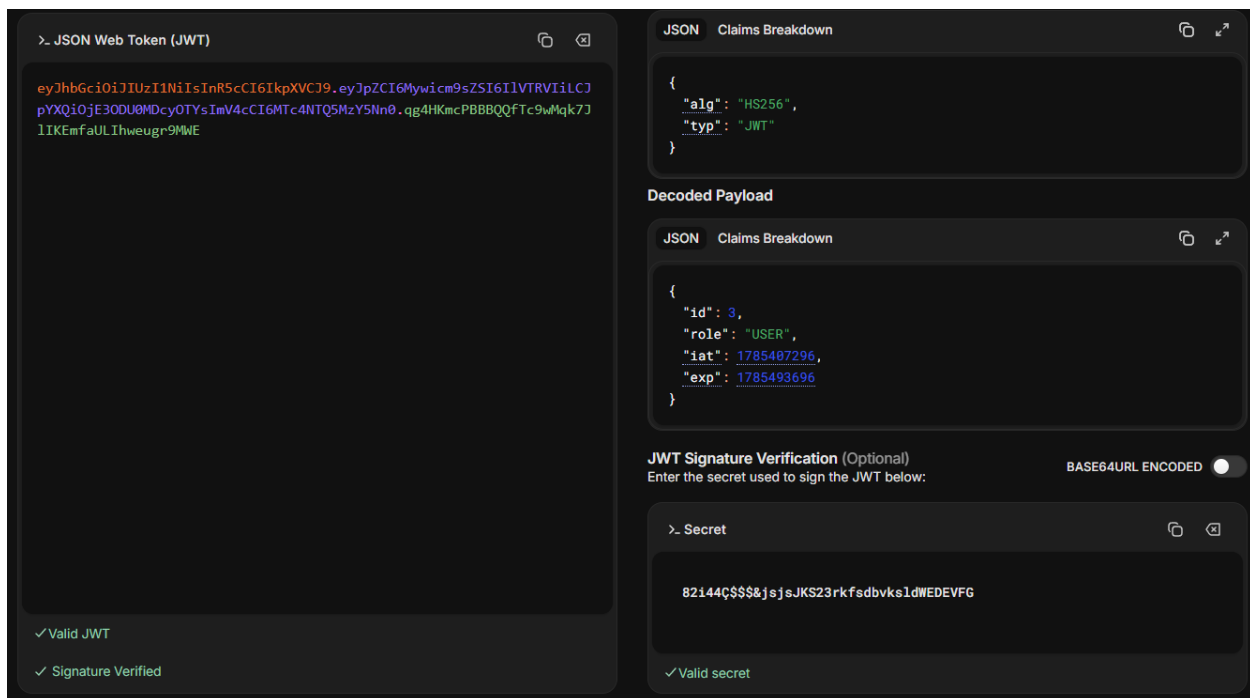


Figura 53. Contenido del JWT generado.

3.2.4. Token de renovación y control de sesión única

Para elevar la seguridad del sistema se aplica una estrategia de **dobles tokens** (token de acceso y token de renovación) junto con el almacenamiento del token en la base de datos para garantizar una **única sesión abierta por usuario**.

- **Token de acceso (token):** Cadena de corta duración (ej. 15 minutos) que sirve para autenticar cada petición HTTP a las rutas protegidas.

- **Token de renovación (token_r):** Cadena de mayor duración (24 horas) que permite solicitar un nuevo par de tokens cuando el de acceso caduca.
- **Sesión única:** Cada vez que un usuario hace login, se guarda su nuevo token de acceso en la columna token de la base de datos SQLite. Si inicia sesión en otro lugar, el token anterior se sobrescribe y la sesión previa queda invalidada.

Para poder implementar este token de renovación, tenemos que modificar el archivo /helpers/jwt.js quedando de la siguiente forma:

```
const jwt = require('jsonwebtoken');

// Genera el token de acceso (corta duración) y el token de renovación (larga duración)
const generateJWT = (id, role) => {
  return new Promise((resolve, reject) => {
    const payload = { id, role };

    // 1. Token de acceso (15 minutos)
    jwt.sign(
      payload,
      process.env.JWTSECRET,
      { expiresIn: '15m' },
      (err, token) => {
        if (err) return reject('No se pudo generar el token de acceso');

        // 2. Token de renovación (24 horas)
        jwt.sign(
          payload,
          process.env.JWTSECRET,
          { expiresIn: '24h' },
          (errR, token_r) => {
            if (errR) return reject('No se pudo generar el token de renovación');

            // Devolvemos ambos tokens
            resolve({ token, token_r });
          }
        );
      }
    );
  });
};

module.exports = { generateJWT };
```

3.2.5. Guardar el token en la base de datos

Añadimos una función en /models/users.js para actualizar la columna token del usuario en la BD:

```
// Actualiza el token activo en la BD para garantizar una única sesión abierta
const updateUserToken = async (id, token) => {
  const db = getDb();
  await db.run("UPDATE users SET token = ? WHERE id = ?", [token, id]);
};
```

3.2.6. Integración en el login

En la función login, tras comprobar las credenciales, generamos los dos tokens, guardamos el de acceso en la base de datos y devolvemos la respuesta completa:

```
// Generamos ambos tokens
const { token, token_r } = await generateJWT(user.id, user.role);

// Guardamos el token de acceso en SQLite para el control de sesión única
await User.updateUserToken(user.id, token);

// 1. Obtenemos el usuario limpio (sin contraseña) ya que el user anterior-
mente declarado, trae la contraseña cifrada, y eso sería una brecha de seguridad
const cleanUser = await User.getUserById(user.id);

// 2. Enviamos 'cleanUser' en la respuesta
res.json({
  ok: true,
  user: cleanUser,
  token,
  token_r
});
```

3.2.7. Endpoint y controlador de renovación

Para procesar las peticiones de refresco cuando el token de acceso caduque, creamos la función `renewToken` en `/controllers/auth.js`:

```
// Función para renovar el token de acceso cuando se requiere
const renewToken = async (req, res = response) => {
  const id = req.id;

  try {
    // Generamos un nuevo par de tokens
    const { token, token_r } = await generateJWT(id, req.role);

    // Actualizamos la base de datos con el nuevo token activo
    await User.updateUserToken(id, token);

    const user = await User.getUserById(id);

    res.json({
      ok: true,
      user,
      token,
      token_r
    });
  } catch (error) {
    console.log(error);
    res.status(500).json({
      ok: false,
      msg: 'Error al renovar el token'
    });
  }
};
```

3.2.8. Middleware validate-jwt.js

Ahora que ya sabemos generar nuestro JWT cuando el login ha sido exitoso, vamos a utilizarlo para proporcionar seguridad en nuestra API.

Una API debe conocer en todo momento y de forma fehaciente quien es el usuario que invoca un endpoint y así poder conceder permisos o no para su uso. Esta identificación se realiza mediante JWT. Lo que tiene que hacer es en cada llamada enviar el JWT que nos identifica a través de la API, de forma que en el backend se pueda comprobar:

- Que el JWT es válido
- La identidad del usuario
- Otras propiedades

El JWT puede enviarse desde el front hacia el back de dos formas, a través de las cabeceras (headers) o a través de un parámetro en la ruta que se invoca. La utilización de los headers es habitual cuando es una aplicación la que utiliza la API aunque en realidad daría igual el mecanismo a través del cual se hace llegar. En nuestro caso vamos a utilizar los headers ya que después será nuestro frontend el que recibirá el JWT cuando se haga login y el que lo empleará en cada llamada para identificarse.

Dicho esto, vamos a modificar el código de nuestro backend de forma que obligatoriamente ante cada ruta que necesite seguridad, esté obligado a recibir un JWT y a validar que dicho JWT sea correcto. Además también aprovecharemos para extraer del JWT el ID y ROL del usuario que lo invoca para posteriores comprobaciones.

Para esto vamos a crear otro middleware llamado **/middleware/validate-jwt.js** con el siguiente contenido. Primero importamos la librería `jsonwebtoken` que utilizaremos para comprobar la firma del JWT. Después creamos la función del middleware con los parámetros `req`, `res` y `next` (como otros middleware). Extraemos el token de la cabecera `x-token` (esta cabecera es nuestra cabecera para enviar nuestro token, podemos llamarla a la cabecera como queramos). Después comprobamos si existen el token, si no existe devolvemos error y por fin viene la parte interesante. Con `jwt.verify` devolvemos el payload decodificado siempre y cuando la firma secreta sea correcta, por tanto si el token ha sido alterado por alguien o no ha sido generado por nuestra aplicación, no se podrán extraer los campos del payload. Contendrá este código:

```
const jwt = require('jsonwebtoken');

// Middleware para verificar la validez del JWT en las peticiones HTTP
const validateJWT = (req, res, next) => {
  // Extraemos el token enviado en los encabezados de la petición
  const token = req.header('x-token');

  // Si no se proporciona token, se interrumpe la petición
  if (!token) {
    return res.status(401).json({
      ok: false,
      msg: 'No hay token en la petición'
    });
  }

  try {
    // Comprobamos la firma del token utilizando la clave secreta del entorno
    const { id, role } = jwt.verify(token, process.env.JWTSECRET);
```

```
// Guardamos el identificador y el rol en el objeto de la petición (req)
req.id = id;
req.role = role;

// Permitimos que la ejecución continúe hacia el siguiente middleware o
controlador
next();

} catch (error) {
  return res.status(401).json({
    ok: false,
    msg: 'Token no válido'
  });
}
};

module.exports = { validateJWT };
```

Las rutas que deberían de utilizar este middleware son todas menos la de login en este caso, y la llamada debería de colocarse antes de las llamadas a la función check.

Finalmente, declaramos la ruta /renew en /routes/auth.js protegiéndola con el middleware de validación:

```
router.get('/renew', validateJWT, renewToken);
```

3.2.9. Pruebas

Ahora ya tenemos un sistema de tokens JWT que permite control de sesión única almacenando el token en la BD y que además renueva el token periódicamente para mayor seguridad. Comprobemos que funciona el login y nos devuelve los dos tokens y los datos del usuario tal que así:

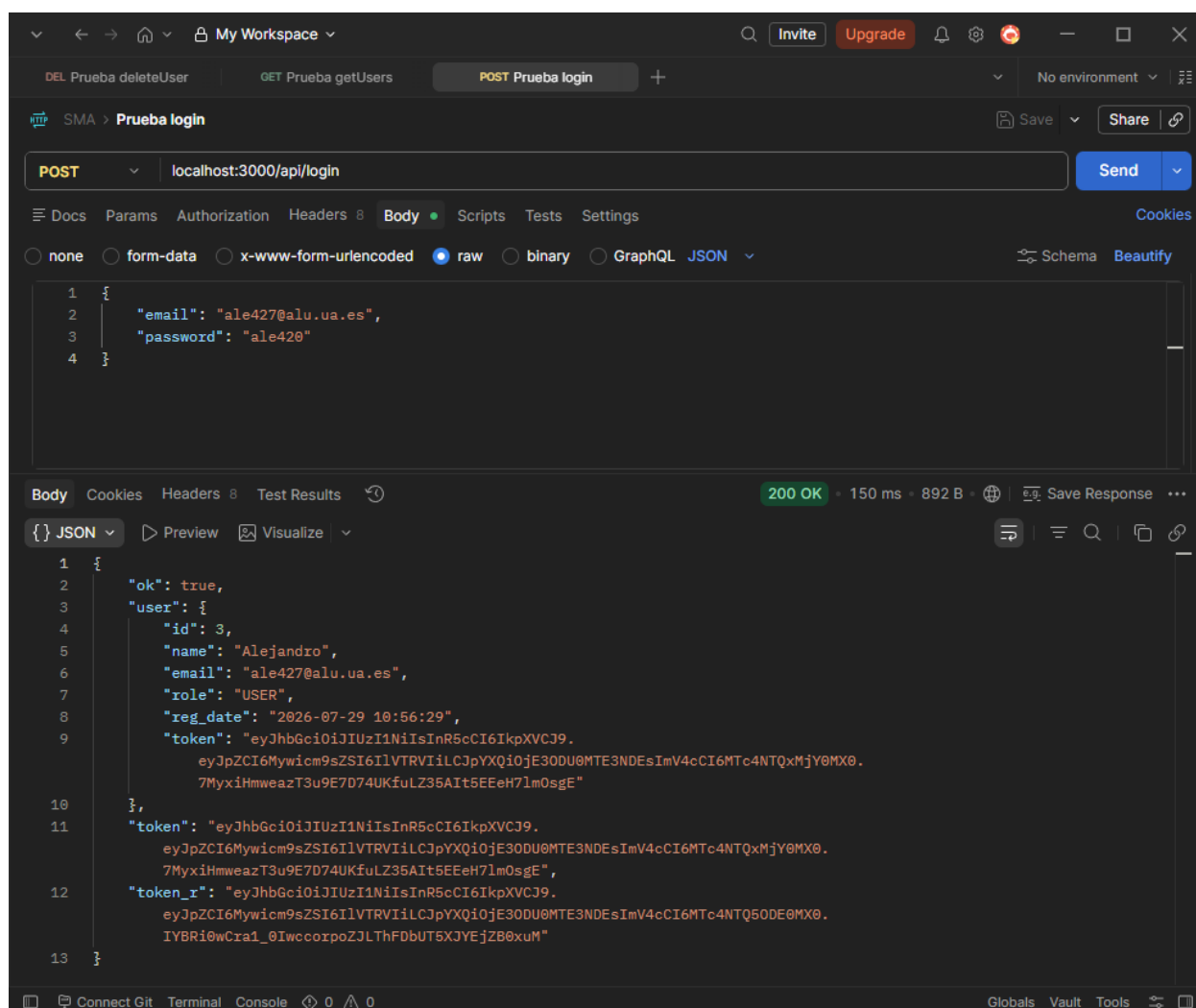


Figura 54. Resultado de generar los tokens.

Si nos fijamos en la BD ahora mismo, el usuario debería de tener un valor de token en su columna "token":

Filtro	token
NULL	
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Mywicm9seSI6IjVTRViiLCJpYXQiOjE3ODU0MTE3NDEsImV4cCI6MTc4NTQxMjY0MX0.7MyxiHmwearT3u9E7D74UKfuLZ35AIt-SEEH7lmOsGE	
NULL	
NULL	
NULL	
NULL	
NULL	

Figura 54. Token almacenado en la BD.

Para continuar con las pruebas necesitaremos el valor de este token, por lo que lo copiaremos y haremos una petición a la ruta `/api/users`, que está protegida con el middleware `validateJWT`. Vamos a probar a mandar la petición sin token, tendr a que salirnos algo as :

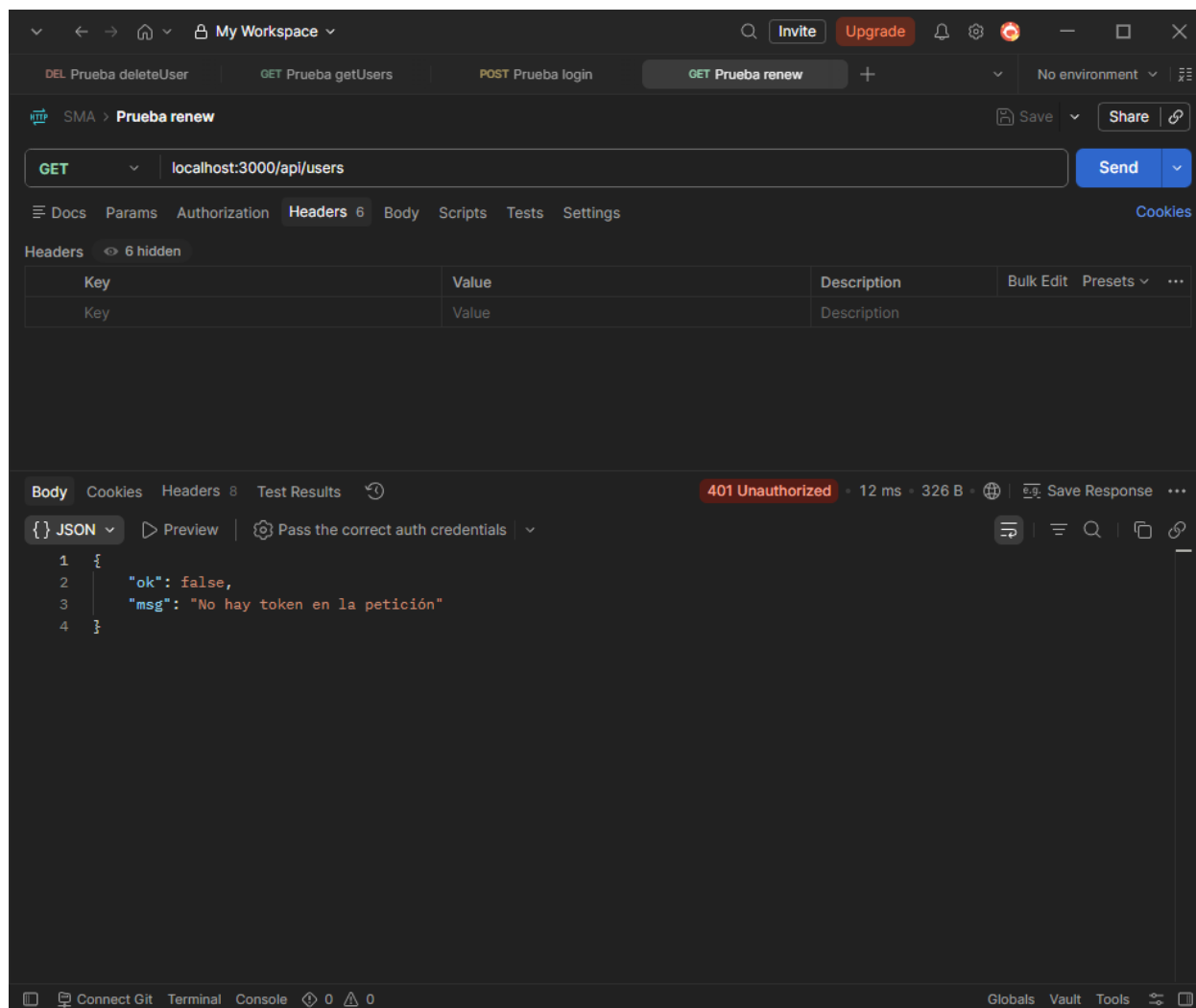


Figura 55. Prueba de petición sin token.

Si probamos a hacerla con el token, debería de funcionar normalmente. Vamos a probar la ruta /renew, para verificar que la API emite un nuevo par de tokens a partir del token de acceso actual. Tenemos que colocar como nombre de la Header "x-token" y como value, la cadena del token_r completa:

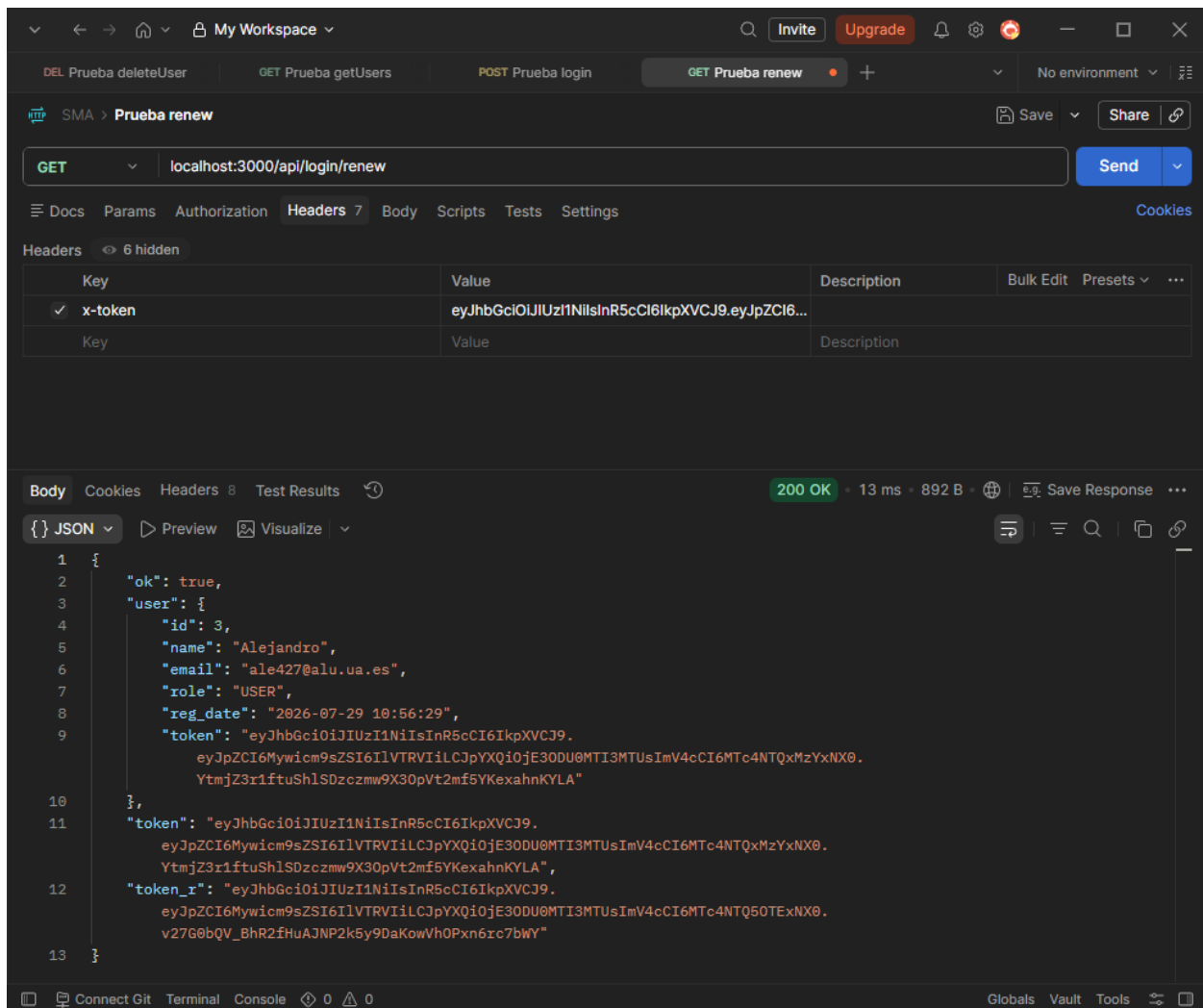


Figura 56. Prueba de petición con token.

3.2.10. Decarga Código 4

Todo lo desarrollado hasta ahora está disponible en el “Codigo43” que puedes descargar desde aquí:

[Descargar Código 4](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo4.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo4.zip>

3.3. Paginación y Promise.all

En una llamada a la API podemos obtener cientos de registros. Esto puede ser inmanejable en el frontend si le devolvemos una cantidad de información demasiado grande. Para evitar este problema se utiliza la paginación. Pagar significa que el número de resultados devueltos va a ser limitado, y además podremos indicar

en que punto de la colección queremos empezar a devolver resultados. Es decir, imagina que nuestra colección tiene 100 elementos (1,2,3...,100), mediante paginación podemos indicar que queremos devolver 10 elementos en cada petición y además indicar que queremos devolverlos a partir del elemento 20, es decir, devolveríamos 20,21,22,23...29. Hay módulos de express que ayudan para realizar esto, pero vamos a aprender a hacerlo completamente a mano y así tener el control total.

Para hacer esto lo primero es tener en cuenta que ahora nuestras rutas deberán de alguna forma recibir como mínimo el punto de la colección (número de registro) a partir del cual devolver resultados. Vamos a trabajar con el ejemplo de `/api/users`. Este endpoint devuelve todos los registros de la colección, si fuesen cientos esto sería incorrecto. Para paginarlo vamos a tener en cuenta que a nuestra ruta le vamos a poder pasar un parámetro llamado "from" con lo que nuestra ruta luciría desde ahora como `/api/users?from=X` siendo X cualquier número. Como siempre hay que tener en cuenta que el usuario puede ser malintencionado y puede no enviarnos ese parámetro o puede enviar un parámetro que no sea un número, nuestro código nos tiene que proteger frente a esto.

En `/controllers/users.js` en el método `getUsers` vamos a modificar el código y añadir lo siguiente:

```
// Desde qué registro se empieza a contar
const from = Number(req.query.from) || 0;
// Cuántos registros por página queremos
const regPerPage = 10;
```

Esto recibe el parámetro 'from' de la query y en caso de que no venga o que no sea un número lo inicializa a 0. También vamos a establecer a 10 el número de registros a devolver en cada consulta (cada página).

Ahora simplemente modificaremos la consulta que tenemos actualmente para hacer `User.getUsers` a lo siguiente:

```
const users = await User.getUsers(from, regPerPage);
```

Esta consulta devolverá solo 10 resultados a partir de la posición 'from' en la base de datos.

Si no hacemos nada más, nuestra función devolvería un resultado similar a lo que teníamos antes solo que limitando el número de registros devueltos. Cuando trabajamos con paginación es necesario devolver información referente a la paginación que genera los resultados, por ejemplo indicando la posición actual en la base de datos (el from), el número de registros por página (regPerPage) e incluso el total de registros en la base de datos, para que así el frontend pueda determinar si mostrar botones siguiente, anterior, números de página, etc. Para poder contar el número de documentos en la colección tenemos que crear una función en nuestro archivo `/models/users.js` tal que así:

```
// Cuenta el total de usuarios en la tabla
const countUsers = async () => {
  const db = getDb();
  return await db.get("SELECT COUNT(*) as total FROM users");
};
```

También tenemos que incluir los nuevos parámetros en el método `getUsers` de `/models/users.js`, que quedaría así:

```
const getUsers = async (from = 0, limit = 10) => {
  const db = getDb();
  const users = await db.all(
    "SELECT * FROM users LIMIT ? OFFSET ?",
    [limit, from]
  );
  return users.map(user => formatUser(user));
};
```

```
}
```

Después volveremos a nuestro archivo `/controllers/users.js` y pondremos el siguiente código en la función `getUsers`:

```
const total = await User.countUsers();
```

Y modificaremos los resultados devueltos de la siguiente forma:

```
res.json({
  ok: true,
  msg: 'getUsers',
  users,
  page: {
    from,
    regPerPage,
    total
  }
});
```

Así junto a la lista de usuarios podemos ver información sobre la paginación.

Solo nos queda hacer una optimización. Ahora mismo estamos lanzando dos funciones contra la base de datos, un `'getUsers'` y un `'countUsers'`, y para cada una de ellas (son funciones asíncronas, pueden tardar más o menos tiempo) hacemos `await`, es decir, obligamos a que se espera a la finalización de la función para continuar con la siguiente. En caso de que tuviesemos que lanzar varias llamadas esto sería ineficiente. Para optimizar el código vamos a lanzarlas todas de forma paralela y cuando terminen entonces continuar. Para ello utilizaremos el método `Promise.all`.

```
const [ users, total ] = await Promise.all([
  await User.getUsers(from, regPerPage),
  await User.countUsers()
]);
```

De esta forma se ejecutan las dos funciones y sus resultados se almacenan de nuevo en `users` y `total` para ser utilizados en `res.json`.

Ahora solo resta encerrar todo el código en un bloque `try/catch` para manejar los posibles errores (por ejemplo un corte de conexión a BD) de forma que quede tal que así:

```
const getUsers = async (req, res) => {
  try {
    await User.initUsersTable();

    const from = Number(req.query.from) || 0;
    const regPerPage = 10;

    // Ejecutamos ambas consultas a la base de datos en paralelo
    const [ users, total ] = await Promise.all([
      User.getUsers(from, regPerPage),
      User.countUsers()
    ]);

    res.json({
      ok: true,
      msg: "getUsers",
      users,
      page: {
        from,
        regPerPage,
        total
      }
    });
  }
};
```

```
});  
  
} catch (error) {  
  console.log(error);  
  return res.status(500).json({  
    ok: false,  
    msg: "Error obteniendo usuarios"  
  });  
}  
};
```

3.3.1. Decarga Código 5

Todo lo desarrollado hasta ahora está disponible en el “Codigo5” que puedes descargar desde aquí:

[Descargar Código 5](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo5.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo5.zip>

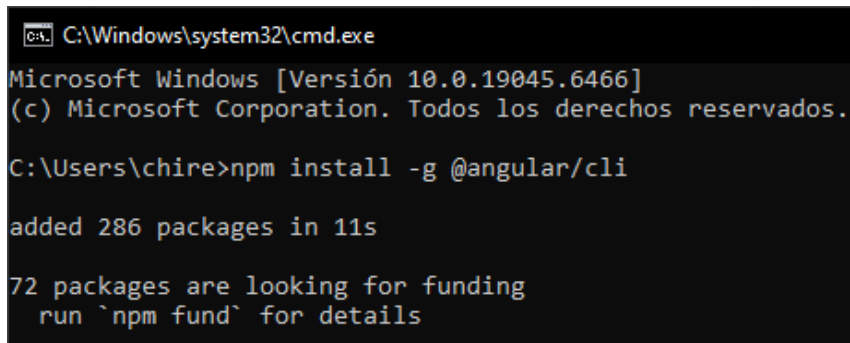
4. Inicio del frontend

4.1. Inicio del proyecto

Para el desarrollo del Frontend, o la parte de cliente, vamos a utilizar Angular, en su versión 22. La versión no es muy importante pues en realidad existen 2 versiones de Angular, la 1 y el resto (2, 3, 4...) Si es posible utiliza siempre la última versión estable de angular ya que te va a permitir tener la mejor compatibilidad con los módulos y plugins. Para instalar angular en realidad lo que vamos a instalar en nuestro sistema es un programa que se ejecutará en la línea de comandos y que nos permitirá crear todos los elementos que formarán nuestro proyecto. Para ello, si no lo tienes instalado todavía ejecuta el siguiente comando en una ventana de comandos:

```
npm install -g @angular/cli
```

Esto utilizará el gestor de paquetes npm (debes tener instalado node y npm en tu sistema) e instalará la última versión de AngularCLI. Verás un resultado como este:



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\chire>npm install -g @angular/cli

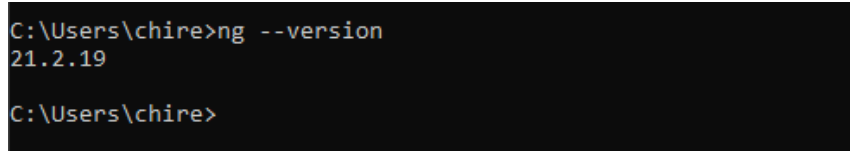
added 286 packages in 11s
72 packages are looking for funding
  run `npm fund` for details
```

Figura 57. Instalacion de angular command line.

La versión que se haya instalado en tu sistema no es importante (con una 9 o 10 o superior es suficiente). Solo con esto ya podemos crear nuestro proyecto, mediante el uso de la línea de comandos de AngularCLI. Para ejecutar comandos utilizaremos la palabra reservado "ng". Ejecuta el siguiente comando:

```
ng --version
```

Deberías ver algo parecido a lo siguiente:



```
C:\Users\chire>ng --version
21.2.19

C:\Users\chire>
```

Figura 58. Visualizando la versión de angular instalada.

Toda la información sobre AngularCLI está disponible la web oficial de Angular CLI <https://angular.io/cli>, sería recomendable que entres y leas un poco sobre Angular CLI. Por ahora vamos a iniciar nuestro proyecto. Para ello vamos hasta una carpeta donde queramos almacenar el código de nuestro proyecto y ejecutamos lo siguiente:

```
ng new frontend
```

A continuación nos hará dos preguntas:

- Que formato de estilos queremos utilizar: vamos a seleccionar CSS por defecto.
- Si queremos habilitar el Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering), a lo que pondremos que no lo queremos usar, escribiendo una n.
- Después nos comentará si queremos usar alguna herramienta de IA y seleccionaremos que no, escribiendo una n.

Deberíamos de ver algo así tras seguir los pasos:

```
C:\Windows\system32\cmd.exe
C:\Users\chire\Desktop>ng new frontend
✓ Which stylesheet system would you like to use? CSS [ https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✓ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE frontend/.prettierrc (173 bytes)
CREATE frontend/angular.json (1982 bytes)
CREATE frontend/package.json (785 bytes)
CREATE frontend/README.md (1521 bytes)
CREATE frontend/tsconfig.json (990 bytes)
CREATE frontend/.editorconfig (331 bytes)
CREATE frontend/.gitignore (666 bytes)
CREATE frontend/tsconfig.app.json (444 bytes)
CREATE frontend/tsconfig.spec.json (456 bytes)
CREATE frontend/.vscode/extensions.json (134 bytes)
CREATE frontend/.vscode/launch.json (490 bytes)
CREATE frontend/.vscode/mcp.json (188 bytes)
CREATE frontend/.vscode/tasks.json (1020 bytes)
CREATE frontend/src/main.ts (228 bytes)
CREATE frontend/src/index.html (307 bytes)
CREATE frontend/src/styles.css (81 bytes)
CREATE frontend/src/app/app.spec.ts (698 bytes)
CREATE frontend/src/app/app.ts (302 bytes)
CREATE frontend/src/app/app.css (0 bytes)
CREATE frontend/src/app/app.html (20488 bytes)
CREATE frontend/src/app/app.config.ts (324 bytes)
CREATE frontend/src/app/app.routes.ts (80 bytes)
CREATE frontend/public/favicon.ico (15086 bytes)
✓ Packages installed successfully.
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
Successfully initialized git.
C:\Users\chire\Desktop>
```

Figura 59. Creación del nuevo proyecto de Angular.

Y para comprobar que todo funciona correctamente, una vez que haya terminado la instalación, accedemos a la carpeta frontend y ejecutamos lo siguiente:

```
cd frontend
ng serve -o
```

Con 'ng serve -o' lanzamos la ejecución de un servidor local que lanzará nuestra aplicación Angular, con el modificador '-o' se abrirá automáticamente un navegador y nos mostrará una interfaz similar a esto, mostrando que nuestra aplicación funciona adecuadamente:

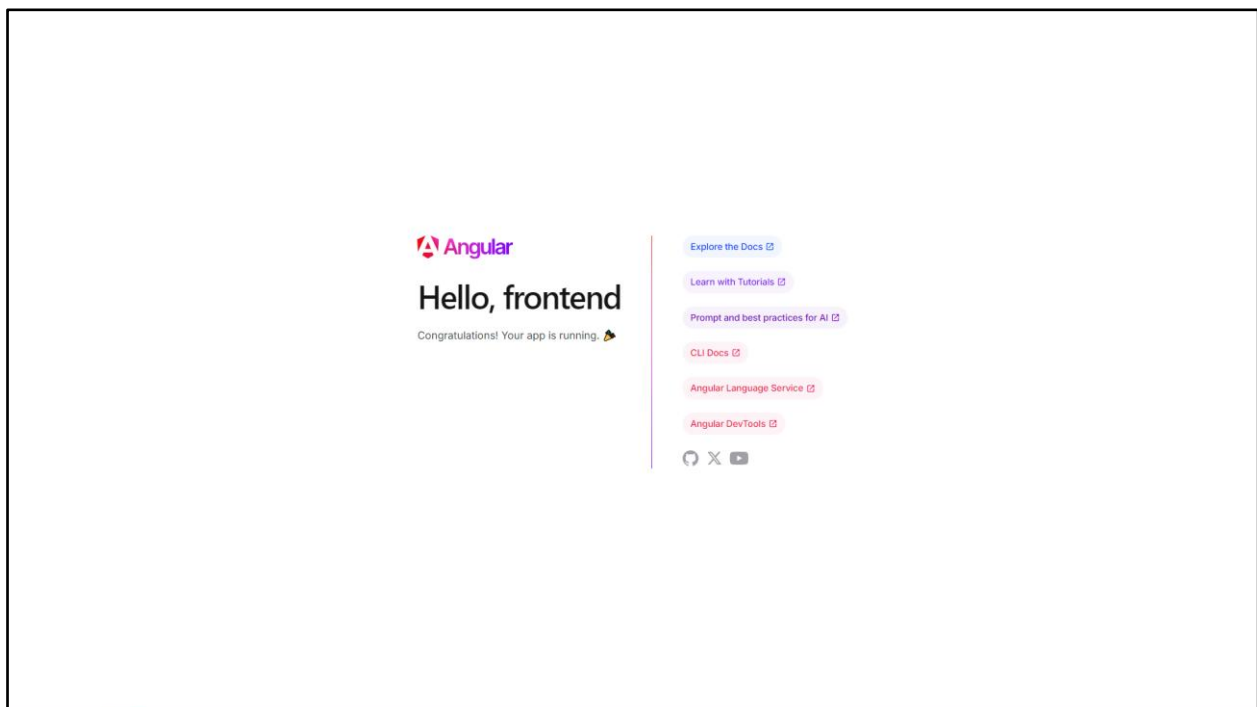


Figura 60. Página web por defecto tras crear un nuevo proyecto.

Una vez que ya tenemos nuestra base de aplicación, vamos a ver como crear las diferentes partes y elementos que formarán parte de ella.

4.2. Diseño basado en componentes

Las aplicaciones web modernas se basan en el concepto de single-page application (SPA). Este tipo de aplicaciones tienen una forma de funcionar característica muy parecida a las aplicaciones de escritorio, ya que en nuestro navegador reside toda la aplicación web, exceptuando los datos, y mediante peticiones concretas (llamadas a la API), se obtienen los datos que se renderizan en el navegador.

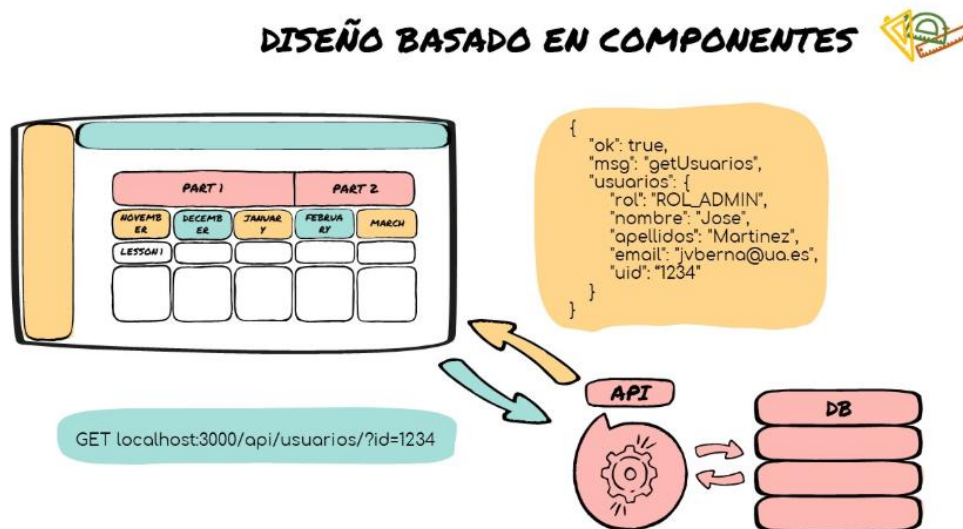


Figura 61. Diagrama de diseño basado en componente.

Las aplicaciones web además suelen ser muy complejas y por tanto es necesario poder dividir en diferentes elementos las distintas partes de la aplicación para lograr así un código mantenible y modular (igual que hicimos en el backend). Para ello en Angular se utilizan los componentes. Los componentes son las piezas mediante las cuales montamos nuestra aplicación. Un componente puede ser una barra de menú lateral, una barra de menú superior, el elemento que muestra las migas de pan, una gráfica, una tabla, un formulario. Cada elemento que se muestra en pantalla puede ser aislado en forma de componente, que recibe y emite datos hacia otros componentes o hacia la API.

4.3. Primeros componentes

Para empezar nuestro sistema vamos a plantearnos el esquema básico de nuestra aplicación. En nuestro caso la aplicación cuando no estamos identificados solo tendrá una interfaz con la que interactuar, el login. Una vez identificados se accederá a un panel de gestión que será muy similar para todos los usuarios, aunque cambiarán las opciones accesibles en función del tipo de usuario que sea: administrador o usuario. Pero primero veamos un poco que tiene nuestro proyecto cuando está vacío.

4.3.1. Estructura del proyecto

Archivos de configuración en la raíz

- **package.json:** Archivo de configuración central donde se definen las dependencias (librerías instaladas), las dependencias de desarrollo y los comandos de ejecución (como `ng serve` o `ng build`).
- **angular.json:** Contiene la configuración global del marco de trabajo (*framework*), definiendo cómo se compila el proyecto, las rutas de estilos globales y los entornos de ejecución.
- **tsconfig.json:** Configuración del compilador de TypeScript.

Toda la lógica y los recursos visuales de la aplicación se ubican dentro de la carpeta **src/**:

- **index.html:** Página HTML única de la aplicación (*Single Page Application*). Contiene la estructura básica del documento e incluye la etiqueta del componente principal (`<app-root></app-root>`), la cual sirve de contenedor para renderizar el contenido dinámico.
- **main.ts:** Punto de entrada ejecutable de la aplicación. Se encarga de inicializar el módulo principal de Angular (AppModule) para arrancar el proyecto.
- **styles.css:** Hoja de estilos en cascada de ámbito global. Las reglas definidas aquí afectan a toda la aplicación.
- **Carpeta app/:** Espacio donde se ubica el código funcional. Contiene:
 - **app.config.ts:** Configuración global de la aplicación, proveedores y servicios.
 - **app.routes.ts:** Archivo para la declaración y gestión del sistema de rutas de navegación.
 - **Componentes de la aplicación:** Bloques funcionales y visuales que componen la interfaz.
- **Carpeta public/:** Directorio ubicado en la raíz destinado a almacenar los recursos estáticos de la aplicación, como imágenes, iconos o archivos de fuentes.

4.3.2. Anatomía de un componente

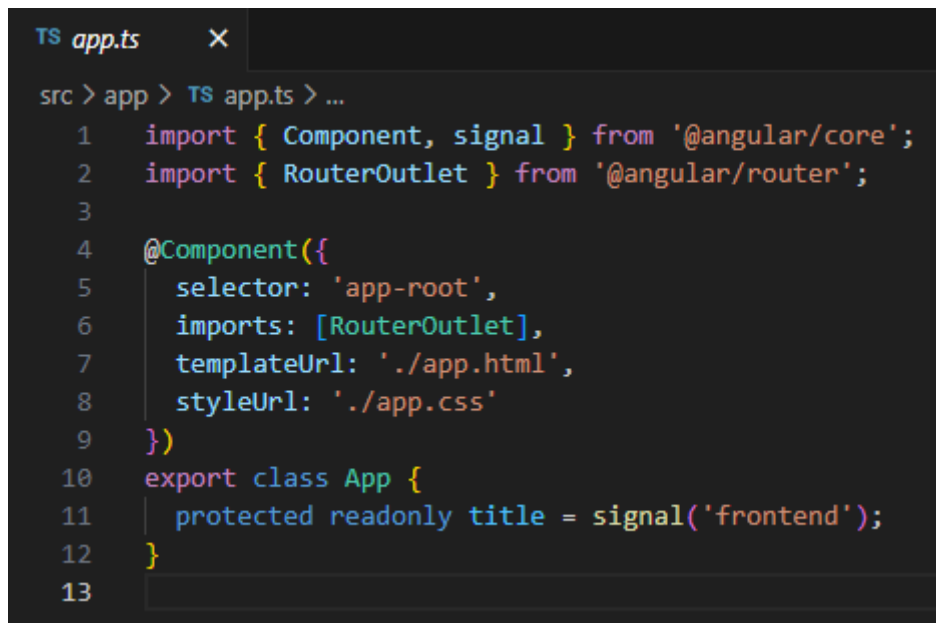
Un **componente** es la unidad fundamental de construcción en Angular. La interfaz de usuario se divide en pequeñas partes independientes y reutilizables, donde cada una de ellas gestiona su propia vista y lógica.

En este proyecto, cada componente se compone de tres archivos principales vinculados:

- **app.html:** Define la estructura visual y los elementos HTML que ve el usuario en pantalla.
- **app.ts:** Contiene la clase en TypeScript que controla el comportamiento, las variables y la lógica de negocio.
- **app.css:** Define las reglas de estilo visual aplicables exclusivamente al HTML de ese componente.

4.3.3. Decorador y metadatos

En el archivo de `app.ts`, la clase se vincula con su vista y sus estilos mediante el decorador **@Component**:



```
TS app.ts  X
src > app > TS app.ts > ...
1  import { Component, signal } from '@angular/core';
2  import { RouterOutlet } from '@angular/router';
3
4  @Component({
5    selector: 'app-root',
6    imports: [RouterOutlet],
7    templateUrl: './app.html',
8    styleUrls: ['./app.css']
9  })
10 export class App {
11   protected readonly title = signal('frontend');
12 }
13
```

Figura 62. Vista el archivo app.ts y su decorador.

- **selector:** Define el nombre de la etiqueta HTML personalizada (en este caso <app-root>) utilizada para incrustar el componente en la página.
- **templateUrl:** Indica la ruta del archivo HTML que contiene la vista.
- **styleUrl:** Especifica el archivo de estilos CSS exclusivo del componente.

4.3.4. Creación de componentes

Ahora vamos a crear los primeros componentes de la aplicación utilizando el comando **ng generate component** (o su versión abreviada **ng g c**). Atendiendo a las especificaciones de nuestro proyecto, la interfaz requiere dos estructuras visuales claramente diferenciadas: por un lado, una pantalla independiente de inicio de sesión (login) basada en correo y contraseña, y por otro lado, un contenedor principal para la parte de administración estructurado con encabezado, menú de navegación lateral y pie de página. Esta arquitectura modular permitirá renderizar unos componentes dentro de otros a través del sistema de rutas, mostrando una pantalla de bienvenida personalizada según el rol del usuario y limitando la visualización de la opción de gestión de usuarios únicamente a quienes tengan rol de administrador.

4.3.5. Arquitectura visual de la aplicación

Antes de comenzar a crear componentes en nuestro proyecto, es conveniente planificar la arquitectura visual de la aplicación. En nuestro sistema contaremos con dos tipos de estructuras o plantillas principales: por un lado, una vista sencilla destinada a páginas aisladas como el inicio de sesión (login), la cual solo requiere un bloque central; y por otro lado, una estructura para la parte de administración (dashboard) que mantendrá elementos fijos en la pantalla como una barra de navegación superior, un menú lateral y un pie de página. Para organizar esta maquetación de forma limpia, crearemos dos componentes contenedores dentro de una carpeta llamada layouts. Para ello, abrimos la consola de comandos dentro de la carpeta del proyecto y ejecutamos la instrucción para generar el componente de autenticación:

```
ng generate component layouts/auth-layout
```

A continuación, generamos el componente contenedor para la zona de administración. Para evitar que el sistema genere el archivo de pruebas automáticas (.spec.ts), el cual no utilizaremos por ahora, añadimos el parámetro `--skip-tests`:

```
ng generate component layouts/admin-layout --skip-tests
```

Tras ejecutar estas órdenes, Angular creará las estructuras de carpetas dentro de `src/app/layouts/` con sus respectivos archivos de plantilla HTML, estilos CSS y lógica TypeScript. Si inspeccionamos la clase en TypeScript (.ts) de cada componente, veremos que cada uno declara un identificador o selector único (`app-auth-layout` y `app-admin-layout`).

Para verificar la incrustación de estos componentes en la aplicación, nos dirigimos al archivo de la vista principal `app.html`, limpiamos todo su contenido y añadimos los dos selectores a modo de etiquetas HTML:

```
<app-auth-layout></app-auth-layout>
<app-admin-layout></app-admin-layout>
```

Asimismo, en el archivo `src/app/app.ts` importamos ambos componentes para que la plantilla principal los reconozca:

```
import { Component, signal } from '@angular/core';
import { AuthLayout } from '../layouts/auth-layout/auth-layout';
import { AdminLayout } from '../layouts/admin-layout/admin-layout';

@Component({
  selector: 'app-root',
  imports: [AuthLayout, AdminLayout],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('frontend');
}
```

Al guardar los cambios en el editor, el servidor local en ejecución (`ng serve`) recompilará el código de forma automática. Al consultar el navegador en `http://localhost:4200`, comprobaremos cómo se renderizan conjuntamente los textos por defecto de ambas plantillas. Esta metodología basada en componentes nos permitirá dividir la interfaz en pequeñas piezas independientes y alternar su visualización según la navegación del usuario. La página debería de verse así, después de los cambios realizados:



Figura 63. Vista del nuevo componente.

4.4. 3.4. Componentes hijos de auth y admin

Una vez estructurados los componentes contenedores principales (`auth-layout` y `admin-layout`), el siguiente paso es configurar la carga de las vistas secundarias dentro de dichos contenedores. Para la sección de autenticación, la especificación de nuestro proyecto requiere únicamente el desarrollo del formulario de inicio

de sesión (login), por lo que omitiremos componentes no requeridos como la recuperación de contraseña.

Para mantener el proyecto organizado, crearemos el componente de inicio de sesión dentro de una carpeta dedicada llamada auth. Abrimos la terminal de comandos y ejecutamos la siguiente instrucción:

ng generate component auth/login --skip-tests

Una vez generado el componente login, preparamos la plantilla raíz de la aplicación (src/app/app.html) para trabajar con rutas dinámicas. Reemplazamos las etiquetas estáticas anteriores por el conector de rutas principal:

```
<router-outlet />
```

E importamos RouterOutlet en src/app/app.ts:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('frontend');
}
```

A continuación, configuramos el sistema de navegación dentro del archivo src/app/app.routes.ts, asociando la ruta /auth con el contenedor AuthLayout y definiendo la ruta hija para el inicio de sesión:

```
import { Routes } from '@angular/router';
import { AuthLayout } from '../layouts/auth-layout/auth-layout';
import { Login } from '../auth/login/login';

export const routes: Routes = [
  {
    path: 'auth',
    component: AuthLayout,
    children: [
      // Si la URL se completa con /login, debe cargar en su interior el Login.
      { path: 'login', component: Login },
      { path: '', redirectTo: 'login', pathMatch: 'full' }
    ]
  }
];
```

Con esta configuración le indicamos al enrutador que siempre que se consulte la ruta /auth debe renderizar el componente contenedor AuthLayout, y que si la URL se completa con /login, debe cargar en su interior el Login.

Para que Angular sepa exactamente en qué lugar del contenedor debe dibujar la vista hija, abrimos la plantilla del componente padre (src/app/layouts/auth-layout/auth-layout.html) y añadimos la directiva interna conservando el texto de la plantilla:

```
<p>auth-layout works!</p>
<router-outlet />
```

Asimismo, debemos asegurarnos de que la clase TypeScript del contenedor (src/app/layouts/auth-layout/auth-layout.ts) tenga importado el módulo del enrutador para reconocer dicha etiqueta:

```
import { Component } from '@angular/core';
import { RouterOutlet } from "@angular/router";

@Component({
  selector: 'app-auth-layout',
  imports: [RouterOutlet],
  templateUrl: './auth-layout.html',
  styleUrls: ['./auth-layout.css'],
})
export class AuthLayout {}
```

Al guardar los cambios y navegar en la aplicación a la dirección `http://localhost:4200/auth/login`, comprobaremos que se muestra el mensaje de prueba del componente de inicio de sesión perfectamente incrustado dentro de la plantilla general de autenticación:



Figura 64. Vista del nuevo componente.

Al sustituir las etiquetas estáticas (`<app-auth-layout>` y `<app-admin-layout>`) en `src/app/app.html` por la directiva `<router-outlet />`, el componente `admin-layout` deja de mostrarse en pantalla. Esto se debe a que la directiva `<router-outlet />` pasa el control visual al enrutador de Angular, el cual únicamente renderiza los componentes cuya ruta coincide con la URL activa en el navegador. Como la ruta consultada es `/auth/login`, el enrutador solo instancia y dibuja el contenedor `AuthLayout` y su hijo `Login`, omitiendo por completo `AdminLayout` al no estar asociado a dicha dirección web.

4.4.1. Zona de administración

Tras configurar la zona de autenticación, abordaremos la estructura de la zona de administración. Todas las páginas de esta sección compartirán una serie de elementos visuales fijos: una barra de navegación superior (navbar), un menú lateral (sidebar) y un pie de página (footer). Además, contaremos con las páginas específicas que se mostrarán en la zona central, como el panel principal (dashboard) y la gestión de usuarios (usuarios).

Para mantener el código organizado, dividiremos estos elementos en dos carpetas: `commons` para los fragmentos de interfaz reutilizables y `pages` para las vistas completas.

Abrimos la terminal de comandos y generamos los componentes comunes omitiendo los archivos de pruebas:

```
ng generate component commons/sidebar --skip-tests
ng generate component commons/navbar --skip-tests
ng generate component commons/footer --skip-tests
```

A continuación, generamos las páginas que compondrán la zona de administración:

```
ng generate component pages/dashboard --skip-tests
```

ng generate component pages/users --skip-tests

El componente contenedor **AdminLayout** es el encargado de agrupar el menú lateral, la barra de navegación y habilitar el espacio para las páginas dinámicas. Para ello, importamos estos elementos en su archivo TypeScript (src/app/layouts/admin-layout/admin-layout.ts):

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Sidebar } from '../../../commons/sidebar/sidebar';
import { Navbar } from '../../../commons/navbar/navbar';

@Component({
  selector: 'app-admin-layout',
  imports: [RouterOutlet, Sidebar, Navbar],
  templateUrl: './admin-layout.html',
  styleUrls: ['./admin-layout.css']
})
export class AdminLayout {}
```

Una vez importados, configuramos la plantilla HTML (src/app/layouts/admin-layout/admin-layout.html) colocando las etiquetas correspondientes junto al conector de rutas:

```
<app-navbar></app-navbar>
<app-sidebar></app-sidebar>
<router-outlet />
```

Dado que el pie de página es un elemento visual que se mostrará tanto en la pantalla de inicio de sesión (AuthLayout) como en la zona de administración (AdminLayout), la práctica más eficiente es colocarlo directamente en la raíz de la aplicación para no duplicar código.

Para ello, importamos el componente Footer en src/app/app.ts:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Footer } from './commons/footer/footer';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Footer],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('frontend');
}
```

Y actualizamos la vista principal (src/app/app.html) para que dibuje el pie de página permanentemente debajo del enrutador dinámico:

```
<router-outlet />
<app-footer></app-footer>
```

El último paso es registrar las nuevas páginas en el sistema de navegación (src/app/app.routes.ts). Añadiremos un nuevo bloque para la ruta /admin, asociándola a AdminLayout y definiendo sus correspondientes vistas hijas:

```
import { Routes } from '@angular/router';
import { AuthLayout } from './layouts/auth-layout/auth-layout';
import { Login } from './auth/login/login';
import { AdminLayout } from './layouts/admin-layout/admin-layout';
```

```
import { Dashboard } from '../pages/dashboard/dashboard';
import { Users } from '../pages/users/users';

export const routes: Routes = [
  { path: '', redirectTo: 'auth', pathMatch: 'full' },
  {
    path: 'auth',
    component: AuthLayout,
    children: [
      { path: 'login', component: Login },
      { path: '', redirectTo: 'login', pathMatch: 'full' }
    ]
  },
  {
    path: 'admin',
    component: AdminLayout,
    children: [
      { path: 'dashboard', component: Dashboard },
      { path: 'users', component: Users },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  }
];
```

Con esta configuración implementada, si navegamos a <http://localhost:4200/admin/dashboard> observaremos cómo se integran correctamente los textos por defecto de la barra de navegación, el menú lateral, la vista dinámica del panel principal y el pie de página global:

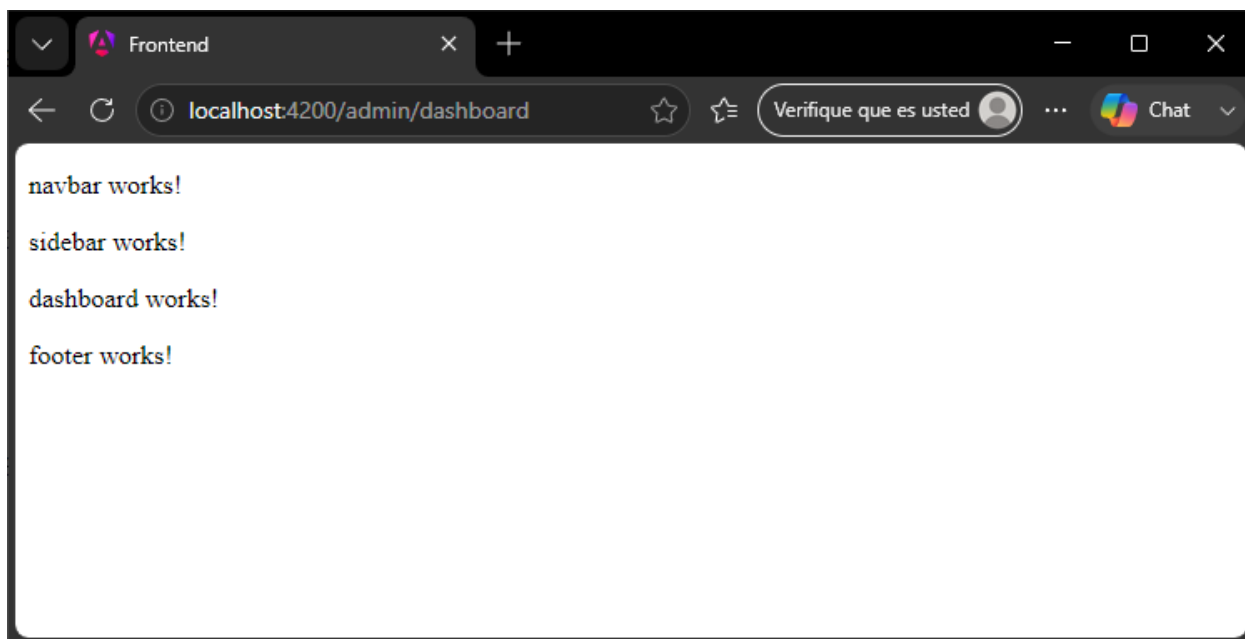


Figura 65. Vista del componente dashboard .

Si cambiamos la ruta a <http://localhost:4200/admin/users>, el único texto que cambiará en pantalla será el del área central, validando el correcto funcionamiento de nuestra arquitectura modular:

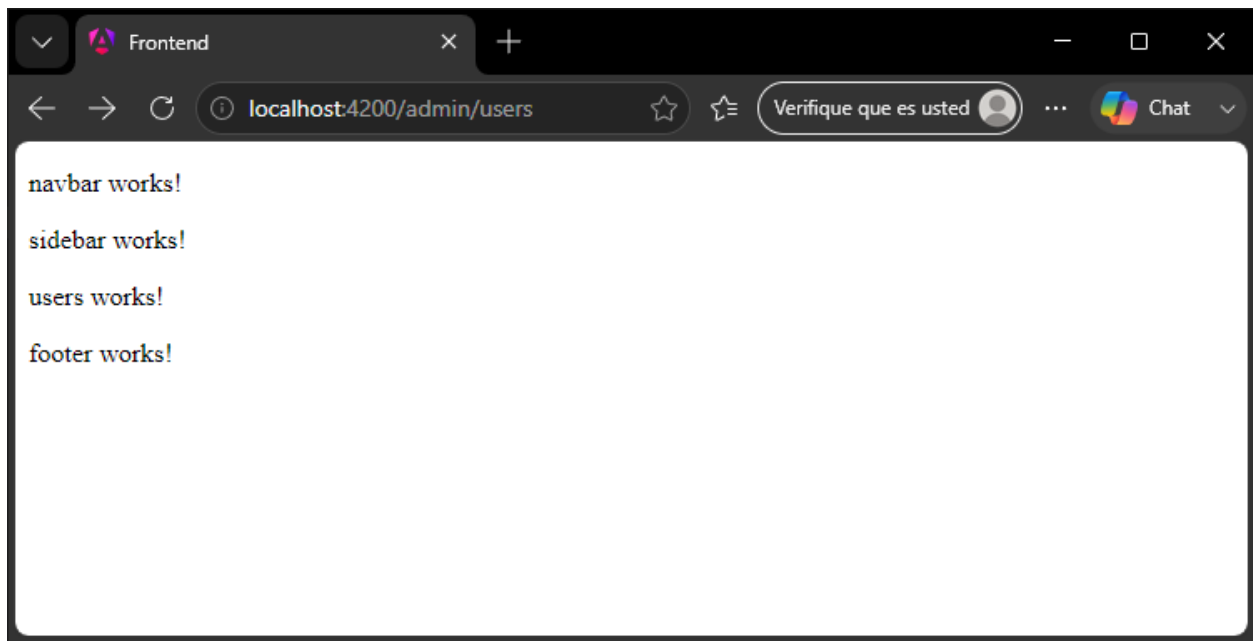


Figura 66. Vista del componente users .

4.5. 3.5. Arreglando rutas

Hasta ahora hemos configurado las rutas de nuestra aplicación de una forma funcional pero poco optimizada para el usuario final. Al utilizar prefijos, las direcciones web resultantes son `/auth/login` o `/admin/dashboard`. Además, esta estructura permite que un usuario acceda a las rutas base (`/auth` o `/admin`) visualizando las plantillas vacías sin contenido.

Para mejorar la experiencia y la seguridad, las rutas ideales y más limpias serían las siguientes:

- `/login` --> Renderiza `AuthLayout` y en su interior `Login`.
- `/dashboard` --> Renderiza `AdminLayout` y en su interior `Dashboard`.
- `/dashboard/users` --> Renderiza `AdminLayout` y en su interior `Users`.
- `/dashboard/*` (cualquier error) --> Redirige a `/dashboard`.
- `/*` (ruta desconocida) --> Redirige a `/login`.

Es decir, buscamos que la URL principal determine directamente la vista final del usuario, cargando el contenedor adecuado de fondo de manera transparente. Además, implementaremos rutas comodín (`**`) para atrapar cualquier dirección web errónea introducida manualmente y redirigirla a un punto seguro.

Para lograr este comportamiento, modificaremos nuestro archivo de rutas `src/app/app.routes.ts` de la siguiente forma:

```
import { Routes } from '@angular/router';
import { AuthLayout } from '../layouts/auth-layout/auth-layout';
import { Login } from '../auth/login/login';
import { AdminLayout } from '../layouts/admin-layout/admin-layout';
import { Dashboard } from '../pages/dashboard/dashboard';
import { Users } from '../pages/users/users';

export const routes: Routes = [
  {
    path: 'login',
    component: AuthLayout,
    children: [
```

```
{ path: '', component: Login }
]
},
{
  path: 'dashboard',
  component: AdminLayout,
  children: [
    { path: '', component: Dashboard },
    { path: 'users', component: Users },
    { path: '**', redirectTo: '' } // Atrapa subrutas erróneas (ej. /dash-
board/berenjena) y redirige al panel
  ]
},
{ path: '', redirectTo: 'login', pathMatch: 'full' },
{ path: '**', redirectTo: 'login' } // Atrapa cualquier ruta global no definida
y redirige al login
];
```

4.6. Descarga Código 6

Todo lo desarrollado hasta ahora está disponible en el “Codigo6.zip” que puedes descargar desde aquí:

[Descargar Código 6](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo6.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo6.zip>

4.7. Estructuración del código

Conforme la aplicación crece, añadir todas las rutas y sus respectivos componentes en un único archivo (app.routes.ts) se convierte en un problema de escalabilidad. Un archivo de configuración de rutas demasiado extenso es difícil de mantener, dificulta la lectura del código y puede generar conflictos cuando varios desarrolladores trabajan sobre él simultáneamente.

Para solucionar esto, aplicaremos el principio de separación de responsabilidades. Vamos a dividir las rutas de forma que cada sección de nuestra aplicación (auth y admin) tenga su propio archivo de configuración. Desde el enrutador principal, únicamente importaremos estos bloques.

Tendremos que crear archivos TypeScript independientes para agrupar las rutas hijas de cada área.

Creamos manualmente un archivo llamado **auth.routes.ts** dentro de la carpeta **src/app/auth/**. En este archivo definiremos únicamente lo que ocurre dentro de la zona de autenticación:

```
import { Routes } from '@angular/router';
import { Login } from '../login/login';

export const authRoutes: Routes = [
  { path: '', component: Login }
];
```

Del mismo modo, creamos un archivo llamado **admin.routes.ts** dentro de la carpeta **src/app/pages/**. Aquí

centralizaremos todas las rutas que se renderizarán dentro del panel de control:

```
import { Routes } from '@angular/router';
import { Dashboard } from '../dashboard/dashboard';
import { Users } from '../users/users';

export const adminRoutes: Routes = [
  { path: '', component: Dashboard },
  { path: 'users', component: Users },
  { path: '**', redirectTo: '' } // Redirección interna de la zona admin
];
```

Finalmente, modificamos el archivo central **src/app/app.routes.ts**. En lugar de tener toda la lógica anidada, importamos las variables **authRoutes** y **adminRoutes** que acabamos de crear y las asignamos a la propiedad **children** de sus respectivos contenedores:

```
import { Routes } from '@angular/router';
import { AuthLayout } from '../layouts/auth-layout/auth-layout';
import { AdminLayout } from '../layouts/admin-layout/admin-layout';

// Importación de las subrutas de autenticación y administración
import { authRoutes } from '../auth/auth.routes';
import { adminRoutes } from '../pages/admin.routes';

export const routes: Routes = [
  {
    path: 'login',
    component: AuthLayout,
    children: authRoutes
  },
  {
    path: 'dashboard',
    component: AdminLayout,
    children: adminRoutes
  },
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: '**', redirectTo: 'login' } // Atrapa cualquier ruta global no definida y redirige al login
];
```

Gracias a esta refactorización, nuestro enrutador principal (app.routes.ts) queda sumamente limpio, actuando como un índice o esqueleto. Si en el futuro necesitamos añadir decenas de páginas nuevas a la zona de administración, el archivo principal no se verá afectado; únicamente modificaremos el archivo especializado admin.routes.ts.

4.8. Descarga Código 7

Todo lo desarrollado hasta ahora está disponible en el “Codigo7.zip” que puedes descargar desde aquí:

[**Descargar Código 7**](#)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo7.zip>

5. Formato mediante plantilla y login

5.1. Plantilla matrix

Para dotar a la aplicación de un aspecto estructurado y una maquetación pulida de forma ágil, implementaremos una plantilla basada en el framework Bootstrap 4 denominada Matrix Admin. Dado que el objetivo principal del desarrollo se centra en la lógica con Angular, apoyarnos en esta plantilla gratuita nos permitirá disponer de un diseño responsivo y funcional sin necesidad de construir todas las hojas de estilo (CSS) desde cero.

El primer paso para adaptar esta plantilla a nuestro proyecto es incorporar sus recursos estáticos. Una vez descargado el paquete original de la plantilla, nos dirigimos a la siguiente ruta interna: **matrix-admin-package-full\matrix-admin-package\matrix-admin-bt4**.

De esta ubicación, necesitamos extraer el contenido de dos carpetas principales para unificarlas en el directorio de recursos públicos de nuestro proyecto en Angular (**public/assets**):

- De la carpeta interna assets de la plantilla, copiamos los directorios: extra-libs, images y libs.
- De la carpeta interna dist de la plantilla, copiamos los directorios con el código de diseño e interacción: css y js.

Pegamos estas cinco carpetas directamente dentro del directorio public/assets/ de nuestro proyecto Angular. La estructura de la carpeta public quedará organizada de la siguiente forma:

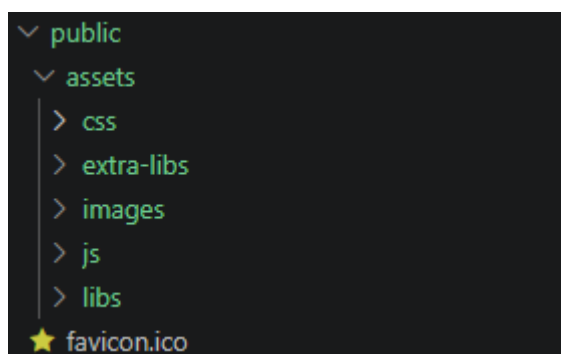


Figura 67. Estructura de la carpeta public.

Con estos recursos integrados en el sistema de archivos, la aplicación tendrá acceso a todas las hojas de estilo, scripts interactivos e imágenes necesarias para empezar a construir y renderizar las vistas dinámicas de la interfaz.

5.2. Fragmentación de la plantilla principal (dashboard)

La plantilla Matrix Admin dispone de dos estructuras visuales clave: una para la zona de administración (Dashboard) y otra para el acceso (Autenticación). A continuación, vamos a despiezar el archivo index.html original de la plantilla para incrustar su código de forma modular en nuestros componentes de Angular, centrándonos primero en la zona de gestión.

5.2.1. Configuración global (src/index.html)

El primer paso es preparar el archivo principal de Angular para que reconozca los estilos generales. Extraemos de la plantilla original todo el contenido de la etiqueta `<head>` (metadatos, enlaces a las hojas de estilo CSS personalizadas e iconos) y las etiquetas `<script>` ubicadas al final del documento.

Pegamos estos elementos en nuestro archivo **src/index.html**, asegurándonos de corregir las rutas para que apunten a nuestra carpeta estática mediante rutas absolutas, quedando de la siguiente forma:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Frontend</title>
  <base href="/">
  <!-- Metadatos de la plantilla -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/png" sizes="16x16" href="/assets/images/favi-
con.png">

  <!-- Hojas de estilo CSS de la plantilla -->
  <link href="/assets/libs/flot/css/float-chart.css" rel="stylesheet">
  <link href="/assets/css/style.min.css" rel="stylesheet">
</head>
<body>

  <!-- Contenedor principal de Angular -->
  <app-root></app-root>

  <!-- Scripts JS de la plantilla -->
  <script src="/assets/libs/jquery/dist/jquery.min.js"></script>
  <script src="/assets/libs/perfect-scrollbar/dist/perfect-scroll-
bar.jquery.min.js"></script>
  <script src="/assets/libs/bootstrap/dist/js/bootstrap.min.js"></script>
  <!-- (Aquí se incluyen el resto de scripts originales de la plantilla con el
prefijo /assets/) -->
  <script src="/assets/js/custom.min.js"></script>

</body>
</html>
```

5.2.2. Componentización de elementos comunes (navbar, sidebar, footer)

Abrimos el archivo index.html original de la plantilla matrix, que se encuentra en la carpeta **/html** del archivo matrix-admin-bt4.zip . Vamos a extraer los grandes bloques estructurales HTML para pegarlos en las vistas de nuestros componentes de Angular, sustituyendo el texto por defecto que traen al generarlos.

- **Barra de navegación (Navbar):** Buscamos el bloque que comienza con la etiqueta `<header class="topbar" data-navbarbg="skin5">` y copiamos todo su contenido hasta la etiqueta de cierre `</header>`. Pegamos este código en nuestro archivo **src/app/commons/navbar/navbar.html**.
- **Menú lateral (Sidebar):** Buscamos el bloque que empieza por la etiqueta `<aside class="left-sidebar" data-sidebarbg="skin5">` y copiamos todo hasta su cierre `</aside>`. Lo pegamos dentro de nuestro archivo **src/app/commons/sidebar/sidebar.html**.

- **Pie de página (Footer):** Buscamos la etiqueta `<footer class="footer text-center">`, copiamos ese bloque hasta `</footer>` y lo pegamos en nuestro archivo **src/app/commons/footer/footer.html**.

5.2.3. Creación del componente breadcrumb

La plantilla incluye un indicador de navegación en la parte superior del contenido para mostrarle al usuario en qué sección se encuentra. Generamos un nuevo componente en nuestra carpeta común:

ng generate component commons/breadcrumb --skip-tests

Una vez generado, volvemos al `index.html` de la plantilla, buscamos el bloque `<div class="page-breadcrumb">` y lo copiamos entero hasta su correspondiente `</div>` de cierre. Lo pegamos dentro de **src/app/commons/breadcrumb/breadcrumb.html**. Tras esto, nos aseguramos de importar este nuevo componente dentro de nuestro contenedor `AdminLayout` para que quede de la siguiente forma:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Sidebar } from '../commons/sidebar/sidebar';
import { Navbar } from '../commons/navbar/navbar';
import { Breadcrumb } from '../commons/breadcrumb/breadcrumb';

@Component({
  selector: 'app-admin-layout',
  imports: [RouterOutlet, Sidebar, Navbar, Breadcrumb],
  templateUrl: './admin-layout.html',
  styleUrls: ['./admin-layout.css']
})
export class AdminLayout {}
```

5.2.4. Ensamblaje en el contenedor admin-layout

Finalmente, estructuramos el contenedor principal de la zona de gestión (`src/app/layouts/admin-layout/admin-layout.html`). Este archivo actuará como el esqueleto central que une todas las piezas que acabamos de recortar.

Del archivo original aprovecharemos los contenedores estructurales (`main-wrapper` y `page-wrapper`) y sustituiremos el resto por nuestros propios selectores de Angular, dejando el espacio dinámico en el centro:

```
<!-- Envoltorio principal de la plantilla -->
<div id="main-wrapper">

  <!-- Componentes fijos superiores y laterales -->
  <app-navbar></app-navbar>
  <app-sidebar></app-sidebar>

  <!-- Envoltorio de la página dinámica -->
  <div class="page-wrapper">

    <!-- Migas de pan -->
    <app-breadcrumb></app-breadcrumb>

    <!-- Contenedor central dinámico donde actúa el enrutador -->
    <div class="container-fluid">
      <router-outlet></router-outlet>
    </div>

  </div>

</div>
```

</div>

Tras este último cambio ya podemos ejecutar el servicio con ng serve. Observaremos que la página dashboard y la página dashboard/users no parecen estar cargando la fuente que se utiliza para representar los iconos de la web. Esto lo solucionaremos en el siguiente punto.

5.2.5. Solución problema con la fuente de los iconos

Para resolver este problema que viene por defecto con la plantilla que estamos utilizando deberemos de acceder al archivo **materialdesignicons.min.css**. Editamos el archivo de forma que busquemos la cadena **//fonts/** y la reemplazamos por **/fonts/** para corregir el error de sintaxis nativo que impedía la lectura de la ruta.

Ahora si ejecutamos el servicio si que deberíamos de ver algo así:

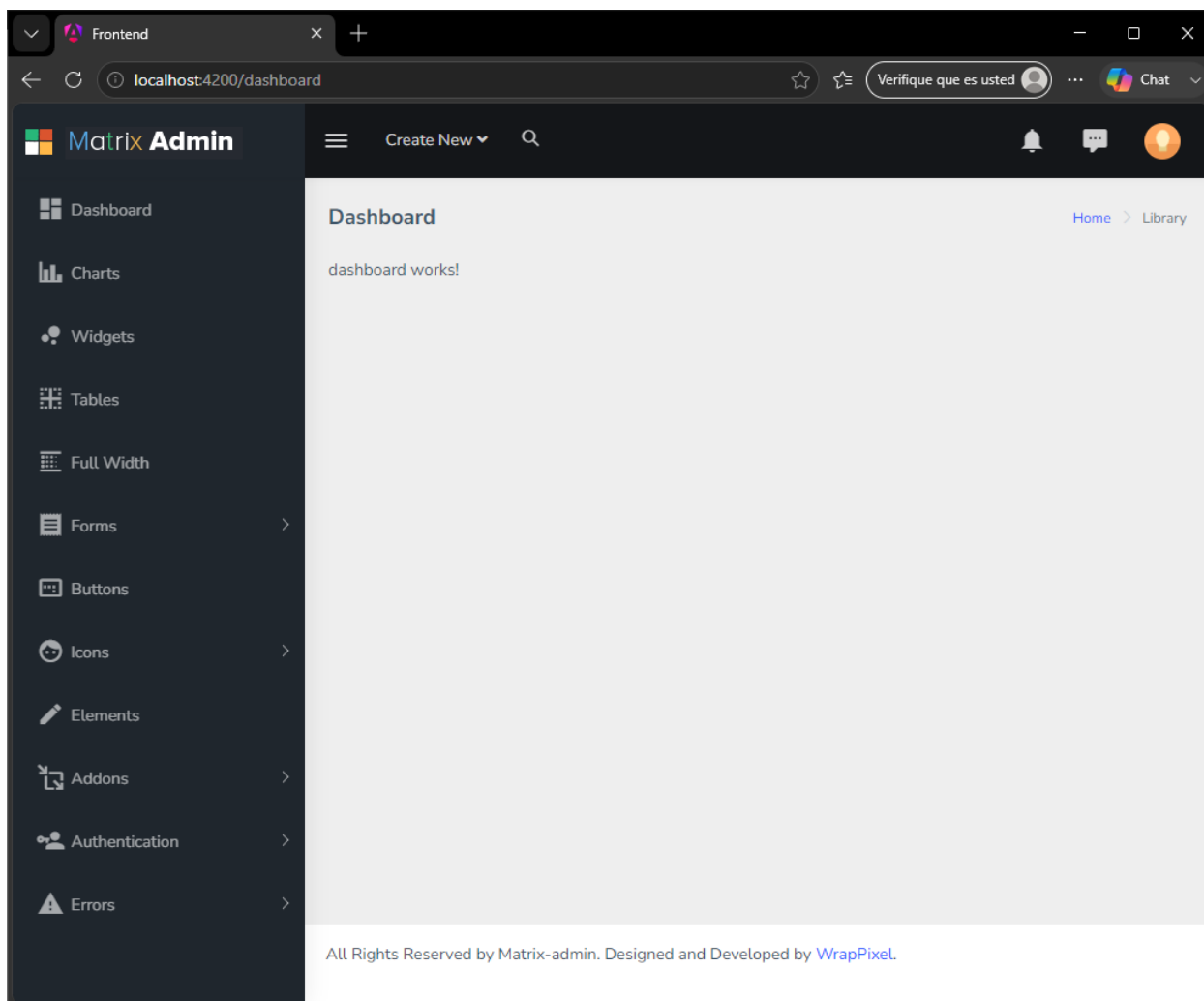


Figura 68. Resultado en la web de aplicar la plantilla.

5.3. Descarga Código 8

Todo lo desarrollado hasta ahora está disponible en el “Codigo8.zip” que puedes descargar desde aquí:

Descargar Código 8

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo8.zip>

5.4. Fragmentación de la plantilla de autenticación (login)

Una vez configurada la zona de gestión (Dashboard), procedemos a estructurar la zona pública de la aplicación, correspondiente al inicio de sesión. Para ello, utilizaremos el archivo authentication-login.html incluido en la carpeta `/html` de la plantilla original.

A continuación, separaremos la vista del formulario de acceso en un componente independiente (Login), el cual se cargará dentro de un contenedor estructural dedicado para las rutas públicas (AuthLayout).

5.4.1. Ensamblaje en el contenedor AuthLayout

El primer paso es preparar el esqueleto que envolverá la página de acceso. Abrimos el archivo original authentication-login.html y localizamos el contenedor principal que da formato a toda la pantalla.

Copiamos el bloque que comienza con la etiqueta `<div class="auth-wrapper d-flex no-block...">` y termina en su correspondiente cierre `</div>`. Lo pegamos en nuestro archivo `src/app/layouts/auth-layout/auth-layout.html`, vaciando su interior para sustituir el formulario estático por el selector del enrutador de Angular:

```
<div class="auth-wrapper d-flex no-block justify-content-center align-items-center bg-dark">
  <div class="auth-box bg-dark border-top border-secondary">

    <!-- Contenedor dinámico donde se cargará el formulario de Login -->
    <router-outlet></router-outlet>

  </div>
</div>
```

5.4.2. Extracción del componente login

Volvemos al archivo authentication-login.html de la plantilla original para extraer exclusivamente el código correspondiente al formulario de inicio de sesión:

- Buscamos el bloque identificado con la etiqueta `<div id="loginform">`.
- Copiamos todo el contenido desde esa etiqueta hasta su cierre `</div>`.
- Pegamos el código íntegro en la vista de nuestro componente de inicio de sesión: **src/app/auth/login/login.html**.

El archivo final login.html debería quedar tal que así:

```
<div id="loginform" class="px-4 pb-4">
  <div class="text-center pt-4 pb-4">
    <span class="db"></span>
  </div>
  <!-- Form -->
  <form class="form-horizontal mt-4" id="loginform" action="index.html">
```

```

<div class="row pb-4">
  <div class="col-12">
    <div class="input-group mb-3">
      <div class="input-group-prepend">
        <span class="input-group-text bg-success text-white" id="basic-ad-
addon1"><i class="ti-user"></i></span>
      </div>
      <input type="text" class="form-control form-control-lg" place-
holder="Username" aria-label="Username" aria-describedby="basic-addon1" re-
quired="">
    </div>
    <div class="input-group mb-3">
      <div class="input-group-prepend">
        <span class="input-group-text bg-warning text-white" id="basic-ad-
addon2"><i class="ti-pencil"></i></span>
      </div>
      <input type="text" class="form-control form-control-lg" place-
holder="Password" aria-label="Password" aria-describedby="basic-addon1" re-
quired="">
    </div>
  </div>
</div>
<div class="row border-top border-secondary">
  <div class="col-12">
    <div class="form-group">
      <div class="pt-4 text-center">
        <button class="btn btn-success" type="submit">Login</button>
      </div>
    </div>
  </div>
</div>
</form>
</div>

```

5.4.3. Ajuste de dimensiones del contenedor principal (footer)

Finalmente, al previsualizar la pantalla de acceso, observaremos que el pie de página (footer) queda desplazado fuera del límite visual, obligando al usuario a desplazarse hacia abajo de forma innecesaria. Esto ocurre porque el contenedor auth-wrapper tiene una altura mínima del 100% de la ventana.

Para corregirlo, realizamos un ajuste fino en la hoja de estilos global:

1. En public/assets/css/style.min.css, buscamos la clase .auth-wrapper. El texto que debemos encontrar en el archivo es este

```
.auth-wrapper{min-height:100vh;position:relative}
```

2. Localizamos su propiedad min-height: 100vh;.
3. Sustituimos ese valor por un cálculo dinámico que reste el tamaño aproximado del footer: min-height: calc(100vh - 60px). Es importante colocar este texto tal cual, porque sin los espacios no lo detecta correctamente.

Con esta modificación, la maquetación se ajustará perfectamente a los bordes de la pantalla en la vista de acceso independientemente de la resolución de esta.

5.5. Routerlink

Para gestionar la navegación entre las distintas vistas de la aplicación en Angular, no se utilizan las etiquetas HTML tradicionales con el atributo href, sino el sistema de enrutamiento nativo del framework. Esto evita que

el navegador recargue la página completa en cada clic, ofreciendo una experiencia fluida. Para indicar las rutas y que Angular las maneje adecuadamente, vamos a añadir a las etiquetas correspondientes la directiva `routerLink="/route"`.

Para aplicar este concepto de forma práctica en nuestro proyecto, vamos a adaptar el menú de navegación lateral (**src/app/commons/sidebar/sidebar.html**), al cual inicialmente le introdujimos el código del elemento sidebar del archivo `index.html` por defecto de la plantilla, y hay que limpiarlo.

Nuestra barra lateral únicamente debe contener dos accesos: "Dashboard" y "Users". Por lo tanto tendremos que eliminar todo el código sobrante. El archivo limpio debería quedar así:

```
<aside class="left-sidebar" data-sidebarbg="skin5">
  <!-- Sidebar scroll-->
  <div class="scroll-sidebar">
    <!-- Sidebar navigation-->
    <nav class="sidebar-nav">
      <ul id="sidebarnav" class="p-t-30">
        <li class="sidebar-item">
          <a class="sidebar-link waves-effect waves-dark sidebar-link" router-
Link="/dashboard" aria-expanded="false">
            <i class="mdi mdi-home"></i><span class="hide-menu">Dashboard</span>
          </a>
        </li>
        <li class="sidebar-item">
          <a class="sidebar-link waves-effect waves-dark sidebar-link" router-
Link="/dashboard/users" aria-expanded="false">
            <i class="mdi mdi-account-multiple"></i><span class="hide-menu">Us-
ers</span>
          </a>
        </li>
      </ul>
    </nav>
    <!-- End Sidebar navigation -->
  </div>
  <!-- End Sidebar scroll-->
</aside>
```

Podemos ver que, en vez de contener una propiedad llamada `href`, los elementos ahora tienen una que se llama `routerLink`, esta es la que nos permite navegar entre las diferentes vistas de una aplicación de una sola página (SPA) sin recargar la página completa.

Finalmente, para que Angular reconozca e interprete correctamente esta nueva directiva dentro de este componente, es obligatorio importar la herramienta `RouterLink` desde `@angular/router` en el archivo controlador de la barra lateral (`sidebar.ts`) de forma que el archivo quede así:

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-sidebar',
  imports: [RouterLink],
  templateUrl: './sidebar.html',
  styleUrls: ['./sidebar.css'],
})
export class Sidebar {}
```

5.5.1. Limpieza del resto de /commons

Tras la fragmentación inicial de la plantilla en componentes independientes (navbar, sidebar, footer y

breadcrumb), los archivos HTML conservaban múltiples elementos interactivos de demostración que no forman parte de nuestra aplicación. Por ello tenemos que limpiar nuestros archivos a los que introdujimos el código de la plantilla directamente, dejando solo lo que vamos a emplear.

navbar.html quedaría así:

```
<header class="topbar" data-navbarbg="skin5">
  <nav class="navbar top-navbar navbar-expand-md navbar-dark">
    <div class="navbar-header" data-logobg="skin5">
      <a class="navbar-brand" routerLink="/dashboard">
        <b class="logo-icon p-l-10">
          
        </b>
        <span class="logo-text">
          
        </span>
      </a>
      <a class="nav-toggler waves-effect waves-light d-block d-md-none"
href="javascript:void(0)"><i class="ti-menu ti-close"></i></a>
    </div>
    <div class="navbar-collapse collapse" id="navbarSupportedContent" data-
navbarbg="skin5">
      <ul class="navbar-nav float-left mr-auto">
        <li class="nav-item d-none d-md-block">
          <a class="nav-link sidebartoggler waves-effect waves-light" href="ja-
vascript:void(0)" data-sidebar-type="mini-sidebar"><i class="mdi mdi-menu font-
24"></i></a>
        </li>
      </ul>
      <ul class="navbar-nav float-right">
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle text-muted waves-effect waves-dark
pro-pic" href="" data-toggle="dropdown" aria-haspopup="true" aria-ex-
panded="false"></a>
          <div class="dropdown-menu dropdown-menu-right user-dd animated">
            <a class="dropdown-item" href="javascript:void(0)"><i class="fa fa-
power-off m-r-5 m-l-5"></i> Log out</a>
          </div>
        </li>
      </ul>
    </div>
  </nav>
</header>
```

Como hemos incluido un routerLink en el logotipo Matrix Admin de la página navbar.html, deberemos de importar RouterLink en nuestro archivo **navbar.ts**, de la misma forma que lo hicimos con sidebar.ts.

breadcrumb.html quedaría así:

```
<div class="page-breadcrumb">
  <div class="row">
    <div class="col-12 d-flex no-block align-items-center">
      <h4 class="page-title">Dashboard</h4>
    </div>
  </div>
</div>
```

En el archivo footer.html podemos poner lo que queramos, normalmente se ponen datos legales, enlaces de navegación y formas de contacto. En este caso podemos poner algo como "Angular app SMA 2026", de forma que quedaría así:

```
<footer class="footer text-center">
  Angular app SMA 2026
</footer>
```

5.6. Arrancar backend y crear JWT

Para empezar a dar vida a la interfaz de Angular y enlazar el entorno público con el privado, el primer paso es levantar nuestro servidor backend. Esto lo realizamos directamente desde la consola de comandos, accediendo al directorio del backend y ejecutando nuestro script de inicio de desarrollo:

npm run start:dev

5.6.1. Generación de un token comodín para pruebas

Durante el desarrollo, estaremos realizando múltiples pruebas con herramientas como Postman. Nuestro backend tiene programada una caducidad estricta para los tokens, lo que resulta muy incómodo a la hora de programar porque nos obligaría a estar iniciando sesión continuamente cada vez que el token expire.

Para agilizar el trabajo y evitar interrupciones, podemos generar un token de larga duración de forma manual a través de la web JWT.io. El proceso que debemos seguir es el siguiente:

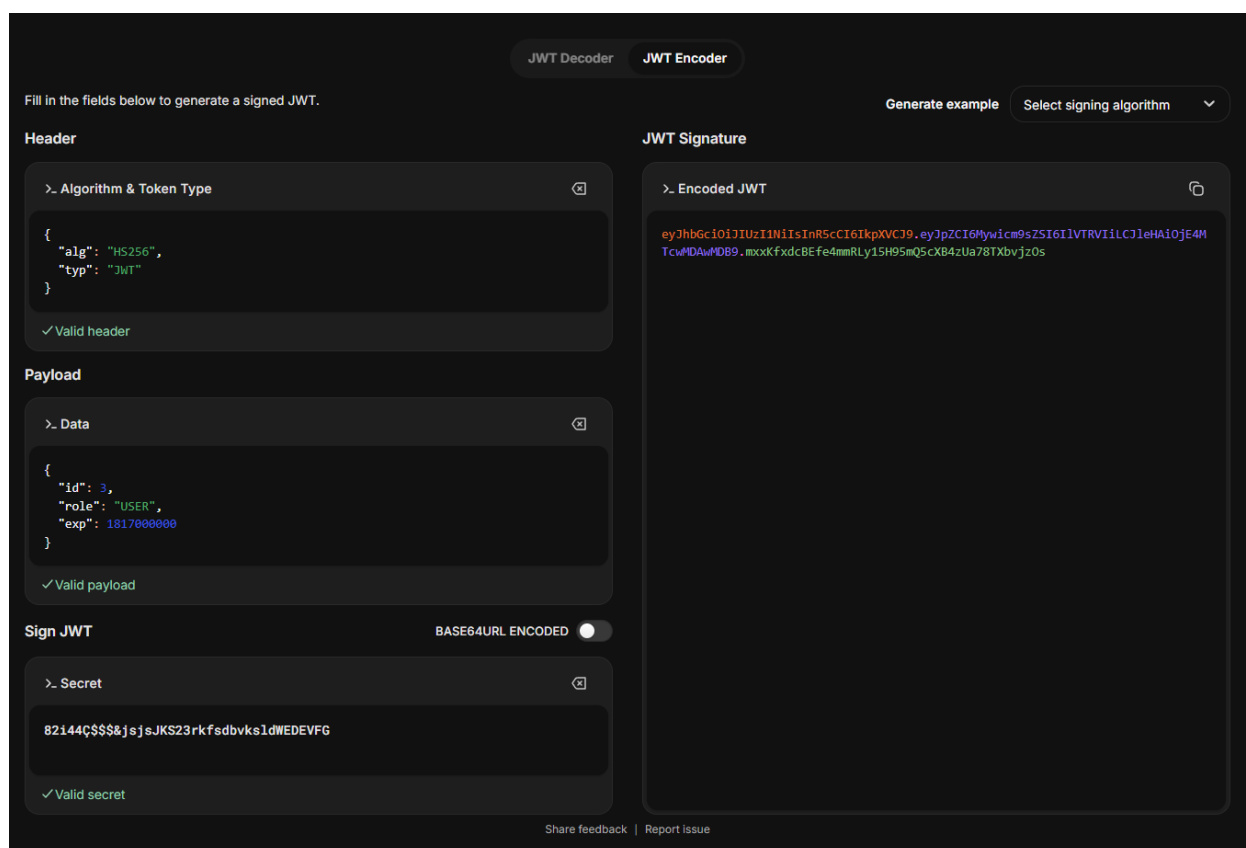
- 1º Accederemos a la página web de JWT.io, y en esta buscamos la sección de JWT Encoder.
- 2º En la sección del Payload (datos), introducir el identificador (id) y el role de un usuario que tengamos en nuestra base de datos. Que podemos ver en DB Browser. Yo utilizaré la información de este, por ejemplo:

3	Alejandro	ale427@alu.ua.es	\$2b\$10\$P/1iThavda9S4Fd/Ijkr5e9C.KKS0Aq9LRzVp2n5vk4cV9t6C.pCG	USER	2026-07-29 10:56:29	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ...
---	-----------	------------------	---	------	---------------------	--

Figura 69. Información mostrada en DB Browser.

- 3º Añadir o modificar la fecha de expiración (exp) para ponerle una validez muy amplia, por ejemplo, de un año.
- 4º En la sección Verify Signature, firmar el token utilizando exactamente la misma clave secreta (JWTSECRET) que tenemos configurada en el archivo .env de nuestro backend.

Nuestra página de JWT.io debería de verse así después de configurarlo todo:



The screenshot shows a web application for encoding JWT tokens. It has two tabs: 'JWT Decoder' and 'JWT Encoder', with 'JWT Encoder' selected. The interface is divided into two main sections: 'Header' and 'Payload'. The 'Header' section contains a text area for the header JSON, which is currently empty. Below it, a 'Valid header' status is shown. The 'Payload' section contains a text area for the payload JSON, which is currently empty. Below it, a 'Valid payload' status is shown. To the right of these sections is a 'JWT Signature' section, which contains a text area for the encoded JWT. The encoded JWT is displayed as a long string of characters. At the bottom of the 'Header' section, there is a 'Sign JWT' button and a 'BASE64URL ENCODED' toggle switch. Below the 'Sign JWT' button is a text area for the secret key, which is currently empty. Below it, a 'Valid secret' status is shown. At the bottom right of the application, there are links for 'Share feedback' and 'Report issue'.

Figura 70. Resultado de codificar JWT.

Al usar la misma clave secreta, nuestro backend aceptará este token generado manualmente como válido. De esta forma, obtenemos un token "comodín" que podemos colocar en las cabeceras de Postman para probar todas nuestras rutas privadas sin preocuparnos por su caducidad mientras programamos el resto de la aplicación.

El token lo utilizaremos de la misma forma que el que necesita renovarse, lo mandaremos en la header "x-token" y en el parámetro "value" pondremos el token.

5.7. Validación formulario - Template vs Reactivo

Una de las características más potentes de Angular es el control absoluto que proporciona sobre los formularios, permitiendo capturar datos, detectar cambios en tiempo real y ejecutar validaciones de forma eficiente. Para implementar este control, Angular ofrece dos estrategias principales: formularios basados en plantillas (*Template-driven forms*) y formularios reactivos (*Reactive forms*).

5.7.1. Formularios basados en plantillas (Template-driven)

En este enfoque, el control del formulario se delega casi en su totalidad al archivo HTML de la vista. Las reglas de validación se insertan directamente en las etiquetas a través de directivas (como `required`, `minlength`, etc.). Aunque es una forma rápida de configurar formularios sencillos, presenta un gran inconveniente: a medida que el formulario crece, el código HTML se vuelve muy complejo, difícil de mantener y termina mezclando la lógica de negocio con la maquetación visual.

5.7.2. Formularios Reactivos (Reactive forms)

El enfoque reactivo propone gestionar el estado y la validación del formulario de forma programática, centralizando toda la lógica en el componente TypeScript y manteniendo la plantilla HTML lo más limpia posible. En lugar de utilizar directivas, creamos un modelo de datos en el controlador y lo vinculamos a la vista.

Este es el enfoque recomendado por Angular para la mayoría de los casos, ya que ofrece múltiples ventajas:

- **Separación de responsabilidades:** La lógica de validación se queda en el controlador (TypeScript), permitiendo que el HTML se limite a su función principal de maquetación y diseño visual.
- **Escalabilidad:** Es mucho más sencillo gestionar formularios con múltiples campos, dependencias entre ellos o validaciones personalizadas complejas.
- **Trazabilidad:** Al basarse en flujos reactivos, es más fácil rastrear los cambios de valor y estado de cada campo en tiempo real.

Para cumplir con las mejores prácticas de desarrollo y asegurar un código escalable, en este proyecto implementaremos exclusivamente formularios reactivos.

5.8. Login y formularios reactivos

Para implementar la captura de datos en la pantalla de inicio de sesión, aplicaremos el enfoque de formularios reactivos. Este componente requiere dos campos: correo electrónico (email) y contraseña (password).

Las herramientas de importación se declararán directamente en el componente que las vaya a utilizar.

5.8.1. Configuración del controlador (login.ts)

Abrimos el archivo **src/app/auth/login/login.ts**. Para poder construir el formulario, importaremos `ReactiveFormsModule` y lo añadiremos al array de imports.

A continuación, inyectaremos el servicio `FormBuilder` a través del constructor. Este objeto proporciona una sintaxis simplificada para crear y agrupar los controles del formulario. Finalmente, definimos la variable `loginForm` y el método `login()` que capturará el evento de envío. El archivo `login.ts` debería quedar así:

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {

  // Declaramos la variable indicando que será de tipo FormGroup
  public loginForm: FormGroup;

  // Inyectamos FormBuilder y asignamos los valores dentro del constructor
  constructor(private fb: FormBuilder) {
    this.loginForm = this.fb.group({
      email: [''],
      password: ['']
    });
  }
}
```

```

    });
}

// Función que se ejecutará al pulsar el botón de submit
login() {
  console.log(this.loginForm.value);
}
}

```

5.8.2. Enlace del formulario en la vista (login.html)

Una vez creado el modelo de datos en el controlador TypeScript, el siguiente paso es enlazarlo físicamente con el código HTML de la plantilla en `src/app/auth/login/login.html`.

1º En la etiqueta `<form>`, eliminamos el atributo tradicional `action="index.html"`. Lo sustituimos por la directiva `[formGroup]="loginForm"` para que Angular asuma el control del formulario, y añadimos el evento `(ngSubmit)="login()"` para interceptar el clic del usuario.

2º En las etiquetas `<input>`, añadimos la propiedad `formControlName` para vincular cada campo visual con su variable correspondiente en el controlador.

El bloque del formulario quedará actualizado de la siguiente manera:

```

<!-- Form -->
<form class="form-horizontal mt-4" [formGroup]="loginForm" (ngSubmit)="login()">
  <div class="row pb-4">
    <div class="col-12">
      <div class="input-group mb-3">
        <div class="input-group-prepend">
          <span class="input-group-text bg-success text-white" id="basic-addon1"><i class="ti-user"></i></span>
        </div>
        <!-- Campo Email -->
        <input type="email" class="form-control form-control-lg" formControlName="email" placeholder="Email" aria-label="Email" aria-describedby="basic-addon1">
      </div>
      <div class="input-group mb-3">
        <div class="input-group-prepend">
          <span class="input-group-text bg-warning text-white" id="basic-addon2"><i class="ti-pencil"></i></span>
        </div>
        <!-- Campo Password -->
        <input type="password" class="form-control form-control-lg" formControlName="password" placeholder="Password" aria-label="Password" aria-describedby="basic-addon1">
      </div>
    </div>
  </div>
  <div class="row border-top border-secondary">
    <div class="col-12">
      <div class="form-group">
        <div class="pt-4 text-center">
          <button class="btn btn-success" type="submit">Login</button>
        </div>
      </div>
    </div>
  </div>
</form>

```

Con esta configuración, el comportamiento nativo del HTML queda anulado. Cada vez que el usuario pulse el botón de acceso, Angular interceptará los datos introducidos y los imprimirá en la consola de depuración del navegador a través de la función `login()`, confirmando el correcto funcionamiento del enlace reactivo, como podemos ver en la consola de la siguiente captura:

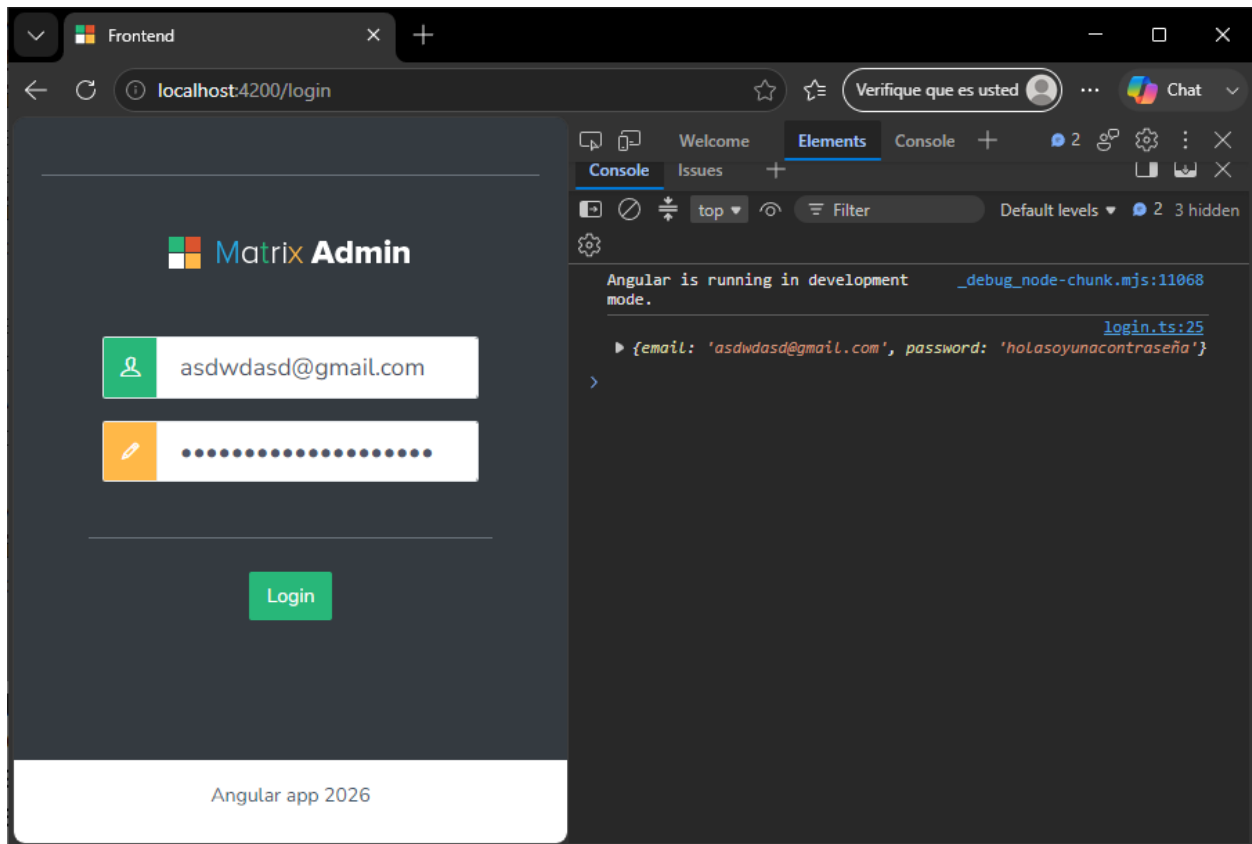


Figura 71. Interfaz login.

5.8.3. Validación del formulario

Para asegurar que los datos introducidos por el usuario son correctos antes de enviarlos al servidor, aplicaremos reglas de validación. Gracias al enfoque reactivo, esta lógica se define directamente en el controlador, evitando saturar el código HTML con directivas.

Angular proporciona la clase genérica `Validators`, la cual incluye múltiples métodos integrados para comprobar el estado de los campos. En nuestro caso de uso, exigiremos que ambos campos sean obligatorios y que el campo de correo tenga un formato válido.

5.8.4. Actualización del controlador (login.ts)

Abrimos el archivo `src/app/auth/login/login.ts` y modificamos la importación para incluir la clase `Validators`. A continuación, actualizamos la inicialización del grupo de controles dentro del constructor.

A partir de ahora, pasaremos un array a cada propiedad: el primer parámetro será el valor por defecto (una cadena vacía) y el segundo parámetro será un array con las validaciones requeridas. Finalmente, añadimos una estructura condicional en la función `login()` para evaluar la validez global del formulario antes de procesarlo.

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {

  public loginForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.loginForm = this.fb.group({
      // Campo obligatorio y con formato de correo electrónico
      email: ['', [Validators.required, Validators.email]],
      // Campo obligatorio
      password: ['', [Validators.required]]
    });
  }

  login() {
    // Comprobamos el estado global del formulario antes de actuar
    if (this.loginForm.status === 'VALID') {
      console.log('Formulario correcto, listo para enviar al servidor.');
```

Con esta modificación, el objeto loginForm evalúa en tiempo real la información que el usuario escribe. Si se intenta enviar el formulario dejando campos en blanco o introduciendo un texto que no corresponde a un correo electrónico en el campo email, la propiedad status del formulario devolverá INVALID. Esto ejecutará la rama de error en la consola, aislando la lógica de negocio y evitando realizar llamadas incompletas o erróneas a nuestro backend.

5.8.5. Feedback visual de validación en tiempo real

Una vez establecido el control reactivo y las reglas de validación en el formulario, el siguiente paso es conectar este estado interno con la interfaz gráfica. El objetivo es mostrar mensajes de error dinámicos que guíen al usuario cuando introduce datos incorrectos.

Dado que el controlador actualiza el estado del formulario de manera reactiva ante cada cambio, la sincronización con la vista es instantánea. Sin embargo, para ofrecer una buena experiencia de usuario, debemos evitar que los mensajes de error aparezcan nada más cargar la página, cuando el usuario aún no ha tenido la oportunidad de rellenar los campos.

Para lograr esto, implementaremos una variable de control que registre si se ha intentado enviar el formulario, y una función que determine cuándo es el momento adecuado para mostrar el error.

5.8.6. Actualización del controlador (login.ts)

Abrimos el archivo **src/app/auth/login/login.ts** e introducimos las siguientes modificaciones:

- 1º Añadimos una variable pública formSubmit inicializada en false.
- 2º Creamos el método isFieldValid(field: string) que evaluará si un campo específico es válido. Esta función devolverá true (ocultando el error) si el campo cumple los requisitos de validación, o si el formulario aún no ha intentado enviarse.
- 3º Actualizamos la función login() para cambiar el estado de formSubmit a true en el momento en que el usuario pulsa el botón.

El código resultante es el siguiente:

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {

  public loginForm: FormGroup;
  public formSubmit: boolean = false;

  constructor(private fb: FormBuilder) {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  // Comprueba si el campo es válido o si aún no se ha intentado enviar
  isFieldValid(field: string): boolean {
    return this.loginForm.get(field)?.valid || !this.formSubmit;
  }

  login() {
    // Registramos que el usuario ha intentado enviar el formulario
    this.formSubmit = true;

    if (this.loginForm.status === 'VALID') {
      console.log('Formulario correcto, listo para enviar al servidor.');
```

```
      console.log(this.loginForm.value);
    } else {
      console.log('Error: Campos incompletos o con formato incorrecto.');
```

```
    }
  }
}
```

5.8.7. Inserción de los mensajes en la vista (login.html)

Con la lógica de control preparada, procedemos a insertar los mensajes de alerta en el archivo **src/app/auth/login/login.html**.

Utilizaremos la directiva estructural @if de Angular para renderizar condicionalmente una etiqueta <p> debajo

de cada campo de entrada. Esta etiqueta solo se mostrará si la función `isFieldValid()` devuelve `false` (es decir, si el campo tiene errores y el usuario ya ha pulsado el botón de inicio de sesión). Aplicaremos clases de Bootstrap como `text-danger` para dar color rojo al texto de advertencia.

El código HTML de los campos quedará configurado de la siguiente manera:

```
<!-- Campo Email -->
<div class="input-group mb-3">
  <div class="input-group-prepend">
    <span class="input-group-text bg-success text-white" id="basic-addon1"><i class="ti-user"></i></span>
  </div>
  <input type="email" class="form-control form-control-lg" formControlName="email" placeholder="Email" aria-label="Email" aria-describedby="basic-addon1">
</div>
<!-- Alerta de validación para el Email -->
@if (!isFieldValid('email')){
  <p class="text-danger mt-1 mb-3">
    El usuario es obligatorio y debe ser un correo electrónico válido.
  </p>
}

<!-- Campo Password -->
<div class="input-group mb-3">
  <div class="input-group-prepend">
    <span class="input-group-text bg-warning text-white" id="basic-addon2"><i class="ti-pencil"></i></span>
  </div>
  <input type="password" class="form-control form-control-lg" formControlName="password" placeholder="Password" aria-label="Password" aria-describedby="basic-addon1">
</div>
<!-- Alerta de validación para el Password -->
@if (!isFieldValid('password')){
  <p class="text-danger mt-1 mb-3">
    El campo contraseña es obligatorio.
  </p>
}
```

Mediante esta integración, la interfaz de la aplicación de Angular permanece limpia al inicio y solo expone las advertencias visuales en el momento preciso en el que el usuario requiere correcciones, mejorando sustancialmente la interactividad de la zona de autenticación.

Podemos observar que el formulario no muestra errores nada más accedemos a la página:

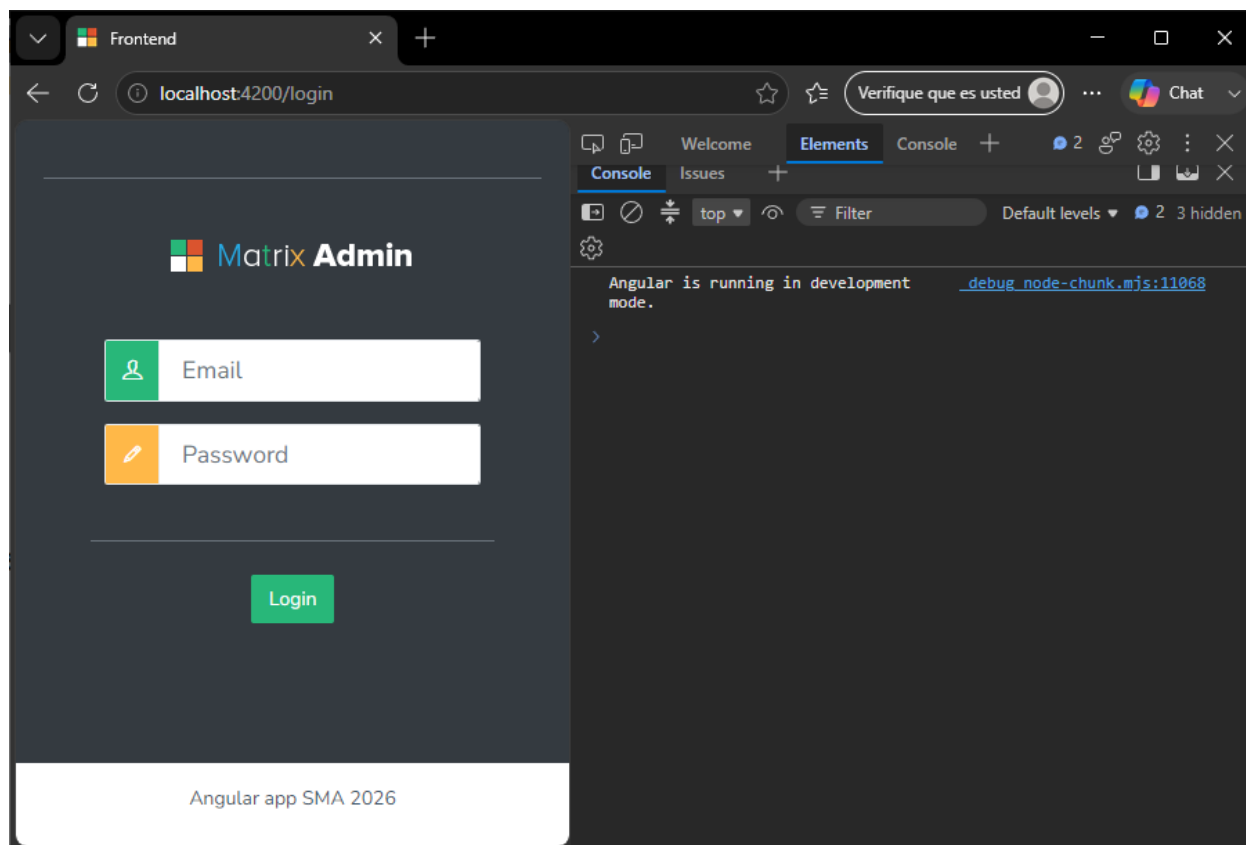


Figura 72. Resultado de los cambios.

Pero si pulsamos el botón de login sin escribir datos por ejemplo, saldrán los errores con total normalidad, puesto que ya se habrá realizado un intento de login y formSubmit será true:

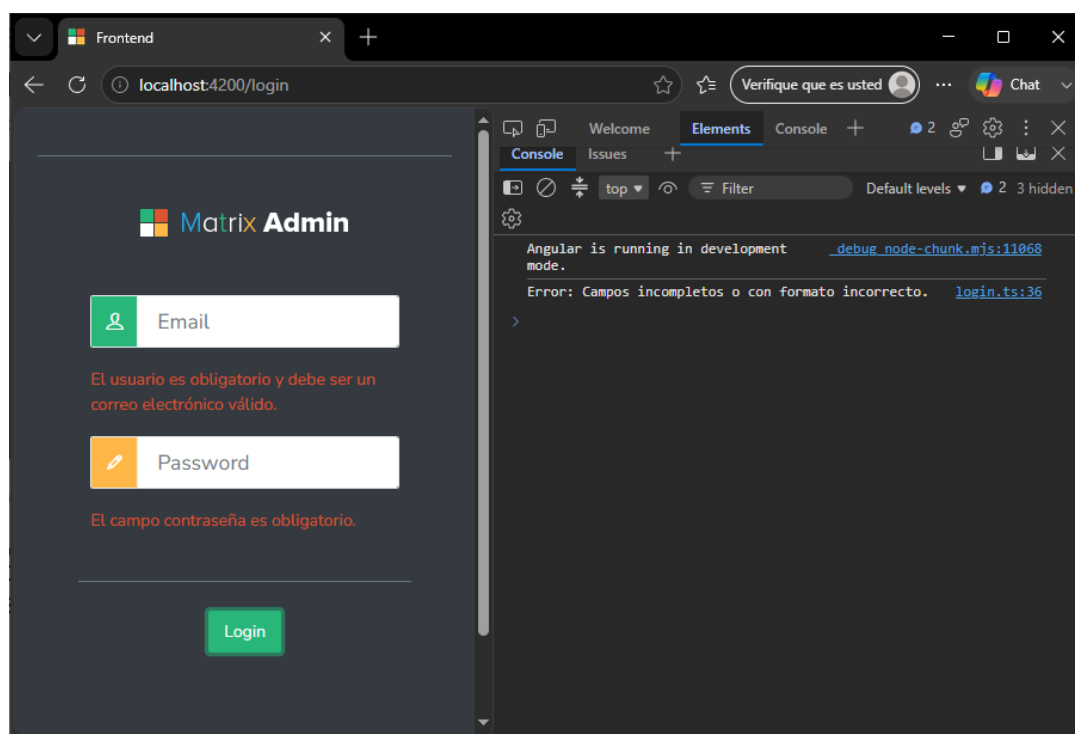


Figura 73. Resultado del intento de envío sin datos.

5.8.8. Alternativa para la evaluación del estado global

Angular ofrece múltiples vías para consultar el estado de un formulario reactivo. En el paso anterior utilizamos la propiedad `status`, que nos devuelve una cadena de texto (como 'VALID' o 'INVALID').

Sin embargo, existe una alternativa más directa y muy utilizada por los desarrolladores: la propiedad booleana `valid`. Esta propiedad devuelve directamente `true` si todas las reglas de validación del formulario se han cumplido, o `false` si existe algún error en cualquiera de los campos.

Podemos optimizar la condición dentro de nuestra función `login()` para utilizar esta propiedad booleana, evitando tener que comparar cadenas de texto. El código de la función `login()` en el archivo **src/app/auth/login/login.ts** se actualizaría de la siguiente manera:

```
login() {  
  // Registramos que el usuario ha intentado enviar el formulario  
  this.formSubmit = true;  
  
  if (this.loginForm.valid) {  
    console.log('Formulario correcto, listo para enviar al servidor.');    console.log(this.loginForm.value);  
  } else {  
    console.log('Error: Campos incompletos o con formato incorrecto.');  }  
}
```

Ambos enfoques son completamente funcionales y el framework los soporta a la perfección. El uso de `this.loginForm.valid` suele ser la opción preferida por convención, ya que las evaluaciones booleanas directas hacen que las estructuras condicionales (`if`) queden ligeramente más limpias y semánticas. Por este motivo nos decantamos por esta opción. Con esto ya estamos listos para hacer el login propiamente. En la siguiente sección lo vemos.

5.9. Login y servicio user

Para que nuestra aplicación interactúe con el servidor (por ejemplo, para enviar los datos de inicio de sesión o solicitar la lista de usuarios), es necesario realizar peticiones HTTP.

Sin embargo, escribir la lógica de estas peticiones de red directamente dentro de los componentes visuales (como `login.ts`) es una mala práctica. Si necesitamos consultar la información de un usuario desde tres pantallas distintas, acabaríamos duplicando el código de conexión en las tres.

Para evitar esto y mantener el código ordenado, Angular utiliza los Servicios. Un servicio es una clase especializada en gestionar la lógica de datos y la comunicación con el backend, encapsulándola de forma que cualquier componente de la aplicación pueda reutilizarla fácilmente.

5.9.1. Creación del servicio user

Para generar nuestro primer servicio, utilizamos la consola de comandos. Ejecutamos la siguiente instrucción en la terminal para crear una carpeta `services` que contendrá el servicio `users`:

ng generate service services/user

Esto generará el archivo **src/app/services/user.ts**. Lo abrimos para inyectar la herramienta `HttpClient` (que nos permitirá hacer peticiones de red) y crear el método `login` que recibirá los datos del formulario.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class UserService {

  // Inyectamos el cliente HTTP para futuras peticiones al backend
  constructor(private http: HttpClient) { }

  // Método que recibirá los datos del formulario
  login(formData: any) {
    console.log('Login invocado desde el servicio de usuario con los datos:');
    console.log(formData);
    // Próximamente: petición POST al backend
  }
}
```

5.9.2. Configuración global del cliente HTTP

Para que el servicio pueda utilizar HttpClient sin generar errores de inyección de dependencias o bloqueos cíclicos en la aplicación, es obligatorio habilitar esta herramienta a nivel global.

Abrimos el archivo de configuración principal de la aplicación (**src/app/app.config.ts**) y añadimos la función `provideHttpClient()` dentro de la lista de proveedores:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient() // Habilitamos las peticiones HTTP en toda la aplicación
  ]
};
```

5.9.3. Inyección del servicio en el componente login

Finalmente, debemos conectar nuestro componente visual (`login.ts`) con el nuevo servicio. Para ello, abrimos el archivo **src/app/auth/login/login.ts** e inyectamos el servicio en el constructor.

A continuación, modificamos la función `login()` para que, en lugar de imprimir los datos directamente, delegue la responsabilidad llamando al método del servicio y pasándole los valores del formulario.

El archivo `login.ts` debería quedar así:

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CommonModule } from '@angular/common';
```

```
import { UserService } from '../services/user';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule, CommonModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {

  public loginForm: FormGroup;
  public formSubmit: boolean = false;

  // Inyectamos el servicio junto al FormBuilder
  constructor(private fb: FormBuilder, private userService: UserService) {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  // Comprueba si el campo es válido o si aún no se ha intentado enviar
  isFieldValid(field: string): boolean {
    return this.loginForm.get(field)?.valid || !this.formSubmit;
  }

  login() {
    // Registramos que el usuario ha intentado enviar el formulario
    this.formSubmit = true;

    if (this.loginForm.valid) {
      // Delegamos la acción al servicio, enviando el objeto con email y password
      this.userService.login(this.loginForm.value);
    } else {
      console.log('Error: Campos incompletos o con formato incorrecto.');
```

5.9.4. Invocación de la API y manejo de respuestas

Una vez que tenemos nuestro servicio creado y el cliente HTTP habilitado globalmente en la configuración de la aplicación, el siguiente paso es utilizarlo para establecer la comunicación real con el backend.

La principal ventaja de utilizar HttpClient de Angular es que nos permite realizar peticiones de red (GET, POST, PUT, DELETE) de una manera muy sencilla, delegando la complejidad técnica y devolviendo la respuesta encapsulada en un flujo asíncrono llamado Observable.

Para conectar nuestro formulario de inicio de sesión con el backend, actualizaremos tanto el servicio como el controlador del componente.

5.9.5. Actualización del servicio (user.ts)

Abrimos el archivo **src/app/services/user.ts**. Vamos a modificar el método login para que deje de imprimir

los datos por consola y, en su lugar, realice una petición POST a la ruta securizada de nuestra API que construimos en Node.js.

El método retornará la petición HTTP directamente, permitiendo que el componente que lo invoque pueda escuchar la respuesta del servidor.

El método login de nuestro archivo debería de quedar así:

```
// Recibe los datos del formulario y realiza la petición POST al backend
login(formData: any) {
  return this.http.post('http://localhost:3000/api/login', formData);
}
```

5.9.6. Actualización de login.ts

Cuando realizamos una petición HTTP en Angular, esta no se ejecuta inmediatamente. Al devolver un Observable, Angular requiere que nos "suscribamos" a él para que la petición se lance y podamos capturar los resultados.

Abrimos el archivo **src/app/auth/login/login.ts** y actualizamos la función login(). En lugar de enviar los datos al vacío, nos suscribiremos (.subscribe()) al método del servicio. La suscripción nos obliga a definir dos caminos: next (qué hacer si el servidor responde correctamente con los tokens) y error (qué hacer si el servidor rechaza el acceso, por ejemplo, por credenciales incorrectas). Solo tenemos que sustituir el contenido del if(this.loginForm.valid) por este código:

```
// Delegamos la acción al servicio, enviando el objeto con email y password
this.userService.login(this.loginForm.value).subscribe({
  next: (response) => {
    // El backend ha validado las credenciales y devuelve el JWT
    console.log('Login exitoso. Respuesta del servidor:', response);
  },
  error: (err) => {
    // El backend rechaza el acceso o hay un problema de conexión
    console.warn('Error en la petición de login:', err);
  }
});
```

Con esta implementación, la aplicación en Angular ya es capaz de enviar las credenciales introducidas por el usuario directamente al puerto 3000 de nuestro entorno local. Si las credenciales son válidas, la consola del navegador mostrará el objeto JSON devuelto por nuestro backend, el cual contendrá los datos limpios del usuario, el token principal y el token de renovación.

Si hacemos login con los datos de un usuario conocido de la BD, podemos observar que efectivamente nos devuelve los datos esperados:

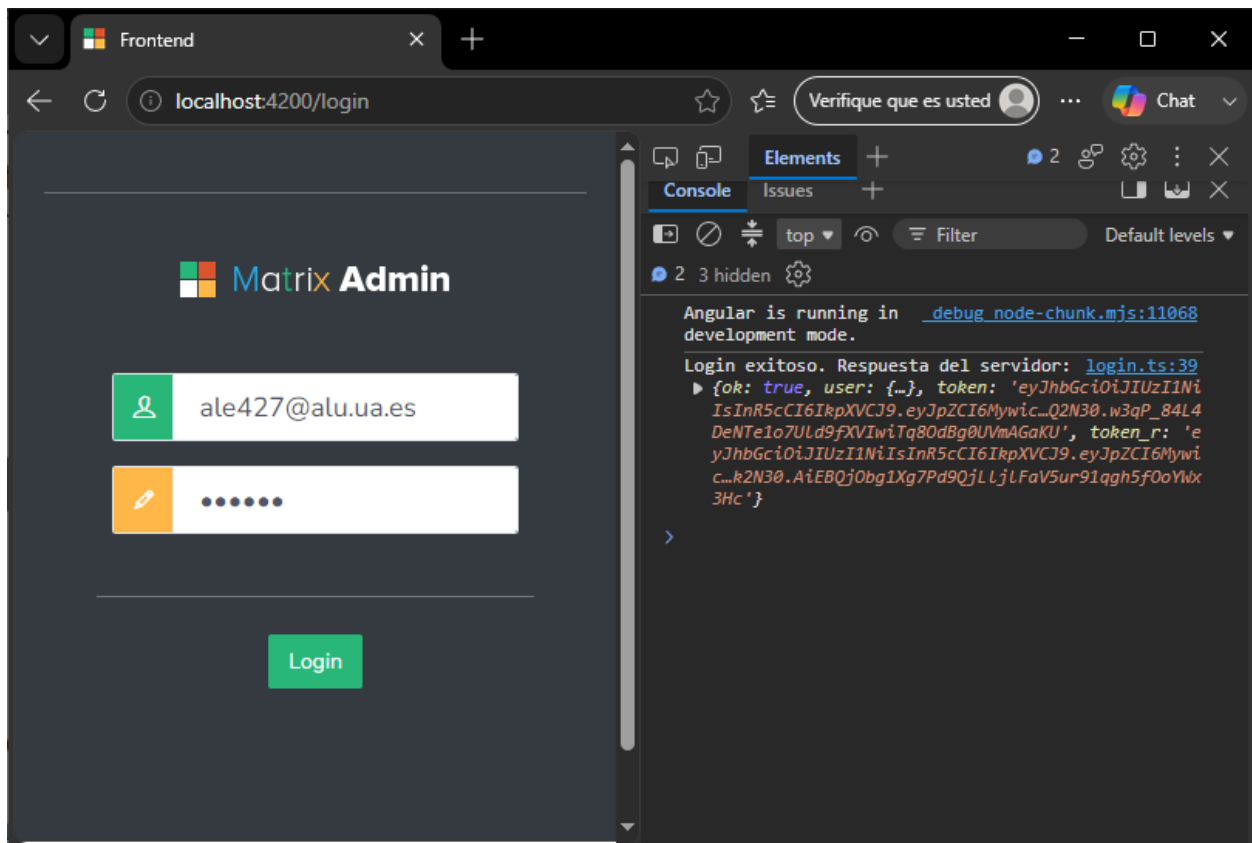


Figura 74. Resultado del login con el token generado.

Ahora ya somos capaces de llamar a la API y recibir la respuesta, en la siguiente sección vemos la lógica de negocio sobre qué hacer con esta respuesta.

5.10. Descarga Código 9

Todo lo desarrollado hasta ahora está disponible en el “Codigo9.zip” que puedes descargar desde aquí:

[Descargar Código 9](#)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo9.zip>

5.11. Interfaces

Hasta este momento, nuestro servicio user estaba recibiendo los datos del formulario bajo el tipo genérico any (formData: any). Aunque esto funciona, desaprovecha una de las mayores ventajas de TypeScript: el tipado estricto.

Para solucionar esto y evitar posibles errores de consistencia en los datos, utilizaremos las Interfaces. Una interfaz es una estructura que actúa como un "contrato", definiendo exactamente qué propiedades debe tener un objeto y de qué tipo debe ser cada una (string, boolean, number, etc.).

El uso de interfaces aporta múltiples beneficios al desarrollo:

- **Autocompletado contextual:** El entorno de desarrollo (IDE) reconocerá las propiedades del objeto y nos sugerirá los campos disponibles.
- **Detección temprana de errores:** Si intentamos enviar una propiedad que no existe o con un tipo de dato incorrecto, TypeScript mostrará un error antes siquiera de compilar la aplicación.

5.11.1. Creación de la interfaz LoginForm

Para mantener el proyecto organizado, crearemos un directorio específico para almacenar todas las interfaces de la aplicación.

Generaremos la interfaz con el siguiente comando:

ng generate interface interfaces/login-form

En este archivo definiremos la estructura exacta que esperamos recibir desde el formulario de inicio de sesión:

```
export interface LoginForm {  
  email: string;  
  password: string;  
}
```

Si en el futuro necesitáramos que alguno de estos campos no fuera obligatorio a nivel de estructura de datos, bastaría con añadir un símbolo de interrogación antes de los dos puntos (por ejemplo, email?: string;), convirtiéndolo en un campo opcional.

5.11.2. Aplicación de la interfaz en el servicio

Una vez definida la interfaz, el último paso es sustituir el tipo genérico any por nuestro nuevo tipo estructurado en el servicio correspondiente.

Abrimos el archivo **src/app/services/user.ts**, importamos la interfaz que acabamos de crear y actualizamos el tipado del parámetro formData en el método login:

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
// Importamos la interfaz  
import { LoginForm } from '../interfaces/login-form';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class UserService {  
  
  constructor(private http: HttpClient) { }  
  
  // Sustituimos 'any' por la interfaz 'LoginForm'  
  login(formData: LoginForm) {  
    return this.http.post('http://localhost:3000/api/login', formData);  
  }  
}
```

Al aplicar este cambio, nuestro servicio UserService ahora es estricto: solo aceptará procesar la petición HTTP si el objeto que se le envía desde el componente de Login cumple rigurosamente con la estructura definida en la interfaz LoginForm, garantizando la integridad de los datos en toda la aplicación.

5.12. Login - almacenar token, acceso o error

Una vez que el backend valida las credenciales y responde afirmativamente, nos devuelve un token JWT. Para que el usuario no tenga que iniciar sesión cada vez que cambie de página, la aplicación debe guardar este token en un lugar accesible. Para ello, los navegadores web proporcionan dos áreas de almacenamiento principal:

- **LocalStorage (almacenamiento local):** Es una porción de memoria persistente en el navegador. Almacena la información en formato de pares clave-valor (ej. "token": "eyJhbG..."). Esta zona es independiente para cada aplicación web (dominio), lo que evita conflictos de nombres con otras páginas. Su característica principal es que los datos persisten aunque se cierre el navegador o se reinicie el equipo, manteniéndose hasta que la aplicación o el usuario los borren explícitamente.
- **SessionStorage (almacenamiento de sesión):** Funciona de manera idéntica al LocalStorage, con la diferencia de que su ciclo de vida es temporal. La memoria se borra automáticamente en cuanto el usuario cierra la pestaña o el navegador por completo.

Es fundamental entender que ambas zonas de almacenamiento son "no confiables". Cualquier dato guardado en localStorage o sessionStorage es visible y manipulable por el usuario a través de las herramientas de desarrollo del navegador. Por este motivo, el backend nunca confía a ciegas en el token almacenado; siempre debe validar su firma criptográfica en cada petición.

5.12.1. Lógica de negocio del Login

Nuestra aplicación implementará la siguiente lógica de negocio al recibir la respuesta del servicio, si el login es correcto (Next): Capturaremos el token de la respuesta, lo guardaremos en el localStorage y utilizaremos el enrutador de Angular para redirigir al usuario a la página principal (/dashboard). Si el login falla (Error): Mostraremos un mensaje visual amigable para informar al usuario de que sus credenciales son incorrectas.

5.12.2. Librería para alertas visuales

Para mostrar el mensaje de error de una forma agradable visualmente, en lugar de utilizar las alertas nativas del navegador, instalaremos la librería SweetAlert2. Ejecutamos el siguiente comando en la terminal (dentro de la carpeta del frontend):

```
npm install sweetalert2
```

5.12.3. Implementación en el controlador (login.ts)

Abrimos el archivo **src/app/auth/login/login.ts**. Para poder realizar la redirección, debemos importar e inyectar la herramienta Router de Angular. Además, importaremos SweetAlert2 para gestionar el aviso de error. A continuación, actualizamos el método .subscribe() para aplicar la lógica de negocio descrita. El código de login.ts quedaría así finalmente:

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { UserService } from '../services/user';
```

```
import { Router } from '@angular/router';
import Swal from 'sweetalert2';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule, CommonModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {

  public loginForm: FormGroup;
  public formSubmit: boolean = false;

  // Inyectamos el servicio junto al FormBuilder y también el Router
  constructor(private fb: FormBuilder, private userService: UserService, private
router: Router) {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  // Comprueba si el campo es válido o si aún no se ha intentado enviar
  isFieldValid(field: string): boolean {
    return this.loginForm.get(field)?.valid || !this.formSubmit;
  }

  login() {
    // Registramos que el usuario ha intentado enviar el formulario
    this.formSubmit = true;

    if (this.loginForm.valid) {
      // Delegamos la acción al servicio, enviando el objeto con email y password
      this.userService.login(this.loginForm.value).subscribe({
        next: (response: any) => {
          // 1. Guardamos el token en la memoria persistente del navegador
          localStorage.setItem('token', response.token);

          // 2. Redirigimos al usuario a la zona privada de administración
          this.router.navigateByUrl('/dashboard');
        },
        error: (err) => {
          // Si el backend devuelve un error (ej. 400 Bad Request), mostramos la
          alerta
          Swal.fire({
            icon: 'error',
            title: 'Error',
            text: 'Invalid user or password',
            confirmButtonText: 'OK',
            confirmButtonColor: '#d33' // Color rojo para mantener consistencia visual
          });
        }
      });
    } else {
      console.log('Error: Campos incompletos o con formato incorrecto.');
```

Podemos observar, que tras aplicar estos cambios, si arrancamos el backend y el frontend y realizamos un intento de login con datos falsos o mal escritos, da un error 400 y sale la alerta correctamente tal que así:

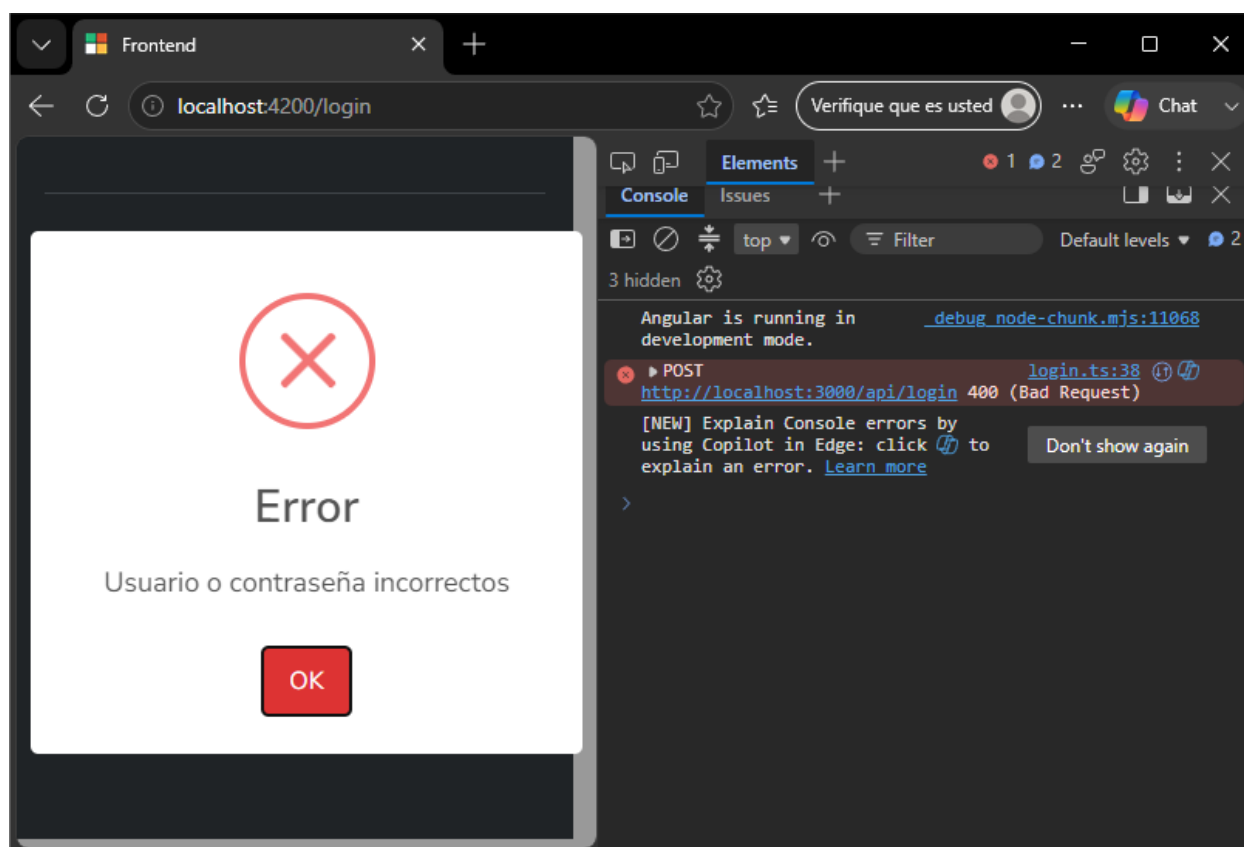


Figura 75. Resultado gráfico del error.

6. Securizar frontend

6.1. Backend llamada validar token

En este punto, la aplicación frontend es capaz de realizar el proceso de inicio de sesión y almacenar un token. Sin embargo, no existe todavía ninguna restricción lógica que impida a un usuario acceder directamente a rutas privadas (como /dashboard) escribiendo la URL en la barra de direcciones del navegador.

Para securizar estas rutas, es necesario garantizar que el usuario posee un token válido en su almacenamiento local. Un token se considera válido si cumple tres condiciones: ha sido firmado por nuestro servidor, el usuario al que pertenece sigue existiendo en la base de datos, y no ha expirado.

6.1.1. Mitigación de vulnerabilidad en tokens huérfanos

El middleware de seguridad que utilizamos para validar los tokens JWT en nuestro backend (validateJWT) se encarga de interceptar la petición, leer la cabecera correspondiente y confirmar la integridad matemática y la vigencia del token. Sin embargo, si nos detenemos en esa única capa de seguridad, existe un escenario de vulnerabilidad: si un usuario es eliminado del sistema, el token almacenado en su navegador seguiría siendo válido hasta cumplir su tiempo de vida original. Si el sistema renovara el token basándose únicamente en la validación criptográfica, el usuario eliminado mantendría acceso indefinido.

Para evitar la generación de tokens huérfanos y asegurar el sistema, es imperativo añadir una comprobación contra la base de datos justo antes de emitir la renovación.

6.1.2. Implementación del controlador de renovación

En el controlador de autenticación (**controllers/auth.js**), se implementa la función `renewToken`. Esta función extrae el identificador del usuario proporcionado por el middleware, comprueba su existencia real en la base de datos mediante la consulta al modelo y, solo en caso afirmativo, procede a generar y almacenar el nuevo par de tokens. Si el usuario no existe, la petición se bloquea de inmediato con un error de autorización. Así es como debería de quedar el archivo `controllers/auth.js`:

```
// Función para renovar el token de acceso cuando se requiere
const renewToken = async (req, res = response) => {
  const id = req.id;

  try {
    // 1. Comprobar que el usuario sigue existiendo en la base de datos ANTES de
    renovar
    const user = await User.getUserById(id);

    if (!user) {
      return res.status(401).json({
        ok: false,
        msg: 'Token inválido: usuario no existe en la base de datos',
      });
    }

    // 2. Generamos un nuevo par de tokens
    const { token, token_r } = await generateJWT(id, req.role);

    // 3. Actualizamos la base de datos con el nuevo token activo
    await User.updateUserToken(id, token);

    res.json({
      ok: true,
      user, // Se devuelve la información verificada del usuario
      token,
      token_r
    });
  } catch (error) {
    console.log(error);
    res.status(500).json({
      ok: false,
      msg: 'Error al renovar el token'
    });
  }
};
```

Mediante este flujo, el endpoint expuesto en `GET /api/login/renew` aplica una estrategia segura de renovación de tokens. Esto permite extender el tiempo de sesión del usuario de forma dinámica con cada petición a la API, al mismo tiempo que garantiza el bloqueo inmediato de cuentas que hayan sido eliminadas.

6.1.3. Protección de rutas y renovación del token en el frontend

Una vez que el backend está preparado para validar y renovar los tokens correctamente, el siguiente paso lógico es adaptar nuestra aplicación para que envíe el token de sesión cada vez que el usuario navegue por zonas privadas.

En este apartado, crearemos un método en nuestro servicio de usuarios que se encargará de realizar una petición a nuestro endpoint de renovación.

6.1.4. Implementación en el servicio (UserService)

Abrimos el archivo **src/app/services/user.ts** y añadiremos un nuevo método llamado `renewToken`. Este método recuperará el token almacenado en el `localStorage` y lo enviará al servidor a través de la cabecera `x-token`.

Tenemos que importar una nueva clase de Angular llamada `HttpHeaders`, que nos permite configurar las cabeceras de la solicitud HTTP que vamos a mandar. El archivo `user.ts` debería de quedar así:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { LoginForm } from '../interfaces/login-form';

@Injectable({
  providedIn: 'root'
})
export class UserService {

  // Definimos la URL base para no repetirla
  private baseUrl = 'http://localhost:3000/api';

  // Inyectamos el cliente HTTP para futuras peticiones al backend
  constructor(private http: HttpClient) { }

  // Recibe los datos del formulario y realiza la petición POST al backend
  login(formData: LoginForm) {
    return this.http.post(`${this.baseUrl}/login`, formData);
  }

  // Nuevo método para validar y renovar el token
  renewToken() {
    // 1. Recuperamos el token del almacenamiento local.
    // Si no existe, enviamos un string vacío.
    const token = localStorage.getItem('token') || '';

    // 2. Configuramos las cabeceras de la petición HTTP,
    // añadiendo el token bajo la clave 'x-token'.
    const headers = new HttpHeaders().set('x-token', token);

    // 3. Realizamos la petición GET al endpoint de renovación,
    // pasando las cabeceras configuradas.
    return this.http.get(`${this.baseUrl}/login/renew`, { headers });
  }
}
```

Con este método ya disponible en nuestro servicio, el siguiente paso será integrarlo en el sistema de enrutamiento de Angular para proteger el acceso a los distintos módulos de la aplicación.

6.1.5. Automatización de la renovación del token

Ahora mismo el token del usuario se actualiza cuando hace login, pero nosotros queremos que haya una renovación periódica utilizando el token de renovación, y esta la tenemos que implementar en nuestro frontend, para que llame cada x minutos a la ruta de renew de nuestro backend.

Para programar esta automatización en Angular sin bloquear la interfaz de usuario, utilizaremos la librería reactiva RxJS, concretamente la función interval.

6.1.6. Actualización del guardado de tokens en login

En primer lugar, debemos asegurarnos de que el proceso de inicio de sesión guarda ambos tokens en la memoria del navegador y, acto seguido, arranca el temporizador. En el archivo **src/app/auth/login/login.ts**, dentro de la suscripción HTTP exitosa, actualizamos el código de la siguiente manera:

```
next: (response: any) => {
  // Guardamos ambos tokens en la memoria persistente del navegador
  localStorage.setItem('token', response.token);
  localStorage.setItem('token_r', response.token_r);

  // Iniciamos el temporizador de renovación en segundo plano con una función
  // que ahora crearemos en el servicio userService
  this.userService.startTokenRenewalTimer();

  // Redirigimos al usuario a la zona privada
  this.router.navigateByUrl('/dashboard');
},
```

6.1.7. Implementación del temporizador en userService

A continuación, abrimos nuestro servicio **src/app/services/user.ts**. Modificaremos el método `renewToken` para que lea y envíe específicamente el token de larga duración (`token_r`). Además, crearemos dos nuevos métodos: uno para iniciar el intervalo de 5 minutos (300,000 milisegundos) y otro para detenerlo (necesario para cuando el usuario cierre sesión). El archivo `user.ts` debería quedar así:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { LoginForm } from '../interfaces/login-form';
import { interval, Subscription } from 'rxjs';

@Injectable({
  providedIn: 'root'
})

export class UserService {

  // Definimos la URL base para no repetirla
  private baseUrl = 'http://localhost:3000/api';
  // Variable para almacenar y controlar el ciclo del temporizador
```

```
private renewalSubscription: Subscription | null = null;

// Inyectamos el cliente HTTP para futuras peticiones al backend
constructor(private http: HttpClient) { }

// Recibe los datos del formulario y realiza la petición POST al backend
login(formData: LoginForm) {
  return this.http.post(`${this.baseUrl}/login`, formData);
}

// Nuevo método para validar y renovar el token
renewToken() {
  const tokenR = localStorage.getItem('token_r') || '';
  const headers = new HttpHeaders().set('x-token', tokenR);

  return this.http.get(`${this.baseUrl}/login/renew`, { headers });
}

// Método para iniciar el temporizador de renovación del token
// Inicia la ejecución de la renovación cada 5 minutos (300,000 ms)
startTokenRenewalTimer() {
  // Si ya existe un temporizador en marcha, lo detenemos para evitar procesos
  // duplicados
  this.stopTokenRenewalTimer();

  this.renewalSubscription = interval(300000).subscribe(() => {
    this.renewToken().subscribe({
      next: (response: any) => {
        // Si la respuesta es exitosa, sobrescribimos los tokens antiguos por los
        // nuevos
        localStorage.setItem('token', response.token);
        localStorage.setItem('token_r', response.token_r);
        console.log('Los tokens han sido renovados exitosamente.');
      },
      error: (err) => {
        // Si el token_r ha caducado o el usuario fue eliminado, se detiene el ciclo
        console.error('Error renovando tokens. Es posible que la sesión haya expirado.', err);
        this.stopTokenRenewalTimer();
      }
    });
  });
}

// Detiene y destruye el temporizador activo
stopTokenRenewalTimer() {
  if (this.renewalSubscription) {
    this.renewalSubscription.unsubscribe();
    this.renewalSubscription = null;
  }
}
```

6.1.8. Persistencia del temporizador ante recargas

Existe un escenario que necesitamos cubrir: si el usuario pulsa F5 y recarga la página por completo, la memoria RAM de la pestaña se limpia y el temporizador en ejecución de RxJS se destruye.

Para garantizar que el ciclo de renovación de 5 minutos sobreviva y se reinicie siempre que exista una sesión activa en el navegador, debemos invocar el arranque en el componente raíz de la aplicación. Para ello, abrimos **src/app/app.ts** y sustituimos su contenido por este código:

```
import { Component, OnInit, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Footer } from '../commons/footer/footer';
import { UserService } from '../services/user';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Footer],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App implements OnInit {
  protected readonly title = signal('frontend');

  // Inyectamos el servicio en el constructor
  constructor(private userService: UserService) {}

  // Implementamos el método del ciclo de vida OnInit
  ngOnInit() {
    // Si la memoria contiene un token de renovación, reactivamos el temporizador
    // al cargar la aplicación
    if (localStorage.getItem('token_r')) {
      this.userService.startTokenRenewalTimer();
    }
  }
}
```

6.1.9. Cambio en formatUser (backend) antes de la comprobación

Existe un detalle que debemos solucionar en la estructura de datos que devuelve el servidor: al realizar la respuesta de autenticación, el objeto de usuario recuperado de la base de datos incluía el token activo, lo que provocaba que este se enviara duplicado (dentro de la propiedad user y de forma independiente en la raíz de la respuesta).

Para evitar esta redundancia y mantener una respuesta limpia, debemos modificar la función de formato en el modelo de datos (**models/users.js**). De esta forma, nos aseguramos de que el objeto de usuario devuelva únicamente la información de perfil necesaria, descartando tanto la contraseña como el token antiguo.

Para aplicar este cambio, abrimos el archivo **models/users.js** y dejamos la función de la siguiente manera:

```
// Función personalizada para limpiar datos sensibles del usuario
const formatUser = (user) => {
  if (!user) return null;

  // Extraemos password y token para descartarlos y nos quedamos con el resto
  const { password, token, ...cleanUser } = user;
  return cleanUser;
}
```

};

Con esta modificación, el backend envía una respuesta limpia y normalizada, centralizando la gestión de los nuevos tokens únicamente en la raíz del objeto JSON.

6.1.10. Prueba de que la renovación automática funciona

Para poder comprobar que el sistema de renovación del token de usuario que hemos creado funciona correctamente, tendremos que cambiar el intervalo de tiempo que se tarda en enviar el token para generar uno nuevo, porque sino perdemos mucho tiempo. Para ello iremos a nuestra función `startTokenRenewalTimer()` en nuestro archivo `src/app/services/user.ts` y cambiaremos el número 300000 de esta línea por un 10000:

```
this.renewalSubscription = interval(300000).subscribe(() => {
```

Este cambio permitirá que se produzca la renovación cada 10 segundos, lo que nos permitirá verlas en directo con las herramientas de desarrollador del navegador. Para ver la solicitud y poder ver la respuesta recibida, deberemos de acceder a la pestaña de "Network", tal y como podemos ver en esta captura:

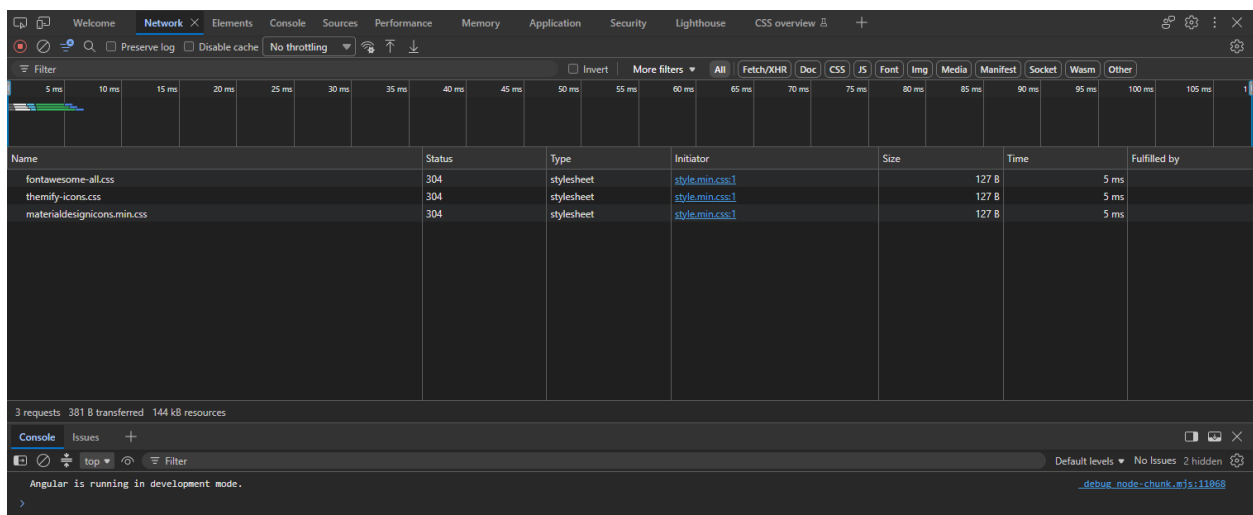


Figura 76. Pestaña network.

Ahora, si esperamos 10 segundos, podremos observar que aparece una solicitud de nombre "renew" en la pestaña de network. Si pulsamos en ella nos debería de salir lo siguiente:

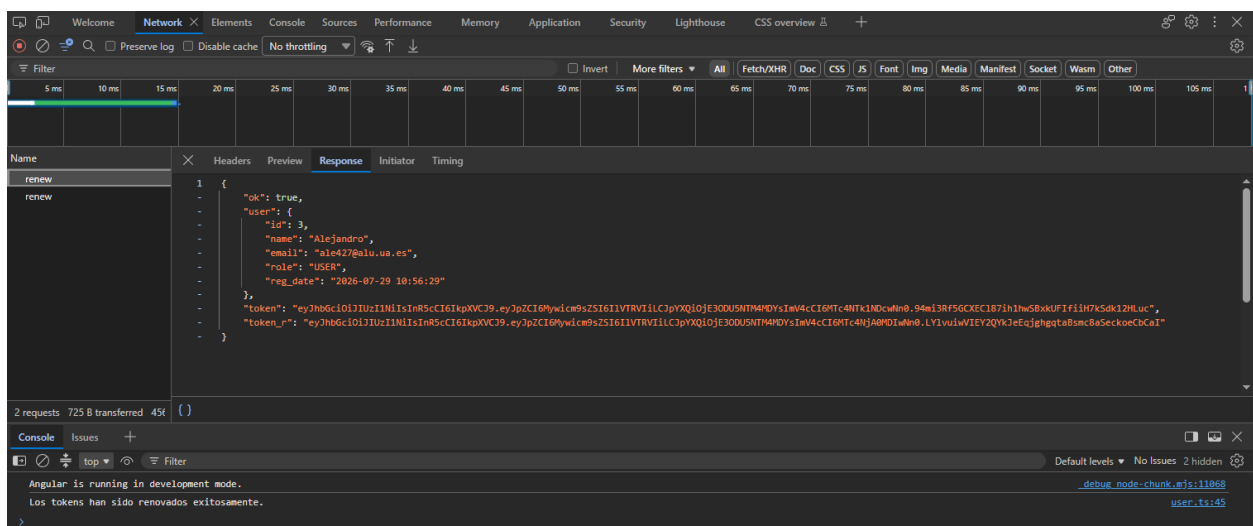


Figura 77. Pestaña newtork tras 10 segundos.

Gracias al mensaje en consola y al token devuelto en la respuesta, podemos concluir que el sistema funciona correctamente, ya que ha renovado el token automáticamente. Ahora deberíamos de devolver la función a su estado original, volviendo a cambiar el 10000 por un 300000.

6.2. Zona dashboard - creando guard

Para securizar la aplicación cliente, utilizaremos guardas de enrutamiento (Guards). Se trata de protecciones integradas en el sistema de enrutamiento que permiten controlar bajo qué condiciones se autoriza la renderización del contenido de una ruta específica.

En las versiones modernas de Angular, estas guardas son funciones que se ejecutan antes de cargar el componente principal de una ruta. Si la validación interna es exitosa (devuelve un valor booleano true), se permite el acceso. Si falla (devuelve false), se bloquea la navegación y es posible forzar una redirección. Una ventaja clave de esta arquitectura es que, al aplicar una guarda sobre una ruta padre, la protección se hereda automáticamente a todas sus rutas derivadas, lo que permite centralizar y reutilizar la lógica de control de acceso en un único archivo.

6.2.1. Creación de la guarda de autenticación

Para generar este nuevo elemento, utilizamos la interfaz de línea de comandos de Angular (CLI). Indicamos que se cree dentro de una carpeta dedicada y utilizamos el parámetro para omitir los archivos de prueba (--skip-tests):

```
ng generate guard guards/auth --skip-tests
```

Durante el proceso de creación, el asistente interactivo preguntará qué interfaces deseamos implementar. Seleccionaremos únicamente CanActivate, ya que es la encargada específica de evaluar si una ruta puede o no ser activada.

El archivo generado (**src/app/guards/auth.guard.ts**) contendrá la estructura básica de la guarda. En los siguientes pasos, programaremos su método canActivate para que consulte con el servicio si el usuario dispone de un token válido.

6.2.2. Aplicación de la guarda en el router

Una vez creada, el siguiente paso es vincular esta guarda a las rutas que deseamos proteger. En nuestro proyecto, las rutas privadas se encuentran agrupadas bajo el archivo de enrutamiento de las páginas internas. Abrimos el archivo app.routes.ts. Primero, importamos la guarda recién creada y, a continuación, la inyectamos en la definición de la ruta dashboard utilizando la propiedad canActivate:

```
import { Routes } from '@angular/router';
import { AuthLayout } from '../layouts/auth-layout/auth-layout';
import { AdminLayout } from '../layouts/admin-layout/admin-layout';

// Importación de las subrutas de autenticación y administración
import { authRoutes } from '../auth/auth.routes';
import { adminRoutes } from '../pages/admin.routes';
```

```
import { authGuard } from './guards/auth-guard';

export const routes: Routes = [
  {
    path: 'login',
    component: AuthLayout,
    children: authRoutes
  },
  {
    path: 'dashboard',
    component: AdminLayout,
    canActivate: [authGuard], // Aplicamos la protección a la ruta padre
    children: adminRoutes
  },
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: '**', redirectTo: 'login' } // Atrapa cualquier ruta global no definida
  // y redirige al login
];
```

Con esta configuración, cualquier intento de acceder a /dashboard o a sus rutas derivadas (como /dashboard/users) provocará que el sistema intercepte la navegación y ejecute de forma automática la clase authGuard.

6.2.3. Lógica de autorización en el guard

Una vez vinculada la guarda a las rutas privadas, es necesario programar su lógica interna. El objetivo de la función authGuard es interceptar el intento de navegación, contactar con el servidor para comprobar si el usuario dispone de una sesión válida y, en base a la respuesta, autorizar el acceso o rechazarlo.

Para realizar esta comprobación, se reutiliza el método renewToken() implementado previamente en el servicio de usuarios (UserService). Este método se encarga de enviar el token actual al backend, si es válido, el servidor responde correctamente, confirmando que la sesión sigue activa y que el usuario no ha sido eliminado de la base de datos.

Para adaptar esta respuesta del servidor y convertirla en una decisión de acceso (true o false), utilizamos las funciones de transformación de RxJS (map y catchError). Dado que estamos utilizando una función en lugar de una clase, extraemos nuestras dependencias mediante la instrucción inject(). Abrimos el archivo **src/app/guards/auth.guard.ts** y lo configuramos de la siguiente manera:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { Observable, of } from 'rxjs';
import { map, catchError } from 'rxjs/operators';
import { UserService } from '../services/user';

export const authGuard: CanActivateFn = (route, state): Observable<boolean> => {
  // Extraemos los servicios necesarios mediante inyección de dependencias funcional
  const userService = inject(UserService);
  const router = inject(Router);

  // Llamamos al método del servicio que comprueba el token contra el backend
  return userService.renewToken().pipe(

    // Si el servidor responde correctamente, la sesión es válida: permitimos el acceso
```

```

    map(() => true),

    // Si ocurre un error (token caducado, modificado o usuario eliminado en BD):
    catchError(() => {
      // Detenemos la navegación y redirigimos al usuario a la pantalla de login
      router.navigateByUrl('/login');

      // Retornamos 'false' para bloquear definitivamente la activación de la ruta
      // privada
      return of(false);
    })
  );
};

```

La seguridad del frontend queda completamente sellada. Cualquier usuario que intente acceder a la URL /dashboard sin haber iniciado sesión previamente recibirá un error del servidor, la guarda interceptará dicho error devolviendo false, y el sistema lo expulsará inmediatamente a la pantalla de inicio de sesión.

6.3. Zona auth - añadir guard

Siguiendo el mismo principio de seguridad aplicado a la zona privada (/dashboard), resulta indispensable gestionar el comportamiento de la navegación en la zona pública de la aplicación, la cual en este proyecto se compone únicamente de la ruta de acceso (/login).

La lógica del flujo de usuario establece que la pantalla de inicio de sesión solo debe ser accesible cuando el visitante no cuenta con una sesión activa en el sistema. Si un usuario ya validado, que dispone de un token vigente almacenado en su navegador, intenta navegar manualmente a la ruta /login, la aplicación debe interceptar esta acción y redirigirlo de manera automática hacia el panel principal (/dashboard). Por consiguiente, la interfaz de inicio de sesión quedará reservada de forma exclusiva para los usuarios anónimos o para aquellos que hayan efectuado un cierre de sesión explícito.

6.3.1. Control directo desde el componente (login.ts)

Dado que en nuestra arquitectura solo disponemos de una única ruta de este tipo, la aproximación más directa y óptima es gestionar esta validación directamente en el ciclo de vida del propio componente visual encargado del inicio de sesión (login.ts).

Para llevar a cabo este control, aprovechamos el método ngOnInit (que se ejecuta en los primeros instantes de inicialización del componente). Desde ahí, invocamos al método renewToken() de nuestro servicio de usuarios. Si el servidor responde correctamente (confirmando que el token guardado sigue siendo válido), forzamos una redirección inmediata utilizando el enrutador de Angular, impidiendo que el formulario de login llegue a estar operativo para ese usuario.

```

import { Component, OnInit } from '@angular/core'; // 1. Añadimos OnInit
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { UserService } from '../services/user';
import { Router } from '@angular/router';
import Swal from 'sweetalert2';

@Component({
  selector: 'app-login',

```

```
imports: [ReactiveFormsModule, CommonModule],
templateUrl: './login.html',
styleUrl: './login.css'
}))
export class Login implements OnInit { // 2. Implementamos OnInit

  public loginForm: FormGroup;
  public formSubmit: boolean = false;

  // Inyectamos el servicio junto al FormBuilder y también el Router
  constructor(private fb: FormBuilder, private userService: UserService, private
router: Router) {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  // 3. Añadimos el ciclo de vida inicial para comprobar la sesión
  ngOnInit(): void {
    this.userService.renewToken().subscribe({
      next: () => {
        // Si el token es válido, evitamos que vea el login y lo enviamos al
dashboard
        this.router.navigateByUrl('/dashboard');
      },
      error: () => {
        // Si no hay token o caducó, se ignora y el componente renderiza el formula-
rio
      }
    });
  }

  // Comprueba si el campo es válido o si aún no se ha intentado enviar
  isFieldValid(field: string): boolean {
    return this.loginForm.get(field)?.valid || !this.formSubmit;
  }

  login() {
    // Registramos que el usuario ha intentado enviar el formulario
    this.formSubmit = true;

    if (this.loginForm.valid) {
      // Delegamos la acción al servicio, enviando el objeto con email y password
      this.userService.login(this.loginForm.value).subscribe({
        next: (response: any) => {
          // Guardamos ambos tokens en la memoria persistente del navegador
          localStorage.setItem('token', response.token);
          localStorage.setItem('token_r', response.token_r);

          // Iniciamos el temporizador de renovación en segundo plano con una función
que ahora crearemos en el servicio userService
          this.userService.startTokenRenewalTimer();

          // Redirigimos al usuario a la zona privada
          this.router.navigateByUrl('/dashboard');
```

```
    },
    error: (err) => {
      // Si el backend devuelve un error (ej. 400 Bad Request), mostramos la
      alerta
      Swal.fire({
        icon: 'error',
        title: 'Error',
        text: 'Usuario o contraseña incorrectos',
        confirmButtonText: 'OK',
        confirmButtonColor: '#d33' // Color rojo para mantener consistencia visual
      });
    }
  });
} else {
  console.log('Error: Campos incompletos o con formato incorrecto.');
```

De este modo, se consolida el "control inverso": mientras el Guard principal protege el panel de control de usuarios no identificados, esta lógica protege la pantalla de inicio de sesión de los usuarios que ya están autenticados, evitando redundancias, dobles inicios de sesión y mejorando significativamente la coherencia de la experiencia de usuario.

6.3.2. Control mediante el guard

Aunque la validación de la sesión desde el ciclo de vida del componente (ngOnInit) resulta funcional cuando existe una única ruta pública, esta estrategia presenta limitaciones arquitectónicas. Supedita las decisiones de navegación al proceso de renderizado de la interfaz, lo que puede provocar parpadeos visuales y mezclar responsabilidades.

Para seguir los estándares de desarrollo de Angular, aislar la lógica y evitar la duplicidad de código, es más aconsejable implementar este "control inverso" mediante un Guard dedicado. Este nuevo elemento, que actuará de forma opuesta al authGuard, se encargará de proteger las rutas públicas (como el inicio de sesión) interceptando a los usuarios que ya disponen de un token válido y redirigiéndolos al panel de control antes de que el componente llegue a instanciarse.

Generamos un nuevo archivo lógico en el sistema de guardas utilizando la consola de comandos. Lo denominaremos no-auth.guard:

```
ng generate guard guards/no-auth --skip-tests
```

A continuación, programamos la función utilizando el tipo CanActivateFn. La lógica de RxJS se invierte respecto a la guarda principal: si el servicio confirma que el token es válido, se bloquea el acceso a la ruta (false) y se fuerza la redirección; si falla (porque no hay sesión), se permite cargar el inicio de sesión (true).

6.4. Optimización del código

Como parte del proceso de mejora de la arquitectura, observamos que el almacenamiento de los tokens de autenticación lo estábamos realizando directamente desde el componente de inicio de sesión (login.ts). Aunque esta implementación era funcional, hacía que el componente asumiera una responsabilidad que realmente no le correspondía, ya que además de gestionar la interfaz de usuario también se encargaba de acceder al localStorage del navegador.

Para evitar esta situación, debemos trasladar toda la lógica relacionada con el almacenamiento de credenciales al servicio de usuarios (UserService). De esta forma, será el propio servicio el encargado de guardar y recuperar los tokens, mientras que los componentes únicamente tendrán que invocar los métodos necesarios sin preocuparse de cómo se realiza dicha operación.

Para conseguirlo, incorporamos varios métodos dentro del servicio que encapsulan el acceso al localStorage. Uno de ellos se encargará de almacenar los tokens tras un inicio de sesión satisfactorio, mientras que otros permitirán recuperarlos cuando sean necesarios para realizar peticiones autenticadas. Gracias a ello, cualquier modificación futura en la forma de almacenar las credenciales solo deberá realizarse UserService del archivo user.ts. Al principio de la definición de la clase de UserService deberemos poner los siguientes métodos:

```
export class UserService {  
  
  // ... declaración de baseUrl, renewalSubscription y constructor  
  
  // Método para centralizar el guardado de los tokens  
  public saveTokens(token: string, tokenR: string): void {  
    localStorage.setItem('token', token);  
    localStorage.setItem('token_r', tokenR);  
  }  
  
  // Métodos para la lectura centralizada de los tokens  
  public getToken(): string {  
    return localStorage.getItem('token') || '';  
  }  
  
  public getRefreshToken(): string {  
    return localStorage.getItem('token_r') || '';  
  }  
  
  // ... resto de la lógica del servicio (login, renewToken, etc.)  
}
```

Una vez disponemos de estos métodos, debemos modificar el componente de inicio de sesión para que deje de interactuar directamente con el localStorage. Ahora, cuando el servidor devuelve una autenticación correcta, el componente simplemente delega el almacenamiento de los tokens al servicio mediante el método saveTokens(). Así debería de quedar el bloque de **next** en la función login():

```
next: (response: any) => {  
  // Invocamos la función saveTokens de nuestro UserService para que esta  
  gestione su almacenamiento  
  this.userService.saveTokens(response.token, response.token_r);  
  
  // Iniciamos el temporizador de renovación en segundo plano con una función  
  que ahora crearemos en el servicio userService  
  this.userService.startTokenRenewalTimer();  
  
  // Redirigimos al usuario a la zona privada  
  this.router.navigateByUrl('/dashboard');  
},
```

Con este cambio conseguimos que cada parte de la aplicación tenga una única responsabilidad. El componente de inicio de sesión queda centrado exclusivamente en gestionar la interacción con el usuario y el flujo de autenticación, mientras que el servicio de usuarios pasa a ser el único responsable del acceso al almacenamiento local. Además de mejorar la organización del código, esta estructura facilitará la implementación de

funcionalidades futuras, como el cierre de sesión o posibles cambios en el mecanismo de persistencia de las credenciales.

6.5. Logout

Una vez implementados los mecanismos de autenticación y renovación automática de credenciales, únicamente queda proporcionar al usuario un mecanismo para cerrar su sesión dentro de la aplicación. Aunque el cierre de sesión se realiza desde un único punto de la interfaz (la opción Logout del componente Navbar), resulta más apropiado que toda la lógica asociada a este proceso resida en el UserService, del mismo modo que ocurre con el inicio de sesión.

Centralizar esta funcionalidad en el servicio aporta una mayor flexibilidad a la arquitectura, ya que cualquier componente de la aplicación podrá forzar el cierre de sesión invocando el mismo método. Esto resulta especialmente útil en situaciones como la expiración definitiva del token, errores de autenticación o cualquier otra circunstancia en la que sea necesario invalidar la sesión del usuario sin depender de la interacción con la interfaz.

Para implementar esta funcionalidad, incorporamos un nuevo método denominado `logout()` dentro de UserService en el archivo `user.ts`. Su funcionamiento consta de dos fases: en primer lugar, realiza una petición al backend para eliminar el token almacenado en la base de datos y garantizar que solo haya una sesión abierta. A continuación, ya sea de forma exitosa o ante un fallo de red, invoca un método auxiliar que elimina del almacenamiento local el token principal y el de renovación, detiene el contador automático y redirige al usuario hacia la pantalla de inicio de sesión, impidiendo que pueda seguir accediendo a las zonas protegidas.

```
logout() {  
  // Obtenemos el token de la memoria  
  const token = this.getToken();  
  
  // Lo preparamos en las cabeceras de la petición  
  const headers = new HttpHeaders().set('x-token', token);  
  
  // Hacemos la petición enviando las cabeceras  
  this.http.get(`${this.baseUrl}/login/logout`, { headers }).subscribe({  
    next: () => this.clearSession(),  
    error: () => this.clearSession() // Si falla la red, también limpiamos local-  
mente  
  });  
}  
  
// Método auxiliar para no duplicar código  
private clearSession() {  
  localStorage.removeItem('token');  
  localStorage.removeItem('token_r');  
  this.stopTokenRenewalTimer();  
  this.router.navigateByUrl('/login');  
}
```

Como este proceso requiere realizar una navegación entre rutas desde el propio servicio, es necesario inyectar previamente el Router de Angular en el constructor del UserService, además de importar la clase correspondiente.

```
import { Router } from '@angular/router';
```

```
constructor(  
  private http: HttpClient,  
  private router: Router  
) {}
```

Una vez disponemos de este método, el componente Navbar únicamente tendrá que invocarlo cuando el usuario pulse la opción de cerrar sesión. De esta forma, el componente permanece centrado en la gestión de la interfaz, mientras que toda la lógica relacionada con la finalización de la sesión continúa agrupada dentro del servicio de usuarios, manteniendo una arquitectura coherente con el resto del sistema.

6.5.1. Ruta de logout en la BD

Como podemos deducir, la ruta logout nos la acabamos de inventar y no la teníamos definida en el backend, por lo que si lo usáramos ahora mismo el logout, funcionaría porque nos cerraría la sesión, pero lo haría porque la respuesta de la solicitud get fallaría y daría error. Vamos a ver como implementamos esta ruta nueva a nuestro backend.

Lo primero que tenemos que hacer es acceder a nuestro archivo **/routes/auth.js**, e añadir la función logout que vamos a crear en el siguiente paso a los import de **/controllers/auth** de la siguiente forma:

```
const { login, renewToken, logout } = require('../controllers/auth');
```

En el mismo archivo añadiremos una nueva solicitud get justo debajo de la solicitud get a la ruta /renew, que se verá así:

```
// Añadimos la nueva ruta para el cierre de sesión, protegida por el JWT  
router.get('/logout', validateJWT, logout);
```

Ahora vamos a proceder a crear la función logout a nuestro **/controllers/auth.js**. Esta función recibirá la petición, extraerá el identificador del usuario (que el middleware validateJWT se encarga de añadir a la petición tras verificar el token) y actualizará la base de datos para borrar el token almacenado. Al borrar el token de la base de datos, garantiza que la sesión anterior quede invalidada de forma permanente.

Añadiremos el siguiente código al archivo **/controllers/auth.js**:

```
const logout = async (req, res) => {  
  // El middleware validateJWT debe haber guardado el identificador del usuario  
  en req.id  
  const id = req.id;  
  
  try {  
    // Buscamos al usuario en la base de datos  
    const user = await User.getUserById(id);  
  
    if (!user) {  
      return res.status(404).json({  
        ok: false,  
        msg: 'Usuario no encontrado'  
      });  
    }  
  
    // Actualizamos el usuario con una cadena vacía para que no tenga ningún to-  
    ken como activo en la BD  
    await User.updateUserToken(id, '');  
  
    res.json({  
      ok: true,
```

```

        msg: 'Sesión cerrada correctamente'
    });

    } catch (error) {
        console.log(error);
        res.status(500).json({
            ok: false,
            msg: 'Hable con el administrador'
        });
    }
};

```

6.5.2. Implementar el logout a nuestra interfaz

Por último, únicamente queda enlazar esta funcionalidad con la interfaz de usuario. Para ello, modificamos la opción Log out del componente navbar.html para que, al hacer clic sobre ella, invoque el método logout() definido previamente en el componente. Este método actuará como intermediario, delegando el proceso completo en el UserService, que será quien elimine las credenciales almacenadas y redirija al usuario a la pantalla de inicio de sesión.

Lo que haremos será añadir la característica (click)="logout()" a nuestro elemento del log out tal que así:

```

<a class="dropdown-item" href="javascript:void(0)" (click)="logout()"><i
class="fa fa-power-off m-r-5 m-l-5"></i> Log out</a>

```

Veremos que nos salta un error que dice que esta función no existe en navbar. Para que esta llamada funcione, el componente Navbar debe disponer de un método logout() que simplemente delegue la operación en el servicio de usuarios. El archivo navbar.ts debería de quedar tal que así:

```

import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
import { UserService } from '../services/user';

@Component({
  selector: 'app-navbar',
  imports: [RouterLink],
  templateUrl: './navbar.html',
  styleUrls: ['./navbar.css'],
})
export class Navbar {

  // Inyectamos el servicio de usuarios para poder acceder a la lógica
  // de autenticación sin implementarla directamente en este componente.
  constructor(private userService: UserService) { }

  // Método que se ejecuta al pulsar el botón de "Log out". Su única responsabi-
  // lidad
  // es delegar el cierre de sesión al UserService.
  logout(): void {
    this.userService.logout();
  }
}

```

6.6. Limpieza de tokens

Cuando las guardas de enrutamiento actúan, una de las operaciones que realizan de forma automática es la redirección entre rutas. Si este proceso no se controla cuidadosamente frente a posibles fallos de comunicación con el servidor, el funcionamiento de la aplicación puede volverse inestable.

Es necesario solucionar el problema de persistencia de datos que ocurre cuando la API devuelve un error (por ejemplo, si el token ha caducado o la sesión ya no es válida). Si el servidor rechaza la validación, la guarda bloqueará el acceso, pero si no se limpia explícitamente el token obsoleto de la memoria del navegador, el sistema conservará unas credenciales inválidas. Esto provoca peticiones redundantes y comportamientos erróneos al intentar reevaluar el estado de la sesión, derivando en un bucle de comprobaciones fallidas entre las guardas y la API.

Para prevenir esta situación, es indispensable interceptar el error devuelto por la API en el momento en que se intenta validar o renovar la sesión, garantizando la eliminación inmediata de los tokens del almacenamiento local.

6.6.1. Validación en UserService

Deberemos modificar el método encargado de consultar el estado de la sesión contra el servidor (renewToken), incorporando el operador catchError de RxJS en la pipe de la petición. De esta forma, si la solicitud HTTP fracasa, el servicio actúa como primera barrera defensiva limpiando la memoria antes de propagar el error a las guardas.

Debemos modificar los imports de rxjs de forma que queden así:

```
import { interval, Subscription, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';
```

Y modificaremos la función de renewToken() de forma que quedará con el siguiente código:

```
renewToken() {
  const tokenR = this.getRefreshToken();

  if (!tokenR) {
    return throwError(() => new Error('No hay token guardado en la memoria'));
  }

  const headers = new HttpHeaders().set('x-token', tokenR);

  return this.http.get(`${this.baseUrl}/login/renew`, { headers }).pipe(
    catchError(err => {
      // Si la API devuelve un error, se fuerza la limpieza
      localStorage.removeItem('token');
      localStorage.removeItem('token_r');

      // Se detiene el temporizador si estuviera en marcha
      this.stopTokenRenewalTimer();

      // Se propaga el error para que las guardas lo capturen y redirijan
      return throwError(() => err);
    })
  );
}
```

6.7. Errores claros al usuario

Nuestro formulario de inicio de sesión ahora mismo tiene dos situaciones que afectan negativamente tanto al rendimiento de la aplicación como a la experiencia del usuario. La primera de ellas se produce cuando el usuario intenta iniciar sesión con un formulario que no supera las validaciones del lado cliente, es decir, que le falta algún campo o que no cumple los requisitos. Aunque Angular muestra correctamente los mensajes de error correspondientes, la aplicación continúa ejecutando el resto del método e intenta realizar igualmente la petición al backend.

Para evitar este comportamiento, es necesario interrumpir la ejecución del método inmediatamente después de detectar que el formulario es inválido. Para ello, basta con incorporar una instrucción `return`, de forma que el proceso de autenticación finalice en ese mismo instante y no lleguemos a enviar ninguna solicitud innecesaria al backend.

Cabe resaltar que nosotros teníamos el bloque de código del login entero dentro de un `if`, normalmente lo que se suele hacer en estas tesituras es invertir el `if` y simplemente ponerlo antes del bloque de código, para que no se siga ejecutando la función si la característica no se cumple.

El método login debería quedar así:

```
login() {  
  // Registramos que el usuario ha intentado enviar el formulario  
  this.formSubmit = true;  
  
  // Si el formulario no es válido, detenemos la ejecución para evitar  
  // realizar una petición innecesaria al servidor.  
  console.log(this.loginForm.valid);  
  if (!this.loginForm.valid) {  
    console.log('Error: Campos incompletos o con formato incorrecto.');    return;  
  }  
  
  // Delegamos la acción al servicio, enviando el objeto con email y password  
  this.userService.login(this.loginForm.value).subscribe({  
    next: (response: any) => {  
      // Invocamos la función saveTokens de nuestro UserService para que esta gestione su almacenamiento  
      this.userService.saveTokens(response.token, response.token_r);  
  
      // Iniciamos el temporizador de renovación en segundo plano  
      this.userService.startTokenRenewalTimer();  
  
      // Redirigimos al usuario a la zona privada  
      this.router.navigateByUrl('/dashboard');  
    },  
    error: () => {  
      // Si se produce un error durante la autenticación, mostramos una alerta al usuario  
      Swal.fire({  
        icon: 'error',  
        title: 'Error',  
        text: 'Usuario o contraseña incorrectos',  
        confirmButtonText: 'OK',  
        confirmButtonColor: '#d33'  
      });  
    }  
  });  
}
```

```
}  
});  
}
```

Además de evitar el envío de peticiones innecesarias, también resulta conveniente mejorar la gestión de los errores que pueden producirse durante el proceso de autenticación. Ahora mismo, la aplicación nos muestra un único mensaje independientemente del origen del problema. Sin embargo, esto no nos permite diferenciar entre un error provocado por credenciales incorrectas y un fallo de comunicación con el servidor.

Para ofrecer una respuesta más adecuada en cada situación, vamos a aprovechar la información contenida en la respuesta HTTP. Si el servidor responde correctamente indicando que las credenciales son incorrectas, se muestra el mensaje enviado por la propia API mediante la propiedad `msg`. En cambio, cuando la petición no llega a completarse porque el backend no está disponible o existe un problema de conexión, dicha propiedad no existe, por lo que utilizaremos un mensaje alternativo informando al usuario de que la operación no ha podido completarse y que debe intentarlo de nuevo más tarde.

Esto lo implementaremos en el bloque error de la función `login()`:

```
error: (err) => {  
  // Si se produce un error durante la autenticación, mostramos una alerta al  
  usuario  
  Swal.fire({  
    icon: 'error',  
    title: 'Error',  
    text: err.error.msg || 'No pudo completarse la petición, por favor inténtelo más tarde',  
    confirmButtonText: 'OK',  
    confirmButtonColor: '#d33'  
  });  
}
```

Gracias a esta comprobación, la aplicación proporciona mensajes más precisos y comprensibles para el usuario final. Cuando el error se debe a unas credenciales incorrectas, muestra el mensaje específico devuelto por el backend. Por el contrario, si la petición no puede completarse por un problema de conectividad o porque el servidor no se encuentra disponible, presenta un mensaje genérico que mejora la experiencia de uso y no genera confusión al usuario.

7. Sidebar dinámico por roles

El sidebar es el menú lateral donde residirán todas las funcionalidades específicas de cada rol, por lo que tiene que poder cargarse de forma dinámica. Vamos a empezar a añadir funcionalidades a nuestra aplicación y lo primero que vamos a hacer es empezar a controlar ya la generación de esta zona del menú. Lo que tenemos que implementar es que el administrador pueda tener una pantalla para gestionar los usuarios registrados de la BD (que aun no hemos implementado) y pueda editar sus datos o borrarlos.

Para conseguir esto, vamos a utilizar el rol que nos devuelve el backend cuando estamos obteniendo el token.

7.1.1. Creación de SidebarService

Este nos servirá para poder centralizar la gestión del menú de la sidebar y para poder filtrarlo dinámicamente según el rol del usuario que acceda al dashboard. Crearemos un nuevo archivo `sidebar.ts` en la carpeta `services` y tendremos que sustituir su contenido por el siguiente código:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class SidebarService {
  // Variable pública que almacena el menú filtrado resultante
  public menu: any[] = [];

  // Método encargado de construir el menú según el perfil
  loadMenu(role: string) {
    const fullMenu = [
      {
        title: 'Dashboard',
        icon: 'mdi mdi-home',
        url: '/dashboard'
      },
      {
        title: 'Users',
        icon: 'mdi mdi-account-multiple',
        url: '/dashboard/users'
      }
    ];

    // Lógica de filtrado: se restringe el acceso a la gestión de usuarios
    if (role === 'ADMIN') {
      this.menu = fullMenu;
    } else {
      this.menu = fullMenu.filter(item => item.title !== 'Users');
    }
  }
}
```

7.1.2. Adaptación del componente Sidebar

Una vez definida la lógica, tendremos que actualizar el componente visual encargado de renderizar el menú. En el archivo **/commons/sidebar/sidebar.ts**, inyectaremos el SidebarService con visibilidad pública, permitiendo que la plantilla HTML utilice la variable menu de forma directa.

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
import { SidebarService } from '../services/sidebar';

@Component({
  selector: 'app-sidebar',
  imports: [RouterLink],
  templateUrl: './sidebar.html',
  styleUrls: ['./sidebar.css'],
})
export class Sidebar {
  // Inyección del servicio para exponer el menú filtrado a la vista
  constructor(public sidebarService: SidebarService) {}
}
```

En la vista HTML (sidebar.html), deberemos reemplazar los elementos de lista estáticos por el bloque de control de flujo **@for** nativo de Angular. Este bloque itera sobre el arreglo dinámico proporcionado por el servicio y genera de forma automática los enlaces permitidos.

7.1.3. Integración con el ciclo de autenticación

El último paso consiste en enlazar el SidebarService con el flujo de datos del usuario. Dado que el rol se obtiene al validar las credenciales contra el servidor, tendremos que modificar src/app/auth/login/login.ts.

Debemos inyectar el SidebarService en su constructor e invocar el método loadMenu() inmediatamente después de extraer de la petición el rol asignado al usuario, tanto en el inicio de sesión (login) como en la comprobación de la sesión en ngOnInit. Esto asegura que el menú se regenere correctamente con los permisos adecuados independientemente de cómo haya accedido el usuario a la plataforma.

Deberemos de añadir el import de SidebarService al principio del archivo:

```
// 1. Importa el servicio del sidebar
import { SidebarService } from '../services/sidebar';
```

Después lo inyectamos en el constructor de forma que nos quede así:

```
constructor(private fb: FormBuilder, private userService: UserService, private
router: Router, private sidebarService: SidebarService) {
  this.loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required]]
  });
}
```

Ahora añadimos la llamada a loadMenu en el bloque next de ngOnInit() de la siguiente forma:

```
next: (response: any) => {
  // Carga el menú si ya había una sesión válida
  this.sidebarService.loadMenu(response.user.role);
  // Si el token es válido, evitamos que vea el login y lo enviamos al
  dashboard
  this.router.navigateByUrl('/dashboard');
},
```

Y por último, lo añadiremos al bloque next de nuestro subscribe a la función login de userService, de forma que se ejecutará cuando haya un inicio de sesión exitoso:

```
.subscribe({
  next: (response: any) => {
    // Invocamos la función saveTokens de nuestro UserService para que esta gestione su almacenamiento
    this.userService.saveTokens(response.token, response.token_r);

    // Iniciamos el temporizador de renovación en segundo plano
    this.userService.startTokenRenewalTimer();

    // Carga el menú tras un inicio de sesión exitoso
    this.sidebarService.loadMenu(response.user.role);

    // Redirigimos al usuario a la zona privada
    this.router.navigateByUrl('/dashboard');
  },
```

De esta forma, si ahora mismo accedemos a nuestra aplicación e iniciamos sesión con un usuario de rol USER, deberíamos de ver esto:

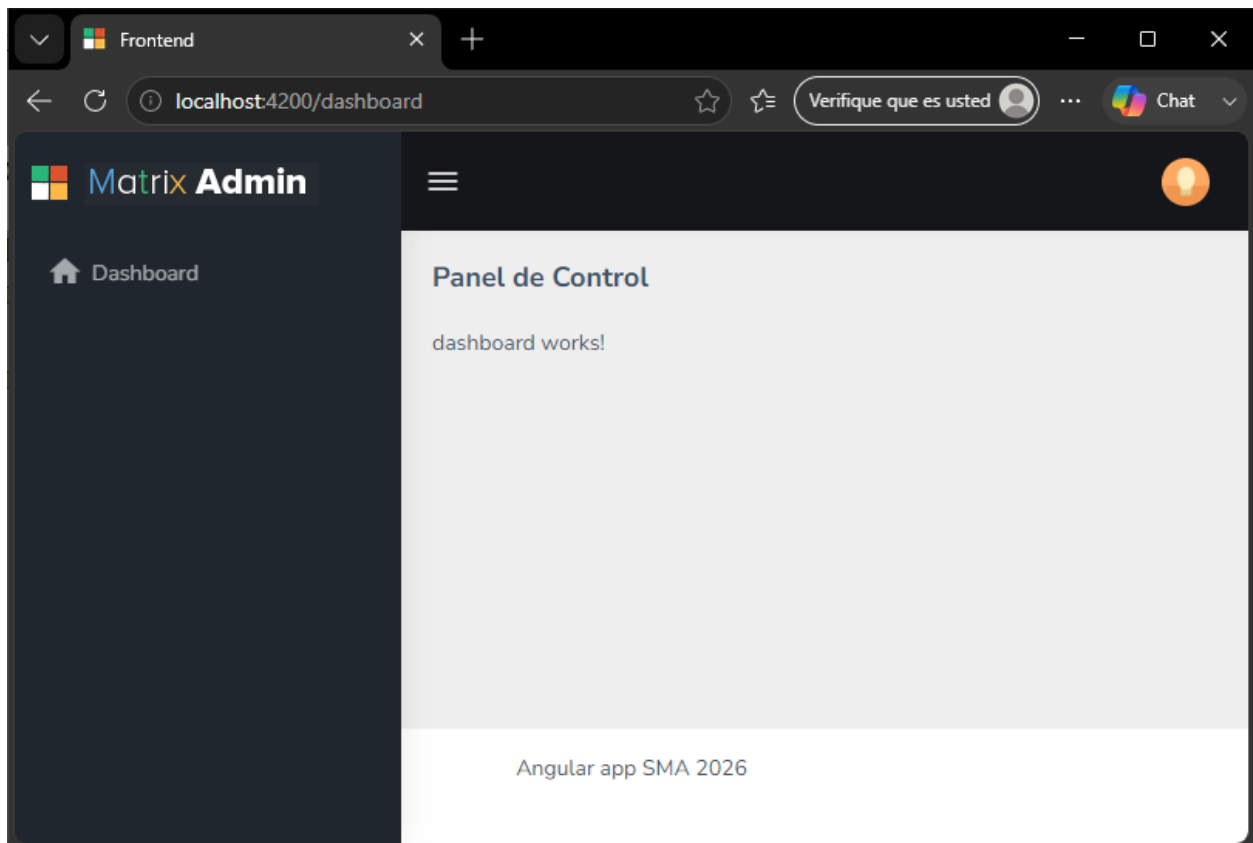


Figura 78. Dashboard principal.

En esta captura podemos observar que efectivamente no hay sección "Users" porque la hemos restringido para que solo la vean los administradores.

Nuestro problema ahora mismo es que si hacemos F5 desaparece todo el contenido de la sidebar, por lo que tenemos que importar el SidebarService a nuestro UserService, de la siguiente forma:

```
import { SidebarService } from './sidebar';
```

Luego lo añadiremos al constructor de forma que quedará así:

```
constructor(private http: HttpClient, private router: Router, private sidebar-
Service: SidebarService) { }
```

Ahora únicamente nos queda reconstruir el menú cuando la renovación del token finalice correctamente. Para ello utilizaremos el operador tap() de RxJS, que nos permite ejecutar código adicional cuando el observable recibe una respuesta, sin modificar dicha respuesta.

Dentro de este bloque llamaremos al método loadMenu(), pasándole el rol recibido desde el backend para volver a generar automáticamente las opciones del menú correspondientes al usuario autenticado. Este es el código que pondremos:

```
tap((response: any) => {
  // Cuando la renovación del token finaliza correctamente, utilizamos el rol
  // devuelto por el backend para reconstruir el menú dinámico con las opcio-
  nes
  // correspondientes al usuario autenticado.
  this.sidebarService.loadMenu(response.user.role);
}),
```

De esta forma, cada vez que el usuario recargue la aplicación y el token se renueve correctamente, el menú

lateral volverá a construirse automáticamente, evitando que desaparezcan las opciones de navegación.

7.2. Página de gestión de usuarios

Lo siguiente que vamos a implementar es la pantalla de gestión de usuarios para que el administrador pueda realizar las operaciones requeridas en el sistema. Esta pantalla mostrará una lista de usuarios que incluirá su nombre, email, contraseña, fecha de alta y token. Además, permitirá buscar registros por nombre o email, ordenar el contenido pulsando sobre las cabeceras y estará paginada mostrando los resultados de 10 en 10. También incluiremos las acciones correspondientes para crear un nuevo registro, editar los datos generales o actualizar de forma segura la contraseña del usuario.

7.2.1. Adaptación del Backend: Búsqueda, ordenación y contraseñas

Antes de trabajar con la interfaz, debemos preparar nuestro backend para soportar la búsqueda, la ordenación por columnas, el cálculo exacto del total de registros para el paginador y una ruta independiente para el cambio de contraseñas.

En nuestro archivo **models/users.js**, añadiremos las funciones para obtener los usuarios con filtros y actualizar la contraseña:

```
// Obtiene los usuarios aplicando filtros de búsqueda y ordenación dinámica
const getUsersWithToken = async (from = 0, limit = 10, search = '', sort = 'id',
order = 'ASC') => {
  const db = getDb();

  // Definimos una lista blanca de columnas permitidas para ordenar.
  // Esto es vital por seguridad, para evitar ataques de inyección SQL.
  const validSortFields = ['id', 'name', 'email', 'reg_date'];
  const validOrders = ['ASC', 'DESC'];

  // Si el campo solicitado no está en la lista blanca, forzamos la ordenación
  // por 'id'.
  const finalSort = validSortFields.includes(sort) ? sort : 'id';
  // Nos aseguramos de que el orden sea estrictamente 'ASC' o 'DESC'.
  const finalOrder = validOrders.includes(order.toUpperCase()) ? order.toUp-
perCase() : 'ASC';

  // Preparamos el término de búsqueda añadiendo los comodines '%' de SQL
  // para que busque coincidencias parciales en cualquier parte del texto.
  const searchQuery = `>${search}<`;

  // Ejecutamos la consulta a la base de datos aplicando los parámetros
  const users = await db.all(
    `SELECT id, name, email, role, reg_date, token
    FROM users
    WHERE name LIKE ? OR email LIKE ?
    ORDER BY ${finalSort} ${finalOrder}
    LIMIT ? OFFSET ?`,
    [searchQuery, searchQuery, limit, from]
  );

  // Mapeamos los resultados para ocultar la contraseña antes de enviarlos.
  // Utilizamos la desestructuración para separar 'password' del resto de pro-
  // piedades.
  return users.map(user => {
    const { password, ...rest } = user;
    return rest;
  });
}
```

```

    });
};

// Cuenta el total de usuarios aplicando el mismo filtro de búsqueda que la ta-
bla.
// Esto es necesario para que el paginador sepa cuántas páginas hay en total.
const countUsers = async (search = '') => {
    const db = getDb();
    const searchQuery = `_${search}%`;
    return await db.get(
        "SELECT COUNT(*) as total FROM users WHERE name LIKE ? OR email LIKE ?",
        [searchQuery, searchQuery]
    );
};

// Obtiene el usuario incluyendo su contraseña.
// Es necesaria para procesos de validación interna, ya que getUserById oculta
la contraseña.
const getUserByIdWithPassword = async (id) => {
    const db = getDb();
    return await db.get("SELECT * FROM users WHERE id = ?", [id]);
};

// Actualiza únicamente la contraseña del usuario en la base de datos.
const updateUserPassword = async (id, password) => {
    const db = getDb();
    await db.run("UPDATE users SET password = ? WHERE id = ?", [password, id]);
    const user = await getUserById(id);
    return formatUser(user);
};

```

A continuación, en **controllers/users.js**, actualizaremos el método `getUsers` para extraer los parámetros de la URL y crearemos el método `updateUserPassword`:

```

const getUsers = async (req, res) => {
    try {
        await User.initUsersTable();

        // Extraemos los parámetros de la URL enviados mediante query strings
        (?from=0&search=...).
        // Si no existen, asignamos valores por defecto para evitar errores.
        const from = Number(req.query.from) || 0;
        const regPerPage = 10;
        const search = req.query.search || '';
        const sort = req.query.sort || 'id';
        const order = req.query.order || 'ASC';

        // Ejecutamos ambas consultas de forma simultánea con Promise.all
        // para reducir el tiempo de espera del servidor.
        const [ users, countResult ] = await Promise.all([
            User.getUsersWithToken(from, regPerPage, search, sort, order),
            User.countUsers(search)
        ]);

        // Extraemos el número directamente de la propiedad 'total' de la con-
        sulta de conteo.
        // Si no hay resultados, asignamos 0.
        const totalRecords = countResult.total || 0;

        // Devolvemos la respuesta al frontend con la información estructurada
        res.json({
            ok: true,
            msg: "getUsers",
            users,

```

```

        page: {
            from,
            regPerPage,
            total: totalRecords
        }
    });

    } catch (error) {
        console.log(error);
        return res.status(500).json({ ok: false, msg: "Error obteniendo usuarios" });
    }
};

// Función encargada de validar y cambiar la contraseña de un usuario
const updateUserPassword = async (req, res) => {
    // Obtenemos el ID de la ruta (/api/users/:id/password)
    const { id } = req.params;
    // Obtenemos las contraseñas enviadas en el cuerpo de la petición
    const { oldPassword, newPassword } = req.body;

    try {
        // Recuperamos al usuario con su contraseña cifrada para poder compararla
        const user = await User.getUserByIdWithPassword(id);
        if (!user) {
            return res.status(404).json({ ok: false, msg: "No existe el usuario" });
        }

        // Comparamos la contraseña antigua introducida por el usuario con el hash de la base de datos
        const validPassword = bcrypt.compareSync(oldPassword, user.password);
        if (!validPassword) {
            return res.status(400).json({ ok: false, msg: "La contraseña antigua es incorrecta" });
        }

        // Si la contraseña antigua es correcta, generamos un nuevo salt y ciframos la nueva contraseña
        const salt = bcrypt.genSaltSync();
        const cpassword = bcrypt.hashSync(newPassword, salt);

        // Guardamos la nueva contraseña cifrada en la base de datos
        await User.updateUserPassword(id, cpassword);

        res.json({ ok: true, msg: "Contraseña actualizada" });

    } catch (error) {
        console.log(error);
        return res.status(500).json({ ok: false, msg: "Error actualizando la contraseña" });
    }
};

```

Y en **routes/users.js**, añadiremos la ruta correspondiente verificando que se envían ambas contraseñas:

```

// Ruta PUT específica para la actualización de contraseñas.
// Emplea los middlewares de validación JWT y verificación de campos obligatorios.
router.put('/:id/password', [
    validateJWT,
    check('oldPassword', 'La contraseña antigua es obligatoria').not().isEmpty(),
],

```

```

    check('newPassword', 'La nueva contraseña es obligatoria').not().isEmpty(),
    check('id', 'El identificador no es válido').isInt(),
    validateFields,
    validateRole
  ], updateUserPassword);

```

7.2.2.7.2. Ampliación del servicio UserService

Ahora vamos a preparar la comunicación con nuestro backend desde Angular. Tenemos que añadir los métodos que realizarán las peticiones HTTP a nuestro archivo **src/app/services/user.ts**.

Añadiremos el siguiente bloque de código al final de nuestra clase UserService para gestionar la obtención, creación, actualización y borrado de usuarios:

```

// Obtener lista de usuarios con parámetros de búsqueda, orden y paginación
getUsers(from: number = 0, search: string = '', sortField: string = 'id',
sortOrder: string = 'ASC') {
    // Recuperamos el token de la memoria local para identificarnos ante el ser-
    vidor
    const token = this.getToken();
    // Lo asignamos a la cabecera 'x-token'
    const headers = new HttpHeaders().set('x-token', token);

    // Construimos la URL adjuntando todas las variables como parámetros de con-
    sulta (query params)
    const url = `${this.ba-
seUrl}/users?from=${from}&search=${search}&sort=${sortField}&order=${sortOr-
der}`;

    // Realizamos la petición HTTP GET
    return this.http.get(url, { headers });
}

// Crear un nuevo usuario
createUser(userData: any) {
    const token = this.getToken();
    const headers = new HttpHeaders().set('x-token', token);
    // Realizamos una petición POST enviando los datos del formulario en el
    cuerpo de la petición
    return this.http.post(`${this.baseUrl}/users`, userData, { headers });
}

// Actualizar los datos generales de un usuario existente
updateUser(id: string | number, userData: any) {
    const token = this.getToken();
    const headers = new HttpHeaders().set('x-token', token);
    // Realizamos una petición PUT indicando el ID en la URL y enviando los nue-
    vos datos
    return this.http.put(`${this.baseUrl}/users/${id}`, userData, { headers });
}

// Actualizar exclusivamente la contraseña de un usuario
updateUserPassword(id: string | number, passwordData: any) {
    const token = this.getToken();
    const headers = new HttpHeaders().set('x-token', token);
    // Realizamos una petición PUT a la ruta específica de contraseñas
    return this.http.put(`${this.baseUrl}/users/${id}/password`, passwordData,
    { headers });
}

// Eliminar un usuario
deleteUser(id: string | number) {

```

```
const token = this.getToken();
const headers = new HttpHeaders().set('x-token', token);
// Realizamos una petición DELETE indicando el ID del usuario a borrar
return this.http.delete(`${this.baseUrl}/users/${id}`, { headers });
}
```

7.2.3.7.3. Creación del componente paginador

Crearemos un nuevo archivo en `src/app/commons/paginator/paginator.ts`:

ng g c commons/paginator

Este componente recibirá el total de elementos y la página actual, calculará el límite y emitirá un evento cuando el administrador cambie de página:

```
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({
  selector: 'app-paginator',
  imports: [],
  templateUrl: './paginator.html',
  styleUrls: ['./paginator.css']
})
export class Paginator {
  // Propiedades de entrada que recibe desde el componente padre (Users)
  @Input() totalItems: number = 0; // Cantidad total de registros en la BD
  @Input() pageSize: number = 10; // Cantidad de registros por página
  @Input() currentPage: number = 1; // Página en la que nos encontramos actualmente

  // Propiedad de salida para notificar al padre cuando el usuario cambia de página
  @Output() pageChange = new EventEmitter<number>();

  // Calcula el número total de páginas dividiendo el total de items entre el tamaño de página
  // Math.ceil redondea hacia arriba (ej: 15 items / 10 = 1.5 -> 2 páginas)
  totalPages(): number {
    return Math.ceil(this.totalItems / this.pageSize);
  }

  // Función ejecutada al hacer clic en Anterior o Siguiente
  changePage(page: number) {
    // Comprobamos que la página solicitada esté dentro de los límites válidos
    // y que no sea la misma en la que ya estamos.
    if (page >= 1 && page <= this.totalPages() && page !== this.currentPage) {
      // Emitimos el número de la nueva página hacia el componente padre
      this.pageChange.emit(page);
    }
  }
}
```

En su archivo de vista (`paginator.html`) renderizaremos los botones de "Previous" y "Next", controlando las clases de validación:

```
<nav aria-label="Navegación de páginas">
  <ul class="pagination justify-content-center">
    <!-- Botón Anterior: Se desactiva si estamos en la primera página -->
    <li class="page-item" [class.disabled]="currentPage === 1">
      <a class="page-link" href="javascript:void(0)" (click)="changePage(currentPage - 1)">Previous</a>
    </li>
```

```

    <!-- Indicador de página actual: Muestra la información matemática -->
    <li class="page-item disabled">
      <span class="page-link text-dark">Page {{ currentPage }} of {{ totalPages() }}</span>
    </li>

    <!-- Botón Siguiente: Se desactiva si estamos en la última página o no hay registros -->
    <li class="page-item" [class.disabled]="currentPage === totalPages() || totalPages() === 0">
      <a class="page-link" href="javascript:void(0)" (click)="changePage(currentPage + 1)">Next</a>
    </li>
  </ul>
</nav>

```

7.2.4.7.4. Componente de formulario (UserForm)

Para las acciones de insertar y modificar información, usaremos el mismo componente, diferenciando mediante variables si está abriéndose para editar datos generales, crear uno nuevo o modificar exclusivamente la contraseña.

Crearemos el archivo **src/app/pages/users/user-form/user-form.ts**:

ng g c pages/users/user-form

La lógica principal de los formularios reactivos reside en su inicio (ngOnInit). Modificaremos dinámicamente los controles para adaptarnos al escenario.

```

import { Component, EventEmitter, Input, OnInit, Output } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  imports: [ReactiveFormsModule],
  templateUrl: './user-form.html',
  styleUrls: ['./user-form.css']
})
export class UserForm implements OnInit {
  // Variables de entrada para determinar el modo del formulario
  @Input() user: any = null; // Si contiene datos, estamos editando. Si es null, creando.
  @Input() editPasswordOnly: boolean = false; // Flag para el modo de cambio de contraseña

  // Eventos de salida para comunicarnos con el padre
  @Output() save = new EventEmitter<any>();
  @Output() cancel = new EventEmitter<void>();

  // Declaración del grupo del formulario reactivo
  public userForm: FormGroup;

  constructor(private fb: FormBuilder) {
    // Definimos la estructura base inicial con todos los campos posibles
    this.userForm = this.fb.group({
      name: ['', Validators.required],
      email: ['', [Validators.required, Validators.email]],
      password: [''],
      role: ['USER', Validators.required]
    });
  }
}

```

```

ngOnInit(): void {
  if (this.editPasswordOnly) {
    // MODO 1: Cambio exclusivo de contraseña
    // Eliminamos los controles de datos generales para que no den error de
validación
    this.userForm.removeControl('name');
    this.userForm.removeControl('email');
    this.userForm.removeControl('role');
    this.userForm.removeControl('password'); // Eliminamos el password gené-
rico

    // Añadimos los controles específicos requeridos por nuestra API para esta
acción
    this.userForm.addControl('oldPassword', this.fb.control('', [Valida-
tors.required]));
    this.userForm.addControl('newPassword', this.fb.control('', [Valida-
tors.required]));

  } else if (this.user) {
    // MODO 2: Edición de datos generales
    // Rellenamos el formulario con los datos existentes del usuario
    this.userForm.patchValue({
      name: this.user.name,
      email: this.user.email,
      role: this.user.role || 'USER'
    });
    // Eliminamos el control de la contraseña porque no se modifica desde aquí
    this.userForm.removeControl('password');

  } else {
    // MODO 3: Creación de un nuevo usuario
    // Mantenemos la estructura base, pero hacemos que el campo de contraseña
sea obligatorio
    this.userForm.get('password')?.setValidators([Validators.required]);
    this.userForm.get('password')?.updateValueAndValidity();
  }
}

onSubmit() {
  // Si todos los campos cumplen las validaciones, emitimos el valor del for-
mulario
  if (this.userForm.valid) {
    this.save.emit(this.userForm.value);
  }
}

onCancel() {
  // Emitimos el evento de cancelación para que el padre cierre la vista
  this.cancel.emit();
}
}

```

La vista user-form.html utilizará condicionales de Angular (@if) para renderizar únicamente los campos que correspondan al estado del componente:

```

<div class="card">
  <div class="card-body">
    <!-- Título dinámico basado en las variables de estado -->
    <h4 class="card-title">
      @if (editPasswordOnly) { Change {{ user?.name }} password }
      @else if (user) { Edit user }
      @else { New user }
    </h4>
  </div>
</div>

```

```

    <form class="form-horizontal mt-4" [formGroup]="userForm" (ngSubmit)="onSubmit()">

        <!-- Mostramos campos de datos si NO estamos en modo contraseña -->
        @if (!editPasswordOnly) {
            <div class="form-group">
                <label>Name</label>
                <input type="text" class="form-control" formCon-
trolName="name">
            </div>
            <div class="form-group">
                <label>Email</label>
                <input type="email" class="form-control" formCon-
trolName="email">
            </div>
            <div class="form-group">
                <label>Role</label>
                <select class="form-control" formControlName="role">
                    <option value="USER">User</option>
                    <option value="ADMIN">Administrator</option>
                </select>
            </div>
        }

        <!-- Mostramos contraseña genérica SÓLO si es un usuario nuevo -->
        @if (!user && !editPasswordOnly) {
            <div class="form-group">
                <label>Password</label>
                <input type="password" class="form-control" formCon-
trolName="password">
            </div>
        }

        <!-- Mostramos los campos dobles SÓLO si estamos en modo cambio de
        contraseña -->
        @if (editPasswordOnly) {
            <div class="form-group">
                <label>Current password</label>
                <input type="password" class="form-control" formCon-
trolName="oldPassword">
            </div>
            <div class="form-group">
                <label>New password</label>
                <input type="password" class="form-control" formCon-
trolName="newPassword">
            </div>
        }

        <!-- Botones de acción del formulario -->
        <div class="form-actions border-top pt-3 mt-4">
            <!-- El botón de guardar se desactiva automáticamente si el for-
            mulario es inválido -->
            <button type="submit" class="btn btn-success text-white mr-2"
[disabled]="userForm.invalid">Save</button>
            <button type="button" class="btn btn-dark" (click)="onCan-
cel()">Cancel</button>
        </div>
    </form>
</div>
</div>

```

7.2.5.7.5. Integración en la vista principal de Users

El último paso consiste en enlazar todas estas piezas en la pantalla principal. Abriremos nuestro archivo **src/app/pages/users/users.ts**. Este componente coordinará todas las acciones, controlará los parámetros y forzará la detección de cambios para evitar fallos visuales.

```
import { Component, OnInit, ChangeDetectorRef } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { UserService } from '../../../services/user';
import { Paginator } from '../../../commons/paginator/paginator';
import { UserForm } from './user-form/user-form';
import Swal from 'sweetalert2';

@Component({
  selector: 'app-users',
  imports: [FormsModule, Paginator, UserForm],
  templateUrl: './users.html',
  styleUrls: ['./users.css']
})
export class Users implements OnInit {
  // Variables para la respuesta de la base de datos
  public users: any[] = [];
  public totalUsers: number = 0;

  // Variables de control de paginación y búsqueda
  public from: number = 0;
  public limit: number = 10;
  public currentPage: number = 1;
  public searchQuery: string = '';
  public sortField: string = '';
  public sortOrder: string = '';

  // Variables de control de la vista para alternar componentes
  public showForm: boolean = false;
  public selectedUser: any = null;
  public editingPassword: boolean = false;

  // Inyectamos el servicio de usuarios y el detector de cambios
  constructor(private userService: UserService, private cdr: ChangeDetectorRef) {}

  ngOnInit(): void {
    // Cargamos los usuarios al iniciar el componente
    this.loadUsers();
  }

  // Método principal de consulta de datos
  loadUsers() {
    // Enviamos todas las variables de estado al servicio
    this.userService getUsers(this.from, this.searchQuery, this.sortField,
this.sortOrder).subscribe({
      next: (res: any) => {
        // Almacenamos los resultados devueltos por el backend
        this.users = res.users;
        this.totalUsers = res.page.total;

        // Forzamos la actualización de la vista HTML para evitar retrasos vi-
suales
        // ocasionados por el ciclo de vida de Angular.
        this.cdr.detectChanges();
      },
      error: (err) => console.error(err)
    });
  }
}
```

```
});  
}  
  
// Método ejecutado al buscar texto  
searchUsers() {  
    this.from = 0; // Reiniciamos la paginación a cero  
    this.currentPage = 1;  
    this.loadUsers(); // Ejecutamos la carga con el nuevo texto almacenado  
}  
  
// Método ejecutado al hacer clic en las cabeceras de la tabla  
sortBy(field: string) {  
    if (this.sortField === field) {  
        // Si hacemos clic en la misma columna, invertimos el orden  
        this.sortOrder = this.sortOrder === 'ASC' ? 'DESC' : 'ASC';  
    } else {  
        // Si cambiamos de columna, establecemos el nuevo campo y orden ascendente  
        this.sortField = field;  
        this.sortOrder = 'ASC';  
    }  
    this.from = 0; // Reiniciamos la paginación a cero tras reordenar  
    this.currentPage = 1;  
    this.loadUsers();  
}  
  
// Método disparado por el componente Paginator  
onPageChange(page: number) {  
    this.currentPage = page;  
    // Calculamos el desfase (offset) matemático multiplicando la página anterior por el límite  
    this.from = (page - 1) * this.limit;  
    this.loadUsers();  
}  
  
// Funciones de apertura y cierre de formularios modificando las variables de estado  
openCreateForm() {  
    this.selectedUser = null;  
    this.editingPassword = false;  
    this.showForm = true;  
}  
  
openEditForm(user: any) {  
    this.selectedUser = user;  
    this.editingPassword = false;  
    this.showForm = true;  
}  
  
openPasswordForm(user: any) {  
    this.selectedUser = user;  
    this.editingPassword = true;  
    this.showForm = true;  
}  
  
hideUserForm() {  
    this.showForm = false;  
    this.selectedUser = null;  
    this.editingPassword = false;  
}  
  
// Recepción del evento de guardado emitido por el formulario  
saveUser(formData: any) {  
    if (this.editingPassword) {  
        // Llamada a la ruta PUT de contraseñas
```

```

        this.userService.updateUserPassword(this.selectedUser.id, formData).subscribe({
      next: () => {
        Swal.fire('Actualizada', 'Contraseña actualizada correctamente', 'success');
        this.hideUserForm();
      },
      error: (err) => Swal.fire('Error', err.error.msg, 'error')
    });
  } else if (this.selectedUser) {
    // Llamada a la ruta PUT de datos generales
    this.userService.updateUser(this.selectedUser.id, formData).subscribe({
      next: () => {
        Swal.fire('Actualizado', 'Usuario actualizado correctamente', 'success');
        this.hideUserForm();
        this.loadUsers();
      },
      error: (err) => Swal.fire('Error', err.error.msg, 'error')
    });
  } else {
    // Llamada a la ruta POST de creación
    this.userService.createUser(formData).subscribe({
      next: () => {
        Swal.fire('Creado', 'Usuario creado correctamente', 'success');
        this.hideUserForm();
        this.loadUsers();
      },
      error: (err) => Swal.fire('Error', err.error.msg, 'error')
    });
  }
}

// Método de eliminación con cuadro de confirmación
deleteUser(id: string | number) {
  Swal.fire({
    title: '¿Estás seguro?',
    text: "Esta acción eliminará permanentemente al usuario.",
    icon: 'warning',
    showCancelButton: true,
    confirmButtonColor: '#3085d6',
    cancelButtonColor: '#d33',
    confirmButtonText: 'Sí, borrar',
    cancelButtonText: 'Cancelar'
  }).then((result) => {
    // Si el usuario acepta la alerta, procedemos con el borrado
    if (result.isConfirmed) {
      this.userService.deleteUser(id).subscribe({
        next: () => {
          Swal.fire('Borrado', 'El usuario ha sido eliminado.', 'success');

          // Lógica de corrección visual: si borramos el único registro que
          quedaba
          // en la última página, forzamos un retroceso de página.
          if (this.users.length === 1 && this.currentPage > 1) {
            this.from -= this.limit;
            this.currentPage -= 1;
          }
          this.loadUsers(); // Recargamos la tabla para confirmar los cambios
        },
        error: (err) => Swal.fire('Error', err.error.msg, 'error')
      });
    }
  });
}

```

```

    }
  }
}

```

En la vista correspondiente (users.html), utilizaremos el bloque condicional nativo de Angular y enlazaremos los métodos a los elementos del buscador y de la tabla.

```

<div class="row">
  <div class="col-12">

    <!-- Bloque condicional: Si showForm es verdadero, cargamos el componente
del formulario -->
    @if (showForm) {
      <app-user-form
        [user]="selectedUser"
        [editPasswordOnly]="editingPassword"
        (save)="saveUser($event)"
        (cancel)="hideUserForm()">
      </app-user-form>

      <!-- Si showForm es falso, cargamos la lista de usuarios y herramientas -->
    } @else {
      <div class="card">
        <div class="card-body">
          <h4 class="card-title">User management</h4>

          <div class="d-flex justify-content-between mb-3 mt-4">

            <!-- Botón de apertura de formulario en modo creación -->
            <button class="btn btn-primary" (click)="openCreateForm()">
              <i class="mdi mdi-plus"></i> New user
            </button>

            <!-- Elementos de búsqueda de texto interactiva -->
            <div class="input-group w-25">
              <!-- [(ngModel)] asocia bidireccionalmente el valor del input con
la variable de clase -->
              <!-- (keyup.enter) detecta la pulsación de la tecla Intro -->
              <input type="text" class="form-control" placeholder="Search by
name or email..." [(ngModel)]="searchQuery" (keyup.enter)="searchUsers()">
              <div class="input-group-append">
                <!-- Botón manual para lanzar la búsqueda -->
                <button class="btn btn-outline-secondary" type="button"
(click)="searchUsers()">
                  <i class="mdi mdi-magnify"></i>
                </button>
              </div>
            </div>
          </div>

          <div class="table-responsive">
            <table class="table table-striped table-bordered">
              <thead>
                <tr>
                  <!-- Cabeceras interactivas con llamadas a la función de orde-
nación sortBy -->
                  <th style="cursor:pointer" (click)="sortBy('name')">Name <i
class="mdi mdi-sort"></i></th>
                  <th style="cursor:pointer" (click)="sortBy('email')">Email <i
class="mdi mdi-sort"></i></th>
                  <th>Password</th>
                  <th style="cursor:pointer" (click)="sortBy('reg_date')">Regis-
tration date <i class="mdi mdi-sort"></i></th>
                  <th>Token</th>
                  <th class="text-center">Actions</th>
                </tr>
              </thead>
            </table>
          </div>
        </div>
      </div>
    }
  </div>
</div>

```

```

        </tr>
    </thead>
    <tbody>
        <!-- Bucle @for nativo de Angular para iterar sobre la lista de
usuarios -->
        @for (user of users; track user.id) {
            <tr>
                <td>{{ user.name }}</td>
                <td>{{ user.email }}</td>
                <td>*****</td> <!-- Ocultamos la contraseña real en la vi-
sualización -->
                <td>{{ user.reg_date || 'N/A' }}</td>

                <!-- Elemento truncado visualmente si el token es muy largo
-->
                <td class="text-truncate" style="max-width: 150px;" ti-
tle="{{ user.token }}">
                    {{ user.token || 'No active session' }}
                </td>
                <td class="text-center">
                    <!-- Botón de edición de datos -->
                    <button class="btn btn-sm btn-info mr-1"
(click)="openEditForm(user)">
                        <i class="mdi mdi-pencil"></i>
                    </button>
                    <!-- Botón de edición de contraseña -->
                    <button class="btn btn-sm btn-warning mr-1"
(click)="openPasswordForm(user)">
                        <i class="mdi mdi-key"></i>
                    </button>
                    <!-- Botón de eliminación -->
                    <button class="btn btn-sm btn-danger"
(click)="deleteUser(user.id)">
                        <i class="mdi mdi-delete"></i>
                    </button>
                </td>
            </tr>
        }
        <!-- Estado vacío por si la base de datos o la búsqueda no de-
vuelven resultados -->
        @empty {
            <tr>
                <td colspan="6" class="text-center py-4">There are no users
registered in the system</td>
            </tr>
        }
    </tbody>
</table>
</div>

<!-- Incrustamos el paginador genérico pasándole las variables de con-
trol -->
<app-paginator
    [totalItems]="totalUsers"
    [pageSize]="limit"
    [currentPage]="currentPage"
    (pageChange)="onPageChange($event)">
</app-paginator>

</div>
</div>
}
</div>
</div>

```

8. Mensajes dinámicos en el dashboard

Vamos a trabajar con la personalización de la pantalla de inicio (Dashboard). El objetivo es que la vista muestre un mensaje de bienvenida dinámico, indicando "Welcome USER" o "Welcome ADMIN" según el rol de la persona autenticada.

Para lograr esto sin necesidad de realizar nuevas peticiones al servidor, aprovecharemos que el token JWT que guardamos en la memoria local durante el login ya contiene esta información. Un token JWT estándar está compuesto por tres partes separadas por puntos, siendo la segunda parte (el payload) la que contiene los datos del usuario.

Modificaremos el controlador de nuestro componente en el archivo **src/app/pages/dashboard/dashboard.ts**. Extraeremos el token a través del UserService, decodificaremos la segunda parte utilizando la función nativa `atob()` de JavaScript y guardaremos el rol en una variable pública:

```
import { Component, OnInit } from '@angular/core';
import { UserService } from '../services/user';

@Component({
  selector: 'app-dashboard',
  imports: [],
  templateUrl: './dashboard.html',
  styleUrls: ['./dashboard.css'],
})
export class Dashboard implements OnInit {
  // Variable pública para mostrar el rol en la vista HTML
  public role: string = '';

  // Inyectamos el servicio para tener acceso a los métodos de lectura del token
  constructor(private userService: UserService) {}

  ngOnInit(): void {
    const token = this.userService.getToken();

    if (token) {
      // Extraemos el cuerpo (payload) del JWT, que es la segunda parte separada
      // por un punto
      const payload = token.split('.')[1];

      // Decodificamos el texto en Base64 y lo convertimos a un objeto JSON evaluable
      const decoded = JSON.parse(atob(payload));

      // Asignamos el rol resultante a la variable de la vista
      this.role = decoded.role;
    }
  }
}
```

Una vez procesado el rol en la lógica del componente, actualizaremos el archivo **src/app/pages/dashboard/dashboard.html** para renderizar el mensaje en la pantalla empleando la interpolación de Angular. Sustituiremos el contenido por este:

```
<div class="row mt-5">
  <div class="col-12 text-center mt-5">
    <h1 class="display-4 text-uppercase">Welcome {{ role }}</h1>
  </div>
</div>
```

Si has encontrado algún error en tu aplicación y no sabes como resolverlo, accede al siguiente enlace para obtener el .zip del proyecto original.

8.1. Descarga Código 10

Todo lo desarrollado hasta ahora está disponible en el “Codigo10” que puedes descargar desde aquí:

[Descargar Código 10](https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo10.zip)

Link de descarga: <https://github.com/jvberna/SMA-UA/blob/main/AppDevelop/examples/codigo10.zip>